

Lab 1

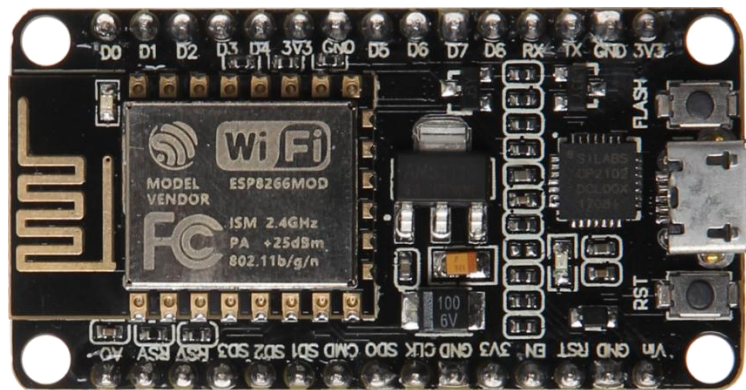
Introduction to NodeMCU

1. NodeMCU (ESP8266) Microcontroller

NodeMCU เป็นบอร์ดไมโครคอนโทรลเลอร์ที่ใช้เฟิร์มแวร์โอเพนซอร์สของ Lua สำหรับชิป ESP32 และ ESP8266 ซึ่งเป็นชิปที่มีระบบการเชื่อมต่อ WiFi ผังบนบอร์ด (System-on-Chip: SoC) ทำให้สามารถเชื่อมต่อกับเครือข่ายได้ง่าย งานได้หลากหลาย มีขนาดเล็ก และมีราคาถูก สามารถพัฒนาโปรแกรมได้โดยใช้ภาษาซี (C language) ผ่าน Arduino IDE [1] โดยในการเรียนวิชานี้จะใช้ชิป ESP8266 ในการเรียนการสอน

1.1 ส่วนประกอบของบอร์ด NodeMCU

บอร์ด NodeMCU ESP8266 จะมีองค์ประกอบดังที่แสดงในรูปต่อไปนี้ [2]



รูปที่ 1.1 NodeMCU ESP8266

ที่มา: <https://joy-it.net/en/products/SBC-NodeMCU>

- Digital Pin: เป็นขาดิจิทัล มีทั้งหมด 9 ขา ตั้งแต่ D0 – D8 โดยทำหน้าที่เป็น GPIO คือ สามารถกำหนดให้เป็นขาอินพุตหรือเอาต์พุตก็ได้ นอกจากนี้ยังสามารถทำงานเฉพาะได้สำหรับบางขา คือ
 - D0: Wake pin ใช้สำหรับกระตุ้นให้ชิปทำงานจากสถานะ deep-sleep
 - D1: SCL pin ใช้ในการสื่อสารแบบ I2C ทั้งโหมด Master และ Slave สามารถทำงานสูงสุดที่ความถี่ 100 kHz
 - D2: SDA pin ใช้ในการสื่อสารแบบ I2C ทั้งโหมด Master และ Slave สามารถทำงานสูงสุดที่ความถี่ 100 kHz
 - D5-D8: สามารถใช้ในการเชื่อมต่อแบบ SPI และ HSPI ทั้งโหมด Master และ Slave รองรับการทำงาน timing ทั้ง 4 โหมดของ SPI format transfer ที่ความถี่สูงสุด 80 MHz และการทำงานแบบ FIFO สูงสุดที่ 64 bytes

- Analog Pin: เป็นขาแอนาล็อก มีเพียง 1 ขา คือ A0 เป็นอินพุตที่สามารถอ่านค่าจากเซ็นเซอร์ และทำหน้าที่เป็น Analog to Digital Converter (ADC) แบบ 10-bit เพื่อแปลงสัญญาณแอนาล็อกจากเซ็นเซอร์มาเป็นสัญญาณดิจิทัลเพื่อใช้ในการประมวลผลในคอนโทรลเลอร์
- Power Pin: บนบอร์ดจะประกอบไปด้วยขา Power ทั้งหมด 4 ขา ดังนี้
 - V_{in} : ใช้ในการรับไฟจากแหล่งพลังงานภายนอกเพื่อจ่ายไฟให้กับ NodeMCU และอุปกรณ์ต่อพ่วงได้โดยตรง โดยสามารถรับได้ไม่เกิน 5 V
 - 3.3V: เป็นเอาต์พุตของตัวควบคุมแรงดันไฟฟ้าออนบอร์ดและสามารถใช้เพื่อจ่ายไฟให้กับส่วนประกอบภายนอกได้
- GND pin: เป็นขาราวนด์ มีทั้งหมด 5 ขา
- RX/TX pin: เป็นขา GPIO และขาที่ใช้ในการรับส่งข้อมูลแบบอนุกรม (Serial Communication) กับอุปกรณ์อื่น
- VU pin: เป็นขาแรงดันไฟ 5 V จากพอร์ต USB
- S2/S3 pin: เป็นขา GPIO สำหรับเชื่อมต่อกับอุปกรณ์ SD card ผ่านอินเทอร์เฟซ Secure Digital Input/Output (SDIO)
- S0/S1/SC/SK pin: เป็นขา GPIO สำหรับเชื่อมต่อกับอุปกรณ์ SD card ผ่านอินเทอร์เฟซ Secure Digital Input/Output (SDIO) และติดต่อสื่อสารกับโมดูลอื่น ๆ ผ่านอินเทอร์เฟซ SPI ที่เร็วกว่า I2C
- EN pin: ขาควบคุมให้โมดูลทำงาน เมื่อมีการจ่ายไฟหรือกำหนดสถานะให้เป็น Active High
- RST pin: ขารีเซ็ตโมดูล เมื่อต่อกับกราวนด์หรือกำหนดสถานะให้เป็น Active Low
- 2.4 GHz Antenna: สายอากาศ มีความถี่ 2.4 GHz
- หลอด LED: ใช้ในการแสดงสถานะบนการอัปโหลดโปรแกรมลงบนบอร์ด รวมถึงแสดงสถานะการทำงานของโปรแกรมหรือการส่งข้อมูล
- Chip ESP8266: หน่วยประมวลผลหลัก 32-bit มีความถี่ 80-160 MHz หน่วยความจำ 128 KB internal RAM + 4 MB external Flash ตัวรับส่งสัญญาณ WiFi มาตรฐาน 802.11 b/g/n ทำงานที่แรงดันไฟฟ้า 3.0-3.6 V และกระแสไฟฟ้าโดยเฉลี่ยที่ 80 mA
- Chip 3.3V LDO Voltage Regulator: ชิปที่ใช้ในการควบคุมแรงดันไฟฟ้าให้คงที่ที่ 3.3 V
- Chip USB to TTL Convertor: เป็นตัวกลางในการสื่อสารระหว่างอุปกรณ์ที่ใช้พอร์ต USB ไปยังบอร์ดไมโครคอนโทรลเลอร์ เช่น การอัปโหลดโปรแกรม
- Flash button: ใช้สำหรับการอัปโหลดโปรแกรม
- Micro USB Connector: ใช้เสียบสาย USB เพื่อจ่ายไฟ 5 V ให้กับบอร์ด และอัปโหลดข้อมูล
- RST button: ใช้ในการรีเซ็ตบอร์ดเพื่อเริ่มการทำงานใหม่

1.2 NodeMCU Specification

ตารางที่ 1.1: ข้อมูลจำเพาะของบอร์ด NodeMCU ESP8266

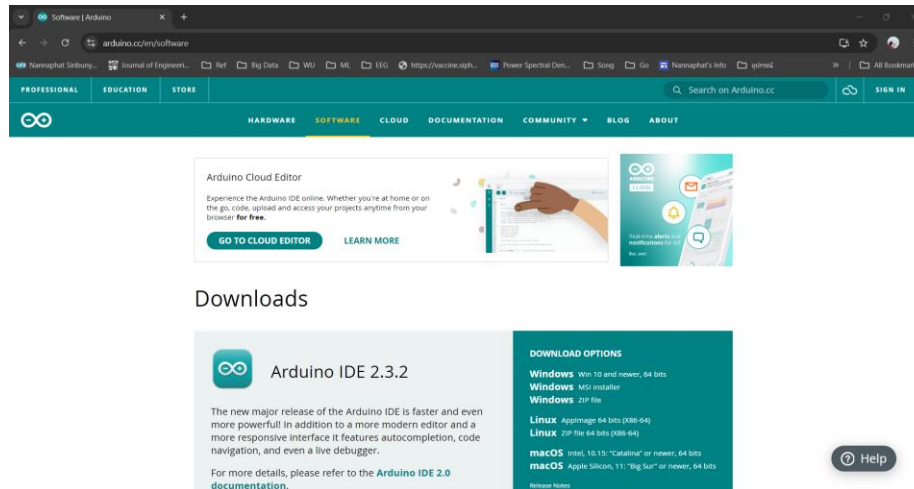
Topic	Specification
Microcontroller	ESP-8266 32-bit
Pin Spacing	0.9" (22.86mm)
Clock Speed	80 MHz
USB Connector	Micro USB
Operating Voltage	3.3V
Input Voltage	4.5V-10V
Flash Memory/SRAM	4 MB / 64 KB
Digital I/O Pins	11
Analog In Pins	1
ADC Range	0-3.3V
UART/SPI/I2C	1 / 1 / 1
WiFi Built-In	802.11 b/g/n

2. Arduino IDE

ในการพัฒนาโปรแกรมสำหรับ NodeMCU ESP8266 จะใช้ภาษาซีในการพัฒนา โดยทำการเขียนโปรแกรมผ่าน Arduino IDE และทำการฝังโปรแกรมลงบนบอร์ด โดยจะต้องดำเนินการเตรียมโปรแกรมให้พร้อมก่อนที่จะทำการเขียนโปรแกรมลงบนบอร์ด โดยมีขั้นตอนดังนี้ [3]

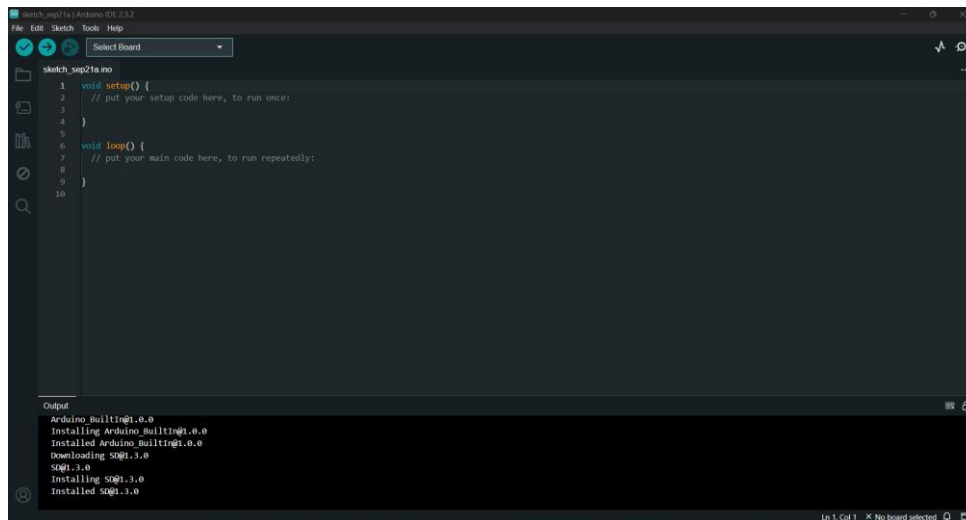
2.1 การติดตั้งโปรแกรม Arduino IDE

- ไปที่เว็บไซต์ <https://www.arduino.cc/en/software> จากนั้นทำการดาวน์โหลดตัวติดตั้งโปรแกรม Arduino IDE ให้ตรงกับระบบปฏิบัติการของคอมพิวเตอร์ที่ใช้



รูปที่ 1.2 หน้าต่างเว็บไซต์ <https://www.arduino.cc/en/software>

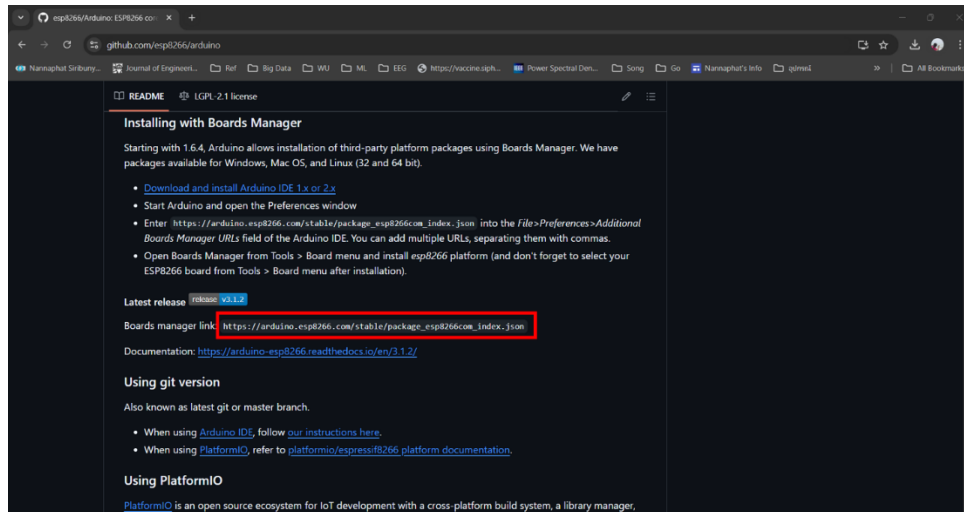
- คลิกปุ่ม Just Download จากนั้นเลือกตำแหน่งจัดเก็บไฟล์แล้วกด Save
- ติดตั้งโปรแกรมตามขั้นตอนให้เสร็จ
- เมื่อเปิดโปรแกรม จะแสดงหน้าจอ ดังรูป



รูปที่ 1.3 หน้าต่างโปรแกรม Arduino IDE

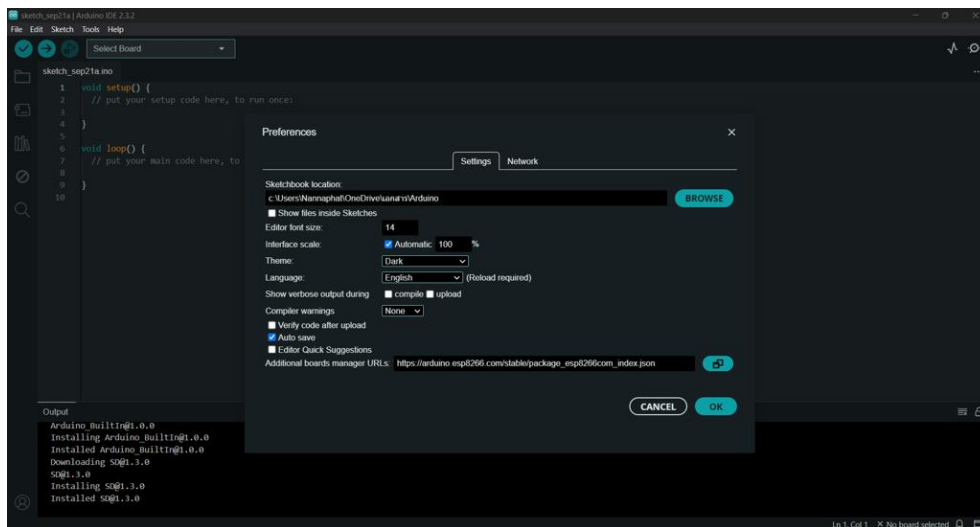
2.2 การติดตั้ง Libraries

- เข้าเว็บไซต์ <https://github.com/esp8266/arduino> จากนั้นเลื่อนลงมาที่ Board Manager Link จากนั้นคัดลอก URL ดังที่แสดงในรูป



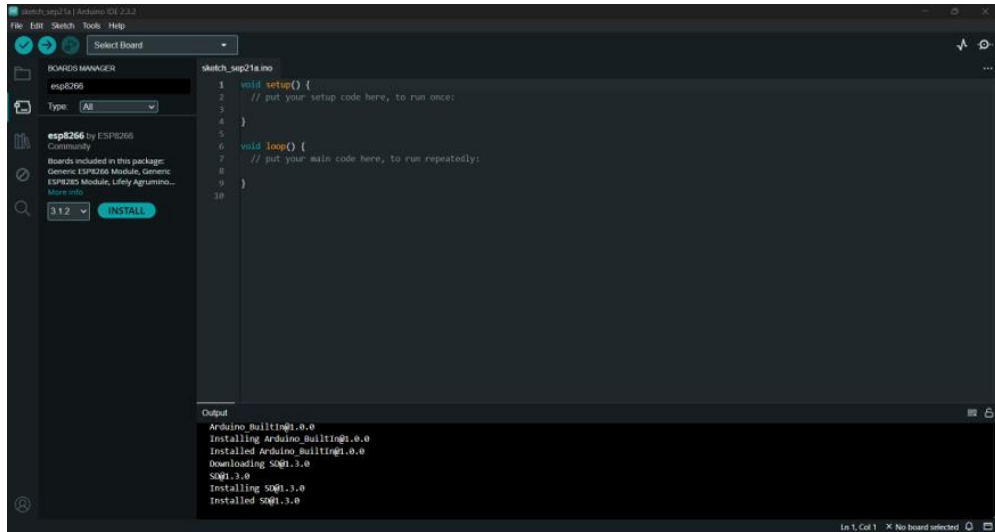
รูปที่ 1.4 ลิงก์สำหรับติดตั้งในการจัดการบอร์ด ESP8266

- ไปที่โปรแกรม Arduino IDE คลิกเมนู File -> Preferences.. ที่แท็บ Settings ให้นำ URL ที่คัดลอกไว้มาวางในช่อง Additional Boards Manager URLs จากนั้นคลิกปุ่ม OK



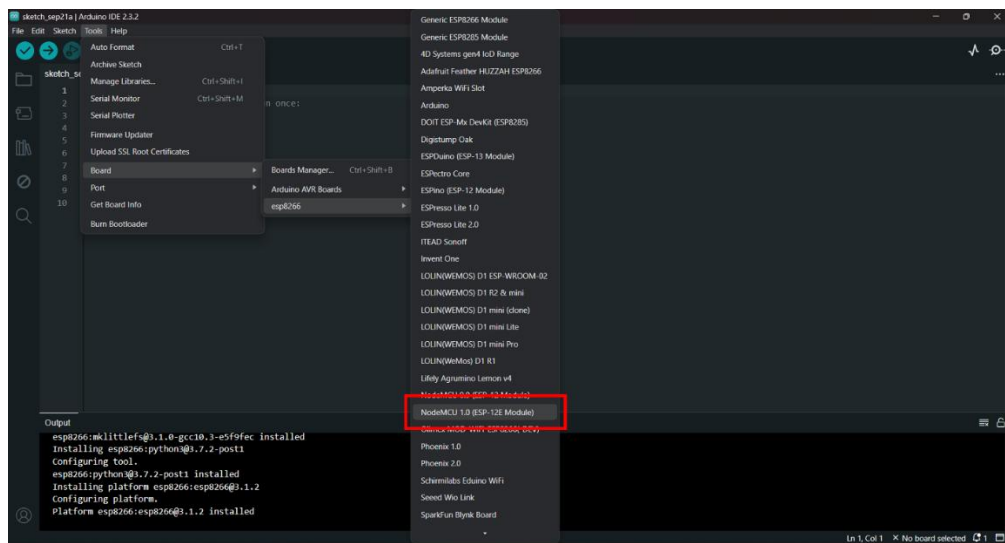
รูปที่ 1.5 Additional Boards Manager URLs

- คลิกที่เมนู Tools -> Board -> Board Manager...
- ในช่องค้นหาพิมพ์คำว่า esp8266 จะปรากฏ Libraries ที่รองรับบอร์ด กดปุ่ม Install เพื่อดาวน์โหลดและติดตั้ง



รูปที่ 1.6 ติดตั้ง Library สำหรับ esp8266

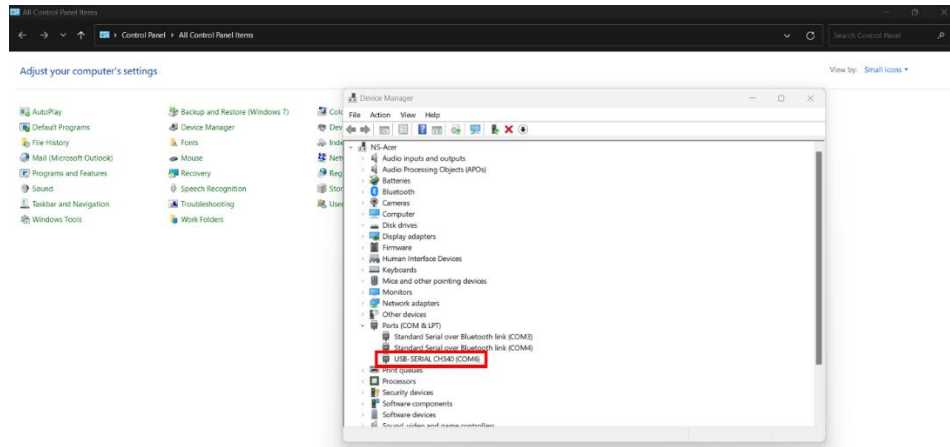
- คลิกเมนู Tools -> Board -> esp8266 เลือก NodeMCU 1.0 (ESP-12E Module)



รูปที่ 1.7 การเลือกบอร์ดสำหรับการเขียนโปรแกรม

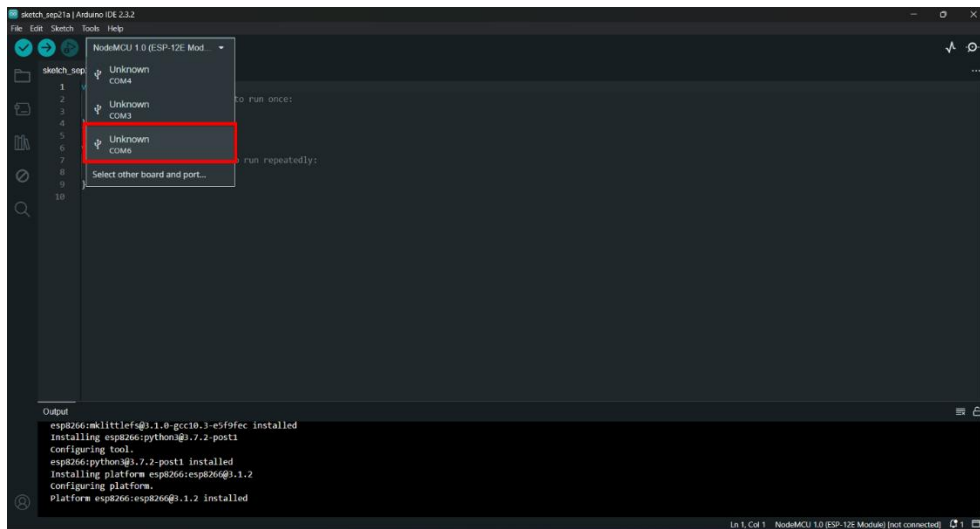
2.3 การติดตั้ง Driver และตรวจสอบ Port

- ต่อบอร์ด NodeMCU กับคอมพิวเตอร์
- ไปที่ Control Panel -> Device Manager จากนั้นไปที่ Ports (COM & PLT) ทำการตรวจสอบพอร์ตที่เชื่อมต่อบอร์ด



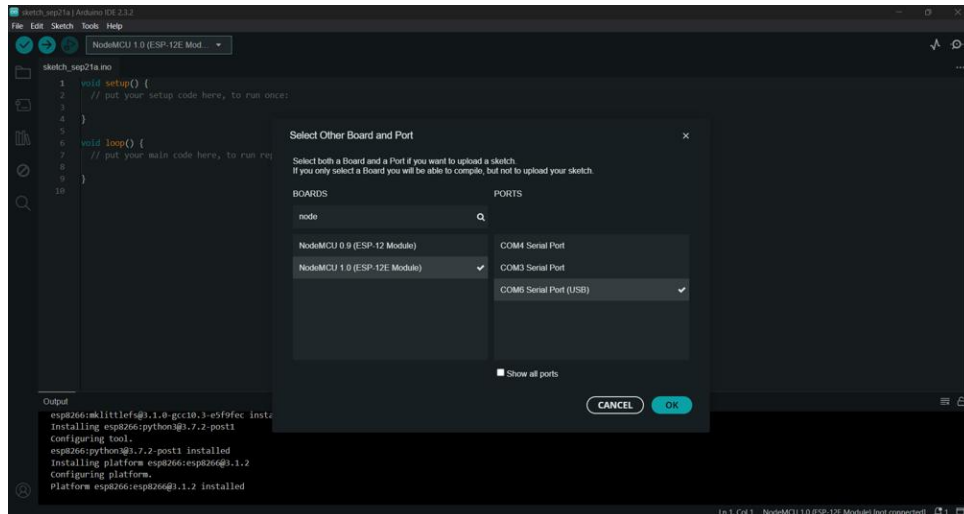
รูปที่ 1.8 พอร์ตที่เชื่อมต่อกับบอร์ด NodeMCU

- กลับมาที่ Arduino IDE ที่แถบเมนู คลิกที่ dropdown ที่มีชื่อ NodeMCU จากนั้นกดเลือกพอร์ตที่ตรงกับใน Device Manager



รูปที่ 1.9 เลือกพอร์ตที่ตรงกับที่คอมพิวเตอร์กำหนด

- ตรวจสอบบอร์ดและพอร์ตให้ถูกต้อง จากนั้นคลิก OK



รูปที่ 1.10 ตรวจสอบบอร์ดและพอร์ตให้ถูกต้อง

3. โครงสร้างภาษาซีสำหรับการเขียน NodeMCU ผ่าน Arduino IDE

การเขียนโปรแกรมภาษาซีสำหรับการใช้งานบนบอร์ด NodeMCU จะมีองค์ประกอบที่สำคัญ 3 ส่วน คือ

1. Header

Header เป็นส่วนที่รวบรวมคำสั่ง ฟังก์ชัน หรือตัวแปรจาก Libraries ซึ่งสามารถ import เข้ามาเพื่อช่วยในการเขียนโปรแกรมได้ โดยปกติแล้วจะมีหรือไม่มีก็ได้ ขึ้นอยู่กับการเรียกใช้งานในการเขียนโปรแกรมนั้น ๆ

2. void setup()

เป็นฟังก์ชันบังคับที่ต้องมีในทุก ๆ โปรแกรม โดยเป็นฟังก์ชันที่จะทำการตั้งค่าต่าง ๆ เพื่อให้โปรแกรมทำงานเพียงรอบเดียวในช่วงที่เริ่มต้นการทำงาน เช่น การตั้งค่าอุปกรณ์เชื่อมต่อ การตั้งค่า Baud Rate เป็นต้น

3. void loop()

เป็นฟังก์ชันบังคับที่ต้องมีในทุก ๆ โปรแกรม โดยจะเป็นส่วนที่ใช้ในการทำงานของบอร์ด โดยจะมีการทำงานต่อเนื่องไปเรื่อย ๆ จนกระทั่งมีคำสั่งในออกจาก loop หรือจนกว่าบอร์ดจะหยุดการทำงาน

4. การทำงานของอุปกรณ์แบบ Active Low และ Active High

Active Low และ Active High เป็นแนวคิดพื้นฐานสำคัญสำหรับการเขียนโปรแกรมไมโครคอนโทรลเลอร์ โดยจะใช้ในการอธิบายสถานะของสัญญาณว่าเมื่ออยู่ในสถานะใดจึงถือว่ามีการทำงาน (active) โดยจะมี 2 สถานะ คือ Active Low และ Active High

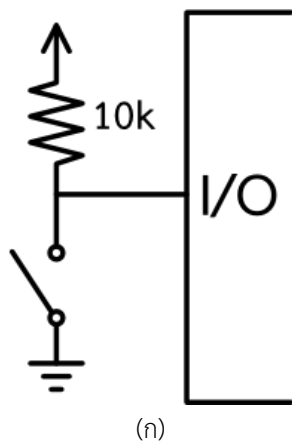
4.1 Active Low หรือ วงจร Pull-Up

Active Low หรือ วงจร Pull-Up คือ การที่อุปกรณ์จะทำงาน (active) ในกรณีที่สัญญาณมีค่าต่ำ (0) หรือต่อสายสัญญาณลง GND ตัวอย่างเช่น สวิตช์แบบ Active Low จะมีการเริ่มต้นการต่อวงจรโดยนำตัวต้านทานมาต่อระหว่างขาอุปกรณ์ I/O และ V_{cc} (การต่อแบบ Pull-Up) จากนั้นต่อสวิตช์ระหว่างขา I/O กับ GND ในกรณีที่ไมกดสวิตช์ ไฟเลี้ยงจาก V_{cc} จะไปเลี้ยงที่ขา I/O ตลอดเวลา หรือมีค่าสัญญาณเป็น 1 หรือ HIGH และเมื่อทำการกดสวิตช์ ขาของ I/O จะถูกต่อลง GND จึงทำให้สัญญาณที่ได้รับมีค่าเป็น 0 หรือ LOW

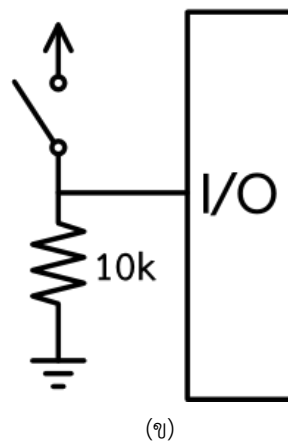
4.2 Active High หรือ วงจร Pull-Down

Active High หรือ วงจร Pull-Down จะทำงานในด้านตรงกันข้ามกับวงจร Pull-Up คือ การที่อุปกรณ์จะทำงาน (active) ในกรณีที่สัญญาณมีค่าสูง (1) หรือต่อสายสัญญาณเข้ากับ V_{cc} ตัวอย่างเช่น สวิตช์แบบ Active High จะมีการเริ่มต้นการต่อวงจรโดยนำตัวต้านทานมาต่อระหว่างขาอุปกรณ์ I/O และขา GND (การต่อแบบ Pull-Down) จากนั้นต่อสวิตช์ระหว่างขา I/O กับ V_{cc} ในกรณีที่ไมกดสวิตช์ ขา I/O จะต่อลง GND ตลอดเวลา หรือมีค่าสัญญาณเป็น 0 หรือ LOW และเมื่อทำการกดสวิตช์ ขาของ I/O จะถูกต่อเข้ากับ V_{cc} จึงทำให้สัญญาณที่ได้รับมีค่าเป็น 1 หรือ HIGH

Active Low/Pull-up



Active High/Pull-down



รูปที่ 1.11 การต่อวงจรในการทำงานบนบอร์ดไมโครคอนโทรลเลอร์ (ก) การต่อวงจรแบบ Active Low (ข) การต่อวงจรแบบ

Active High ที่มา <http://www.atrobt.com/2015/10/active-low.html>

Lab 1: ทดสอบการทำงานของบอร์ด

- พิมพ์โค้ดต่อไปนี้

```
void setup() {  
  pinMode(D4, OUTPUT);  
}  
  
void loop() {  
  digitalWrite(D4, LOW);  
  delay(1000);  
  digitalWrite(D4, HIGH);  
  delay(2000);  
}
```

- กดปุ่ม Verify เพื่อตรวจสอบโค้ด รอกจนกระทั่งตรวจสอบเสร็จ
- กดปุ่ม Upload เพื่ออัปโหลดโค้ดไปยังบอร์ด รอกจนกระทั่งอัปโหลดโค้ดเสร็จ

ผลการทำงานหลังจากอัปโหลดโค้ดลงบอร์ด

Lab 2

การใช้งาน Serial Monitor

Serial Monitor เป็นเครื่องมือที่สำคัญในการสร้างโปรเจกต์ด้วย Arduino สามารถใช้เป็นเครื่องมือในการดีบั๊ก ทดสอบแนวคิด หรือทำการสื่อสารกับบอร์ดโดยตรงได้ โดยที่การรับส่งข้อมูลระหว่างบอร์ดกับคอมพิวเตอร์จะเป็นการส่งข้อมูลแบบอนุกรม (Serial Communication) โดยจะต้องทำการตั้งค่า Baud Rate ให้ตรงกัน โดยค่าความเร็วมาตรฐานที่ใช้คือ 9600 baud และค่าสูงสุดที่ใช้ได้ 115200 baud เนื่องจากถ้าค่าสูงกว่านี้จะส่งผลกระทบต่อความผิดพลาดได้ง่าย โดยทำการกำหนดค่า Baud Rate ได้จากคำสั่ง Serial.begin() ในโค้ด และตั้งค่า Baud Rate ในหน้าต่าง Serial Monitor ให้ตรงกัน

Lab 2.1: Basic Serial Monitor

ขั้นตอนในการทำ

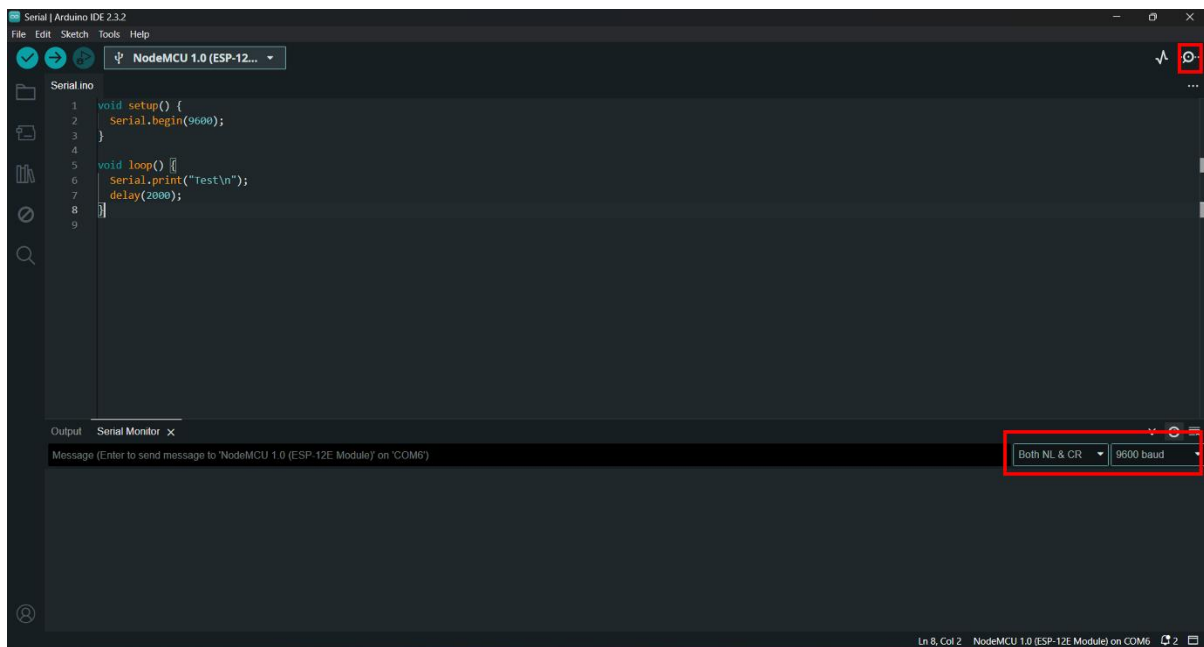
- พิมพ์โค้ดต่อไปนี้แล้วทำการ Verify โค้ด แล้วอัปโหลดโค้ดลงบอร์ด

```
void setup() {
  Serial.begin(9600);
}

void loop() {
  Serial.print("Test\n");
  delay(2000);
}
```

- เปิดหน้าต่าง Serial Monitor บริเวณมุมขวาของแถบเครื่องมือ และทำการปรับค่า Baud Rate ให้เท่ากับ 9600 และปรับโหมดให้เป็น Both NL & CR

ผลการทำงานหลังจากอัปโหลดโค้ดลงบอร์ด



รูปที่ 2.1 การเปิดหน้าต่าง Serial Monitor และตั้งค่า Baud Rate

Lab 2.2: การส่งข้อมูลจากแป้นพิมพ์ผ่าน Serial Monitor ไปยังบอร์ด

ขั้นตอนในการทำ

- พิมพ์โค้ดต่อไปนี้แล้วทำการ Verify โค้ด แล้วอัปโหลดโค้ดลงบอร์ด

```
const int ledPin = D4;

void setup() {
  pinMode(ledPin, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  if(Serial.available() > 0){
    char inRead = Serial.read();
    if(inRead == '1') {
      digitalWrite(ledPin, LOW);
      Serial.println("put 1");
    } else if(inRead == '0') {
      digitalWrite(ledPin, HIGH);
      Serial.println("put 0");
    } else if (inRead > '1') {
      Serial.println("Please enter 0 or 1 only");
    }
    delay(100);
  }
}
```

- เปิดหน้าต่าง Serial Monitor บริเวณมุมขวาของแถบเครื่องมือ และทำการปรับค่า Baud Rate ให้เท่ากับ 9600 และปรับโหมดให้เป็น Both NL & CR
- ทดลองพิมพ์เลข 0 1 และ 2 ลงในช่อง Message (Enter to send message to NodeMCU) แล้วสังเกตผลลัพธ์ที่เกิดขึ้น

ผลการทำงานหลังจากอัปโหลดโค้ดลงบอร์ด

Lab 2.3: การใช้งาน Serial Plotter

ขั้นตอนในการทำ

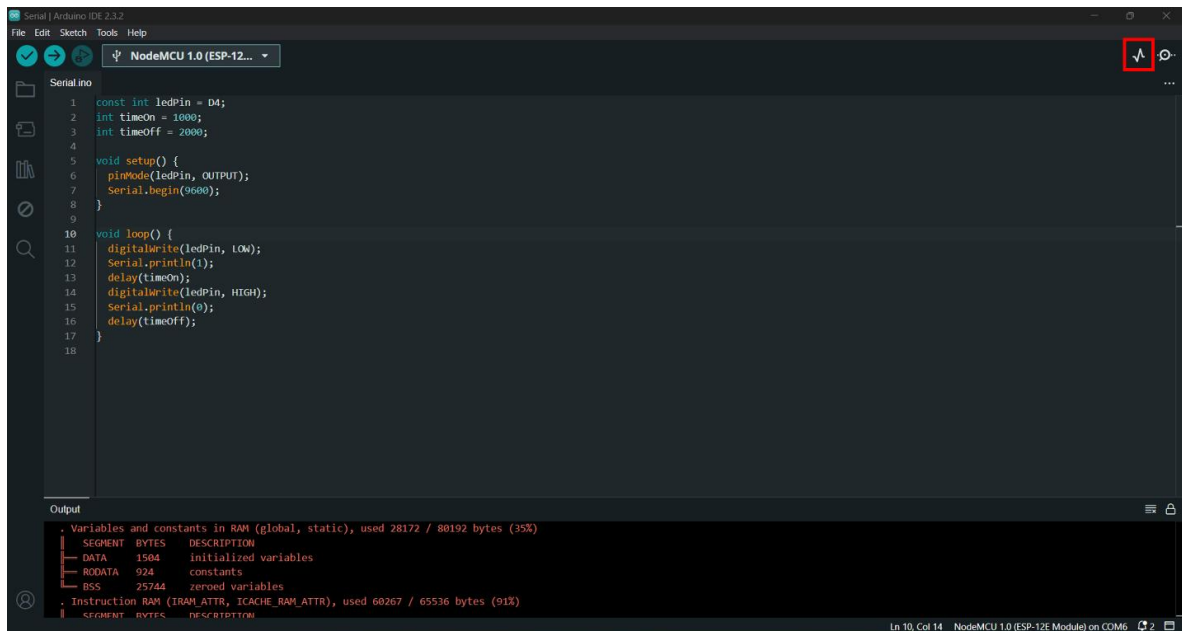
- พิมพ์โค้ดต่อไปนี้แล้วทำการ Verify โค้ด แล้วอัปโหลดโค้ดลงบอร์ด

```
const int ledPin = D4;
int timeOn = 1000;
int timeOff = 2000;

void setup() {
  pinMode(ledPin, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  digitalWrite(ledPin, LOW);
  Serial.println(1);
  delay(timeOn);
  digitalWrite(ledPin, HIGH);
  Serial.println(0);
  delay(timeOff);
}
```

- เปิดหน้าต่าง Serial Plotter กดเลือก value 1 สังเกตผลที่มีการเปลี่ยนแปลง



รูปที่ 2.2 การเข้าหน้าต่าง Serial Plotter

- เปิด interpolate ให้เป็น on สังเกตผลที่มีการเปลี่ยนแปลง

ผลการทำงานหลังจากอัปโหลดโค้ดลงบอร์ด

Lab 3

พื้นฐานการใช้ขา Input/Output (GPIO)

General Purpose Input Output (GPIO) เป็นขาของบอร์ด NodeMCU ที่ใช้ในการเชื่อมต่อกับอุปกรณ์อินพุตและเอาต์พุต โดยสามารถเขียนโค้ดเพื่อออกคำสั่งให้แต่ละขาที่ต้องการ เพื่อให้สามารถทำงานได้ตามที่ต้องการ

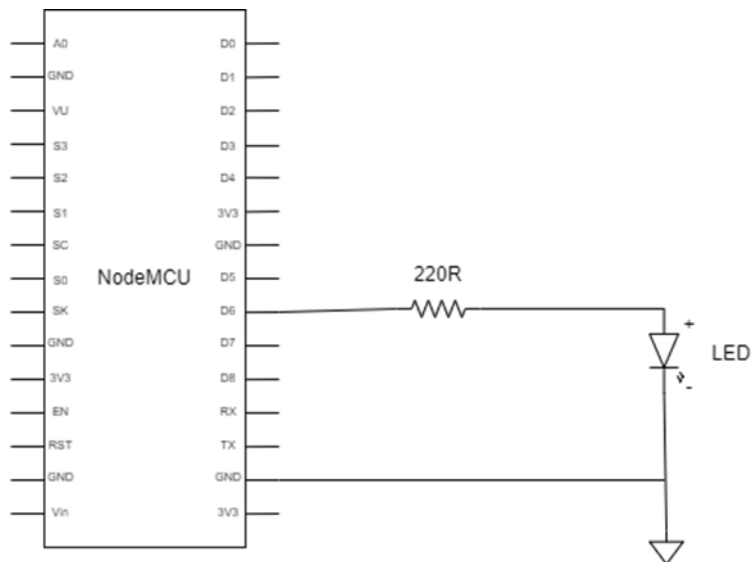
Lab 3.1: วงจรไฟกะพริบโดยใช้ LED (Digital Output)

อุปกรณ์ที่ต้องใช้

- บอร์ด NodeMCU 1 ตัว
- หลอด LED 1 อัน
- ตัวต้านทานขนาด 220 Ω 1 อัน
- สาย Jumper
- Breadboard 1 อัน

ขั้นตอนในการทำ

- ต่ วงจรดังที่แสดงในรูป



รูปที่ 3.1 วงจรของแลบที่ 3.1

- พิมพ์โค้ดต่อไปนี้แล้วทำการ Verify โค้ด แล้วอัปโหลดโค้ดลงบอร์ด

```

void setup() {
  pinMode(D6, OUTPUT);
}

void loop() {
  digitalWrite(D6, HIGH);
  delay(1000);
  digitalWrite(D6, LOW);
  delay(1000);
}

```

ผลการทำงานหลังจากอัปโหลดโค้ดลงบอร์ด

- จากตารางรหัสมอร์สต่อไปนี้จะปรับปรุงโค้ดเพื่อให้วงจรสามารถส่งสัญญาณเป็นชื่อเล่นของนักศึกษา

ตารางที่ 3.1 ตารางรหัสมอร์ส

ตัวอักษร	รหัส	ตัวอักษร	รหัส	ตัวอักษร	รหัส
A	. -	J	. ---	S	...
B	- ...	K	- . -	T	-
C	- . -	L	. - .	U	.. -
D	- ..	M	--	V	... -
E	.	N	- .	W	. - -
F	.. -	O	---	X	- .. -
G	-- .	P	. - .	Y	- . - -
H		Q	-- . -	Z	-- ..
I	..	R	. - .		

ผลการทำงานหลังจากอัปโหลดโค้ดลงบอร์ด

ส่งไฟล์โค้ดที่ทำการแก้ไขแล้วใน Google Classroom ใน Lab 3

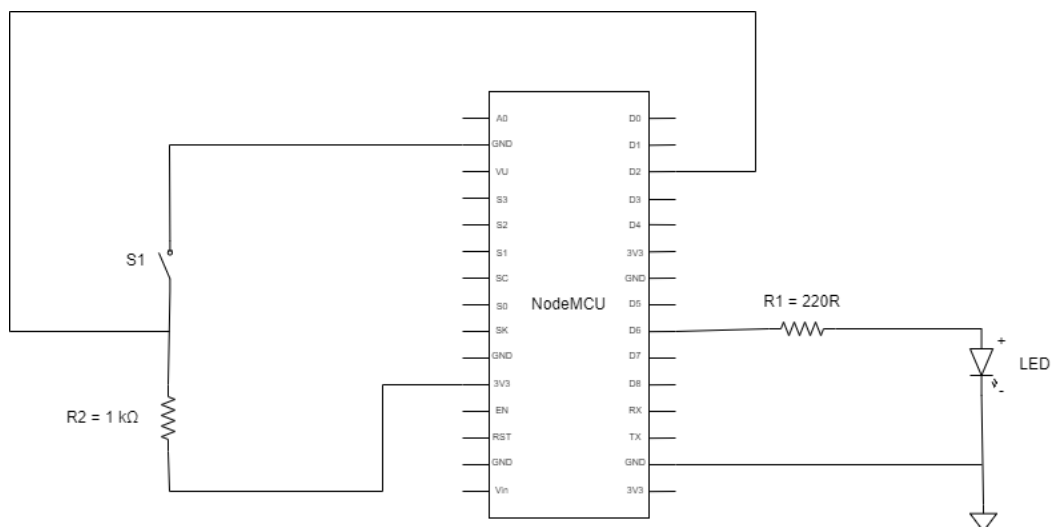
Lab 3.2: เปิด-ปิดหลอด LED โดยใช้ switch

อุปกรณ์ที่ต้องใช้

- บอร์ด NodeMCU 1 ตัว
- หลอด LED 1 อัน
- ตัวต้านทานขนาด 220 Ω 1 อัน
- ตัวต้านทานขนาด 1 k Ω 1 อัน
- สาย Jumper
- Breadboard 1 อัน
- Tact Switch (สวิตช์กดติดปล่อยดับ) 1 อัน

ขั้นตอนในการทำ

- ต่อวงจรดังที่แสดงในรูป



รูปที่ 3.2 วงจรของแลบที่ 3.2

- พิมพ์โค้ดต่อไปนี้แล้วทำการ Verify โค้ด แล้วอัปโหลดโค้ดลงบอร์ด

```
void setup() {  
  pinMode(D2, INPUT);  
  pinMode(D6, OUTPUT);  
}  
  
void loop() {  
  int buttonStatus = digitalRead(D2);  
  if (buttonStatus == 0) {  
    digitalWrite(D6, HIGH);  
  } else {  
    digitalWrite(D6, LOW);  
  }  
  delay(100);  
}
```

ผลการทำงานหลังจากอัปโหลดโค้ดลงบอร์ด

วงจรที่ต่อเป็นวงจรแบบใด เพราะอะไร

Lab 4

การแปลงสัญญาณแอนะล็อกเป็นสัญญาณดิจิทัล และ การแปลงสัญญาณดิจิทัลเป็นแอนะล็อก (Analog to Digital Convertor and Digital to Analog Convertor)

โดยปกติแล้วเซ็นเซอร์จะรับสัญญาณทางกายภาพเข้ามา ซึ่งจะได้เป็นสัญญาณแอนะล็อก การที่บอร์ดไมโครคอนโทรลเลอร์จะสามารถทำงานได้จะต้องทำการแปลงสัญญาณจากแอนะล็อกเป็นดิจิทัล (Analog to Digital Convertor: ADC) สำหรับบอร์ด NodeMCU ESP8266 นั้นจะมีขา A0 เป็นขาอินพุตแบบแอนะล็อก 1 ช่อง โดยมีความละเอียดในการแปลงสัญญาณขนาด 10 bit หรือ 2^{10} หรือ 1024 ระดับ ซึ่งสามารถระบุข้อมูลตัวเลขแบบแอนะล็อกได้ตั้งแต่ 0 – 1023 ระดับ ซึ่งสามารถเทียบแรงดันบนบอร์ดได้ตั้งแต่ 0 – 3.3 V ซึ่งสามารถคำนวณได้จากสมการต่อไปนี้

$$\text{Analog Level} = \frac{1023 \times V_{in}}{V_{max}}$$

เมื่อ Analog Level คือ ระดับของสัญญาณแอนะล็อก มีค่าตั้งแต่ 0 – 1023

V_{in} คือ ค่าแรงดันขาเข้า (ค่าสัญญาณจากเซ็นเซอร์)

V_{max} คือ ค่าแรงดันสูงสุดของบอร์ด ซึ่งสำหรับบอร์ด NodeMCU ESP8266 มีค่าที่ 3.3 V

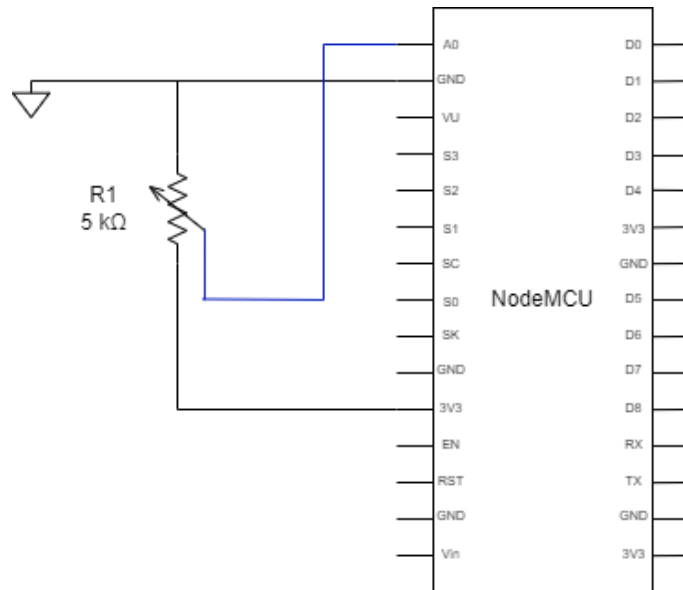
Lab 4.1: การวัดสัญญาณแอนะล็อก

อุปกรณ์ที่ต้องใช้

- บอร์ด NodeMCU 1 ตัว
- ตัวต้านทานปรับค่าได้ขนาด 10 kΩ 1 ตัว
- สาย Jumper
- Breadboard 1 อัน

ขั้นตอนในการทำ

- ต่วงจรดังที่แสดงในรูป



รูปที่ 4.1 วงจรของแลบที่ 4.1

- พิมพ์โค้ดต่อไปนี้แล้วทำการ Verify โค้ด แล้วอัปโหลดโค้ดลงบอร์ด

```
const int analogPin = A0;
int sensorValue = 0;

void setup() {
  Serial.begin(115200);
}

void loop() {
  sensorValue = analogRead(analogPin);
  Serial.println(sensorValue);
  delay(50);
}
```

- เปิด Serial Monitor พร้อมทำการปรับค่าตัวด้านทานปรับค่าได้โดยการหมุน
- ปิดหน้าต่าง Serial Monitor แล้วเปิดหน้าต่าง Serial Plotter พร้อมทำการปรับค่าตัวด้านทานแบบปรับค่าได้โดยการหมุน

ผลการทำงานหลังจากอัปโหลดโค้ดลงบอร์ด

Lab 4.2: กำหนดช่วงของเอาต์พุตด้วยฟังก์ชัน map

function map -> map(value, fromLow, fromHigh, toLow, toHigh);

เมื่อ value คือ ค่าที่ต้องการแปลง

fromLow คือ ค่าตัวเลขต่ำสุดของช่วงเดิม

fromHigh คือ ค่าตัวเลขสูงสุดของช่วงเดิม

toLow คือ ค่าตัวเลขต่ำสุดของช่วงใหม่

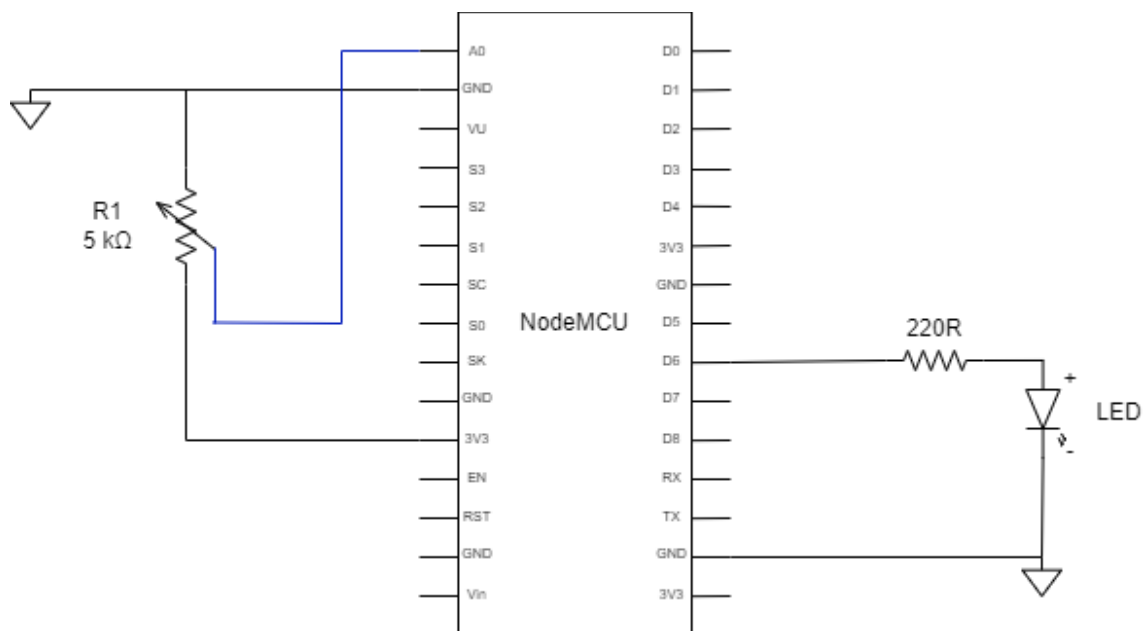
toHigh คือ ค่าตัวเลขสูงสุดของช่วงใหม่

อุปกรณ์ที่ต้องใช้

- บอร์ด NodeMCU 1 ตัว
- หลอด LED 1 อัน
- ตัวต้านทาน 220 Ω 1 ตัว
- ตัวต้านทานปรับค่าได้ขนาด 10 k Ω 1 ตัว
- สาย Jumper
- Breadboard 1 อัน

ขั้นตอนในการทำ

- ต่อดังที่แสดงในรูป



รูปที่ 4.2 วงจรของแลบที่ 4.2

- พิมพ์โค้ดต่อไปนี้แล้วทำการ Verify โค้ด แล้วอัปโหลดโค้ดลงบอร์ด

```
const int analogPin = A0;
int sensorValue = 0;
int newValue = 0;

void setup() {
  Serial.begin(115200);
  pinMode(D6, OUTPUT);
}

void loop() {
  sensorValue = analogRead(analogPin);
  newValue = map(sensorValue, 0, 1023, 100, 1500);

  Serial.println(newValue);

  digitalWrite(D6, HIGH);
  delay(newValue);
  digitalWrite(D6, LOW);
  delay(50);
}
```

- เปิด Serial Monitor พร้อมทำการปรับค่าตัวด้านทานปรับค่าได้โดยการหมุน
- ปิดหน้าต่าง Serial Monitor แล้วเปิดหน้าต่าง Serial Plotter พร้อมทำการปรับค่าตัวด้านทานแบบปรับค่าได้โดยการหมุน

ผลการทำงานหลังจากอัปโหลดโค้ดลงบอร์ด

Lab 4.3: การจำลองการแปลงสัญญาณดิจิทัลเป็นแอนาล็อก

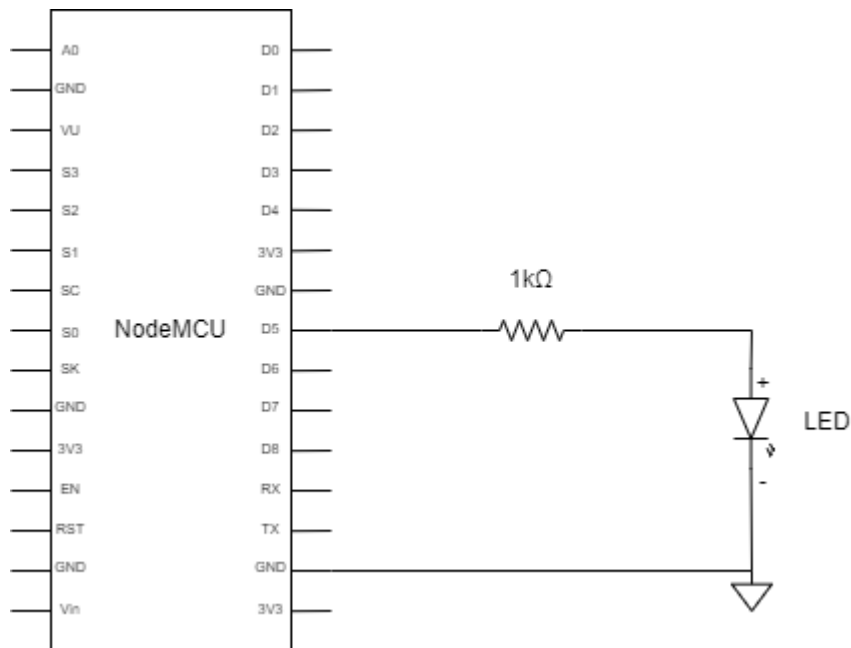
สำหรับการแปลง สัญญาณดิจิทัลเป็นแอนาล็อก (Digital to Analog Conversion - DAC) บน ESP8266 ไม่มี DAC แท้ในตัว แต่สามารถ จำลองสัญญาณแอนาล็อกได้ด้วย PWM (Pulse Width Modulation) ซึ่งใช้ในการควบคุมค่าเฉลี่ยของแรงดันไฟฟ้าในช่วงเวลาหนึ่งได้อย่างมีประสิทธิภาพ

อุปกรณ์ที่ต้องใช้

- บอร์ด NodeMCU 1 ตัว
- ตัวต้านทานขนาด $1\text{k}\Omega$ 1 ตัว
- ตัวเก็บประจุขนาด $10\text{ }\mu\text{F}$
- หลอด LED 1 อัน
- สาย Jumper
- Breadboard 1 อัน

ขั้นตอนในการทำ

- ต่อดังต่อไปนี้แสดงในรูป



รูปที่ 4.3 วงจรของแลบที่ 4.3

- พิมพ์โค้ดต่อไปนี้แล้วทำการ Verify โค้ด แล้วอัปโหลดโค้ดลงบอร์ด

```
int pwmPin = D5;

void setup() {
  pinMode(pwmPin, OUTPUT);
}

void loop() {
  for (int i = 0; i <= 1023; i += 10) {
    analogWrite(pwmPin, i);
    delay(10);
  }
  for (int i = 1023; i >= 0; i -= 10) {
    analogWrite(pwmPin, i);
    delay(10);
  }
}
```

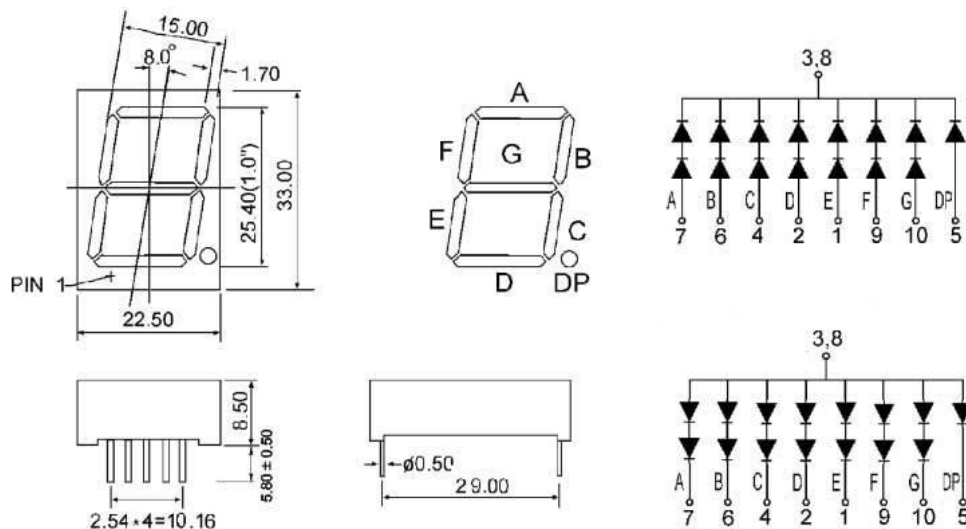
ผลการทำงานหลังจากอัปโหลดโค้ดลงบอร์ด

Lab 5

จอแสดงผลแบบเจ็ดส่วน (7-Segment Display)

จอแสดงผลแบบเจ็ดส่วน (Seven Segment Display) คืออุปกรณ์แสดงผลแบบเอาต์พุตที่ทำหน้าที่แสดงข้อมูลในรูปแบบรูปภาพ ข้อความ หรือตัวเลขทศนิยม ซึ่งเป็นอีกทางเลือกหนึ่งสำหรับจอแสดงผลแบบดอทเมทริกซ์ที่ซับซ้อนกว่า จอแสดงผลแบบเจ็ดส่วนนี้ใช้กันอย่างแพร่หลายในนาฬิกาดิจิทัล เครื่องคิดเลขพื้นฐาน มิเตอร์อิเล็กทรอนิกส์ และอุปกรณ์อิเล็กทรอนิกส์อื่น ๆ ที่ใช้แสดงข้อมูลเชิงตัวเลข ประกอบด้วยไดโอดเปล่งแสง (LED) เจ็ดส่วน ซึ่งประกอบขึ้นเหมือนเลข 8

จอแสดงผลแบบเจ็ดส่วนสร้างชุดตัวเลขแสดงผลได้ตั้งแต่ 0 ถึง 9 และสามารถสร้างตัวอักษรได้ A, B, C, D, E และ F ซึ่งรูปแบบนี้สามารถใช้แสดงตัวเลขตั้งแต่ 0 ถึง 9 และตัวอักษรตั้งแต่ 0 ถึง 9 ที่เป็นเลขฐานสิบหก โดยปกติแล้วจะมีจุด (DP) จุดแสดงผลอาจอยู่ทางด้านซ้ายหรือด้านขวาของรูปแบบการแสดงผล

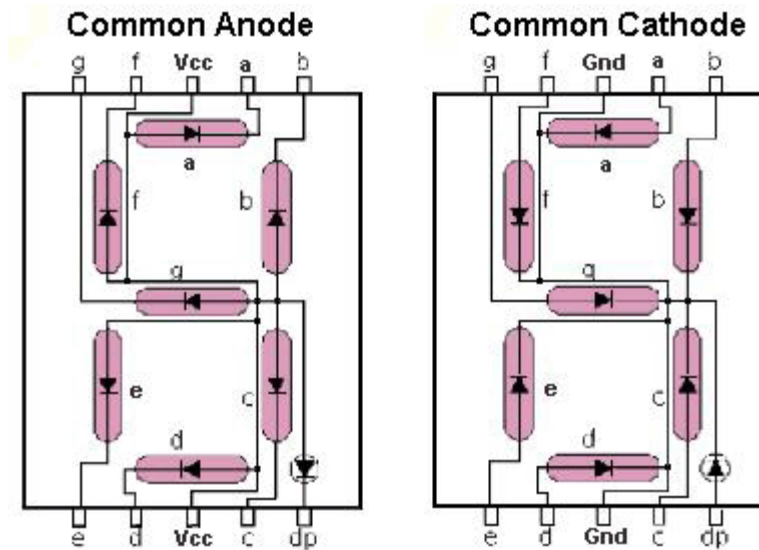


รูปที่ 5.1 ไดอะแกรมของจอแสดงผลแบบเจ็ดส่วน

จอแสดงผลแบบเจ็ดส่วนมีสองประเภทตามประเภทของการใช้งาน ได้แก่ จอแสดงผลแบบแอโนดร่วม (Common Anode Display) และจอแสดงผลแบบแคโทดร่วม (Common Cathode Display)

1. **จอแสดงผลแบบเจ็ดส่วนแบบแคโทดร่วม (Common Cathode Display)** การเชื่อมต่อแคโทดทั้งหมดของเซกเมนต์ LED จะเชื่อมต่อกันที่ลอจิก 0 หรือกราวด์ และใช้ลอจิก 1 ผ่านตัวต้านทานจำกัดกระแสเพื่อส่งต่อไปยังขั้วบวกแต่ละขั้วจาก a ไปยัง g

2. จอแสดงผลแบบเจ็ดส่วนแบบแอโนดร่วม (Common Anode Display) การเชื่อมต่อแอโนดทั้งหมดของเซกเมนต์ LED จะเชื่อมต่อกันที่ลอจิก 1 และใช้ลอจิก 0 ผ่านตัวต้านทานจำกัดกระแสไปยังแคโทดของส่วนเฉพาะจาก a ไปยัง g



รูปที่ 5.2 โครงสร้างของจอแสดงผลแบบเจ็ดส่วนรูปแบบแคโทดร่วมและแอโนดร่วม

ตารางที่ 5.1 ตารางความจริงของตัวเลขสำหรับจอแสดงผลแบบเจ็ดส่วน

Decimal Digit	Individual Segments Illuminated						
	a	b	c	d	e	f	g
0	1	1	1	1	1	1	0
1	0	1	1	0	0	0	0
2	1	1	0	1	1	0	1
3	1	1	1	1	0	0	1
4	0	1	1	0	0	1	1
5	1	0	1	1	0	1	1
6	1	0	1	1	1	1	1
7	1	1	1	0	0	0	0
8	1	1	1	1	1	1	1
9	1	1	1	1	0	1	1

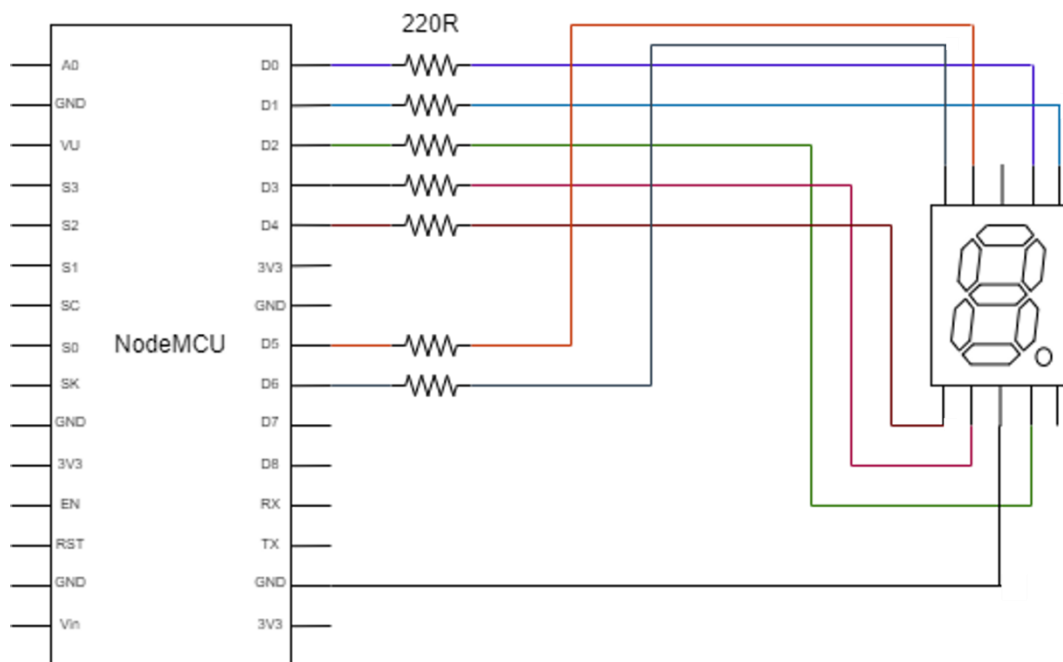
Lab 5.1: การทำงานของจอแสดงผลแบบเจ็ดส่วน

อุปกรณ์ที่ต้องใช้

- บอร์ด NodeMCU 1 ตัว
- จอแสดงผลแบบเจ็ดส่วนแบบแคโทดร่วม 1 หลัก 1 ตัว
- ตัวต้านทานปรับค่าได้ขนาด 220Ω 7 ตัว
- สาย Jumper
- Breadboard 1 อัน

ขั้นตอนในการทำ

- ต่อยังวงจรดังที่แสดงในรูป



รูปที่ 5.3 วงจรของแลบที่ 5.1

- พิมพ์โค้ดต่อไปนี้แล้วทำการ Verify โค้ด แล้วอัปโหลดโค้ดลงบอร์ด

```
const int segmentPins[] = {16, 5, 4, 0, 2, 14, 12};

const byte numbers[10][7] = {
  {1,1,1,1,1,1,0},
  {0,1,1,0,0,0,0},
  {1,1,0,1,1,0,1},
  {1,1,1,1,0,0,1},
  {0,1,1,0,0,1,1},
  {1,0,1,1,0,1,1},
  {1,0,1,1,1,1,1},
  {1,1,1,0,0,0,0},
  {1,1,1,1,1,1,1},
  {1,1,1,1,0,1,1}
};

void setup() {
  for(int i=0; i<7; i++) {
    pinMode(segmentPins[i], OUTPUT);
  }
}

void loop() {
  for(int n=0; n<10; n++) {
    for(int i=0; i<7; i++) {
      digitalWrite(segmentPins[i], numbers[n][i]);
    }
    delay(1000);
  }
}
```

ผลการทำงานหลังจากอัปโหลดโค้ดลงบอร์ด

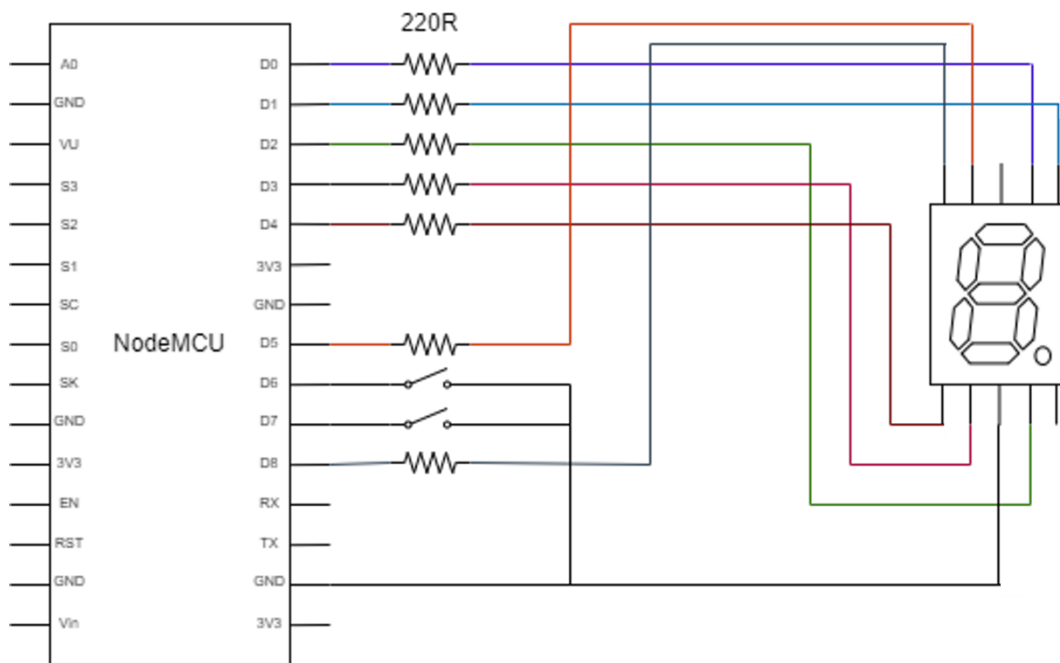
Lab 5.2: การควบคุมจอแสดงผลแบบเจ็ดส่วนด้วยสวิตช์

อุปกรณ์ที่ต้องใช้

- บอร์ด NodeMCU 1 ตัว
- จอแสดงผลแบบเจ็ดส่วนแบบแคโทดร่วม 1 หลัก 1 ตัว
- ตัวต้านทาน 220 Ω 7 ตัว
- สวิตช์กดติดปล่อยดับ 2 ตัว
- สาย Jumper
- Breadboard 1 อัน

ขั้นตอนในการทำ

- ต่อดังที่แสดงในรูป



รูปที่ 5.4 วงจรของแลบที่ 5.2

- พิมพ์โค้ดต่อไปนี้อย่างถูกต้องแล้วทำการ Verify โค้ด แล้วอัปโหลดโค้ดลงบอร์ด (พิมพ์โค้ดในไฟล์เดียวกัน)

```
const int segmentPins[] = {16, 5, 4, 0, 2, 14, 15};

const byte numbers[10][7] = {
  {1,1,1,1,1,1,0},
  {0,1,1,0,0,0,0},
  {1,1,0,1,1,0,1},
  {1,1,1,1,0,0,1},
  {0,1,1,0,0,1,1},
  {1,0,1,1,0,1,1},
  {1,0,1,1,1,1,1},
  {1,1,1,0,0,0,0},
  {1,1,1,1,1,1,1},
  {1,1,1,1,0,1,1}
};

const int buttonUp = D7;
const int buttonDown = D6;

int currentNumber = 0;

void displayNumber(int num) {
  for (int i = 0; i < 7; i++) {
    digitalWrite(segmentPins[i], numbers[num][i]);
  }
}
```

```

void setup() {
  for (int i = 0; i < 7; i++) {
    pinMode(segmentPins[i], OUTPUT);
  }

  pinMode(buttonUp, INPUT_PULLUP);
  pinMode(buttonDown, INPUT_PULLUP);

  displayNumber(currentNumber);
}

void loop() {
  if (digitalRead(buttonUp) == LOW) {
    delay(200); // debounce
    currentNumber++;
    if (currentNumber > 9) currentNumber = 0;
    displayNumber(currentNumber);
    while (digitalRead(buttonUp) == LOW);
  }

  if (digitalRead(buttonDown) == LOW) {
    delay(200); // debounce
    currentNumber--;
    if (currentNumber < 0) currentNumber = 9;
    displayNumber(currentNumber);
    while (digitalRead(buttonDown) == LOW);
  }
}

```

ผลการทำงานหลังจากอัปโหลดโค้ดลงบอร์ด

Lab 6

การขัดการทำงาน (Interrupt)

การขัดการทำงาน (Interrupt) คือ การร้องขอให้ตัวประมวลผลหยุดทำงานในขณะที่กำลังดำเนินการอยู่ โดยทำการเก็บค่าที่กำลังดำเนินการอยู่ล่าสุดไว้ในหน่วยความจำ และเปลี่ยนไปทำงานคำสั่งที่ถูกร้องขอเข้ามา เมื่อดำเนินการเสร็จ ระบบจะกลับไปประมวลผลงานเดิมที่ค้างอยู่ ซึ่งสามารถนำไปประยุกต์ใช้งานได้หลายรูปแบบ ทำให้ระบบมีประสิทธิภาพในการทำงานโดยไม่สูญเสียเวลามาก ซึ่งการขัดการทำงานสามารถแบ่งได้เป็น 2 ประเภท คือ

1. External Interrupt: การขัดการทำงานจากภายนอก เช่น การเปลี่ยนสถานะลอจิกของพอร์ตใดพอร์ตหนึ่ง
2. Internal Interrupt: การขัดการทำงานจากภายใน เช่น การขัดการทำงานที่เกิดจากตัวนับเวลา (Timer)

โดยสามารถควบคุมการขัดการทำงานได้ 2 รูปแบบ คือ

1. Disable Interrupt: การควบคุมให้คอนโทรลเลอร์ไม่ตอบสนองต่อการขัดการทำงาน เมื่อเกิดการขัดการทำงาน คอนโทรลเลอร์จะปล่อยผ่าน
2. Enable Interrupt: การควบคุมให้คอนโทรลเลอร์ตอบสนองต่อการขัดการทำงาน

เงื่อนไขในการขัดการทำงานมี 3 รูปแบบ คือ

1. CHANGE: การเปลี่ยนสถานะจาก 0 เป็น 1 หรือจาก 1 เป็น 0
2. RISING: การเปลี่ยนสถานะจาก 0 เป็น 1
3. FALLING: การเปลี่ยนสถานะจาก 1 เป็น 0

ซึ่งบอร์ด ESP8266 สามารถตั้งการขัดการทำงานได้ทุกขา GPIO ยกเว้นขา GPIO16 และมีคำสั่งที่ใช้สำหรับการขัดการทำงาน 2 คำสั่ง คือ

1. attachInterrupt() ใช้สำหรับการเริ่มตรวจสอบ Interrupt
2. detachInterrupt() ใช้สำหรับการยกเลิกตรวจสอบ Interrupt

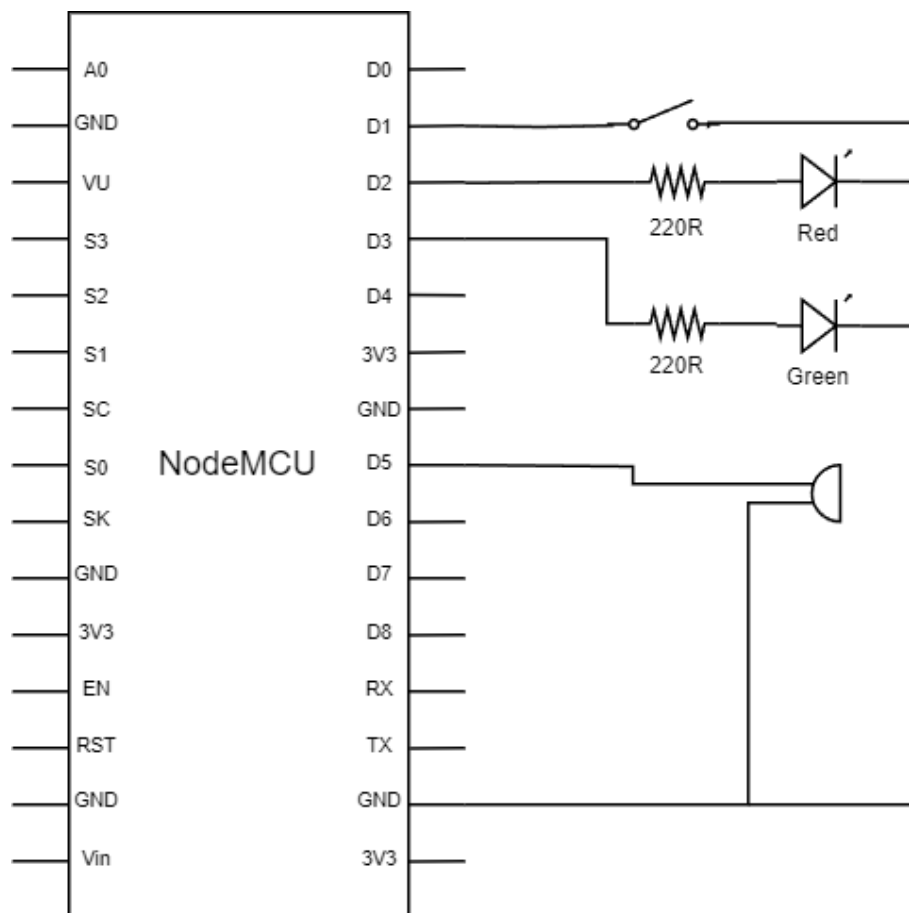
Lab 6.1: การทำงานของวงจรการขัดการทำงาน

อุปกรณ์ที่ต้องใช้

- บอร์ด NodeMCU 1 ตัว
- ตัวต้านทานขนาด 220Ω 2 ตัว
- หลอด LED สีแดง 1 ตัว
- หลอด LED สีเขียว 1 ตัว
- Active Buzzer 1 ตัว
- สวิตช์กดติดปล่อยดับ 1 ตัว
- สาย Jumper
- Breadboard 1 อัน

ขั้นตอนในการทำ

- ต่อดังต่อไปนี้ดังที่แสดงในรูป



รูปที่ 6.1 วงจรของแลบที่ 6.1

- พิมพ์โค้ดต่อไปนี้อย่างถูกต้องแล้วทำการ Verify โค้ด แล้วอัปโหลดโค้ดลงบอร์ด (พิมพ์โค้ดในไฟล์เดียวกัน)

```
const int buttonPin = D1;
const int redLED = D2;
const int greenLED = D3;
const int buzzer = D5;

volatile bool changeFlag = false;
volatile int lastButtonState = HIGH;

void ICACHE_RAM_ATTR handleInterrupt() {
    int currentState = digitalRead(buttonPin);
    if (currentState != lastButtonState) {
        changeFlag = true;
        lastButtonState = currentState;
    }
}

void setup() {
    pinMode(buttonPin, INPUT_PULLUP);
    pinMode(redLED, OUTPUT);
    pinMode(greenLED, OUTPUT);
    pinMode(buzzer, OUTPUT);

    attachInterrupt(digitalPinToInterrupt(buttonPin),
        handleInterrupt, CHANGE);
    Serial.begin(115200);
}
```

```

void loop() {
  if (changeFlag) {
    changeFlag = false;
    Serial.println("Interrupt: CHANGE");
    int state = lastButtonState;

    digitalWrite(buzzer, HIGH); // ควรตั้งเลย์ทันทึ
    delay(1000);
    digitalWrite(buzzer, LOW);

    if (state == LOW) {
      digitalWrite(redLED, HIGH);
      Serial.println("Interrupt: FALLING");
      delay(200);
      digitalWrite(redLED, LOW);
    } else {
      digitalWrite(greenLED, HIGH);
      Serial.println("Interrupt: RISING");
      delay(200);
      digitalWrite(greenLED, LOW);
    }
  }
}

```

ผลการทำงานหลังจากอัปโหลดโค้ดลงบอร์ด

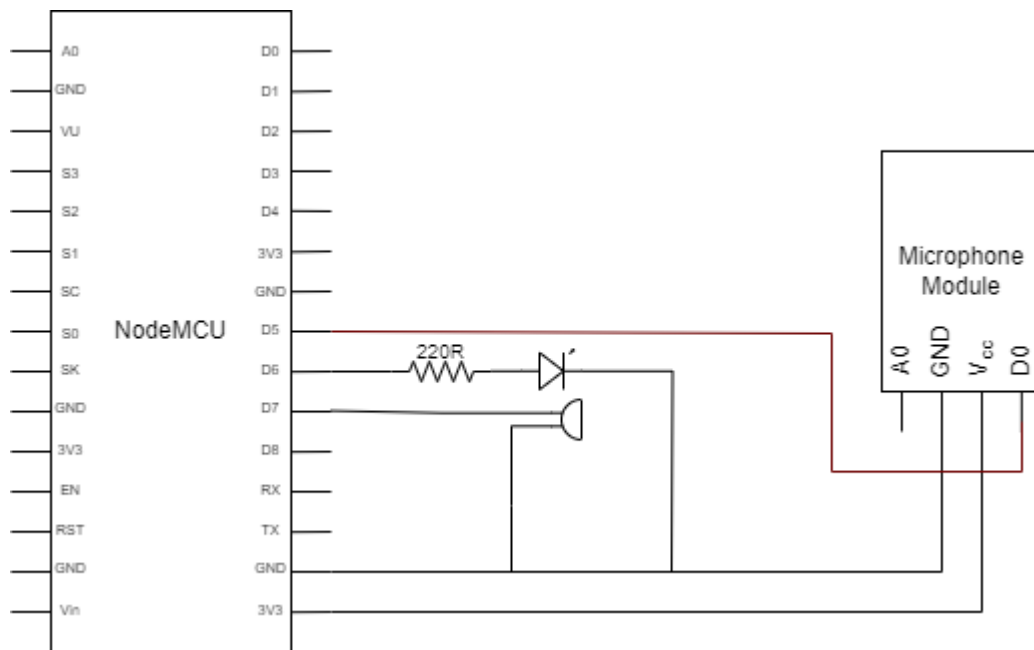
Lab 6.2: วงจรตรวจจับเสียง

อุปกรณ์ที่ต้องใช้

- บอร์ด NodeMCU 1 ตัว
- ตัวต้านทานขนาด 220Ω 1 ตัว
- หลอด LED 1 ตัว
- Passive Buzzer 1 ตัว
- Microphone Module 1 ตัว
- สาย Jumper
- Breadboard 1 อัน

ขั้นตอนในการทำ

- ต่อยังวงจรดังที่แสดงในรูป



รูปที่ 6.2 วงจรของแลบที่ 6.2

- พิมพ์โค้ดต่อไป่นี้แล้วทำการ Verify โค้ด แล้วอัปโหลดโค้ดลงบอร์ด (พิมพ์โค้ดในไฟล์เดียวกัน)

```
const int micPin = D5;
const int ledPin = D6;
const int buzzer = D7;

volatile bool soundDetected = false;
int toneFreq = 1000;

void ICACHE_RAM_ATTR handleSound() {
    soundDetected = true;
}

void setup() {
    pinMode(micPin, INPUT);
    pinMode(ledPin, OUTPUT);

    attachInterrupt(digitalPinToInterrupt(micPin), handleSound, FALLING);
    Serial.begin(115200);

    tone(buzzer, toneFreq);
}

void loop() {
    if (soundDetected) {
        soundDetected = false;

        toneFreq = (toneFreq == 1000) ? 1500 : 1000;
        tone(buzzer, toneFreq);

        Serial.print("Sound detected! Changed frequency to ");
        Serial.println(toneFreq);

        digitalWrite(ledPin, HIGH);
        delay(200);
        digitalWrite(ledPin, LOW);
    }

    delay(10);
}
```

ผลการทำงานหลังจากอัปโหลดโค้ดลงบอร์ด

Lab 7

การแสดงผลผ่านโมดูลจอ

สำหรับระบบ IoT การแสดงผลผ่านจอ LCD นั้นได้รับความนิยมและนำไปใช้อย่างกว้างขวาง ซึ่งปัจจุบันมีโมดูลจอ LCD ที่นิยมใช้อยู่ 2 ประเภท คือ

1. Character LCD เป็นจอที่ใช้แสดงผลตัวอักษร ซึ่งมีหลายขนาดที่วางขายตามท้องตลาด เช่น โมดูลจอ 16x2 หรือ 1602 Character LCD เป็นต้น
2. Graphic LCD หรือ OLED เป็นจอที่ใช้ในการแสดงผลเป็นตัวอักษรและภาพกราฟิก ซึ่งมีความละเอียดและสามารถแสดงเป็นสีได้ โดยระบุขนาดเป็นจำนวนจุด (pixel) เช่น โมดูลจอ OLED 128x64 0.96" คือ โมดูลจอ OLED ที่มีจำนวนจุดแนวนอน 128 จุด และแนวตั้ง 64 จุด โดยมีขนาดความยาวหน้าจอ 0.96 นิ้ว เป็นต้น

การสื่อสารระหว่างโมดูลจอ Character LCD กับไมโครคอนโทรลเลอร์ ตามปกตินั้นจะเป็นการส่งข้อมูลแบบขนาน (Parallel Transmission) ซึ่งต้องใช้ขาทั้งหมด 16 ขาในการสื่อสาร ซึ่งเป็นวิธีที่ไม่เป็นที่นิยมเนื่องจากมีจำนวนสายที่ค่อนข้างมาก จึงทำให้เกิดอุปกรณ์เสริมเพื่อทำการแปลงการสื่อสารเป็นรูปแบบอนุกรม (Serial Transmission) โดยทำการเชื่อมต่อผ่านโปรโตคอล I2C ส่วนการสื่อสารระหว่างโมดูลจอ OLED กับไมโครคอนโทรลเลอร์นั้นสามารถสื่อสารผ่านโปรโตคอล SPI หรือ I2C ก็ได้ เนื่องจากชิปในโมดูลนั้นรองรับทั้ง 2 โปรโตคอล



(ก)



(ข)

รูปที่ 7.1 โมดูลจอ LCD (ก) 1602 Character LCD (ข) Graphic LCD หรือ OLED ที่มา:

<https://www.thaieasyelec.com/category/lcd-oled/96>

รูปที่ 7.2 วงจรของแลบที่ 7.1

- พิมพ์โค้ดต่อไปนี้แล้วทำการ Verify โค้ด แล้วอัปโหลดโค้ดลงบอร์ด

```
#include <Wire.h>

void setup() {
  Wire.begin();
  Serial.begin(115200);
  while(!Serial);
}

void loop() {
  Serial.println("\nScanning...");

  for (byte i = 8; i < 120; i++) {
    Wire.beginTransmission(i);

    if (Wire.endTransmission() == 0) {
      Serial.print("LCD Module Address: ");
      Serial.print(i, DEC);
      Serial.print(" (0x");
      Serial.print(i, HEX);
      Serial.print(")\n");
    }
  }
  delay(5000);
}
```

- เปิด Serial Monitor เพื่อดูผลลัพธ์ที่เกิดขึ้น

ผลการทำงานหลังจากอัปโหลดโค้ดลงบอร์ด

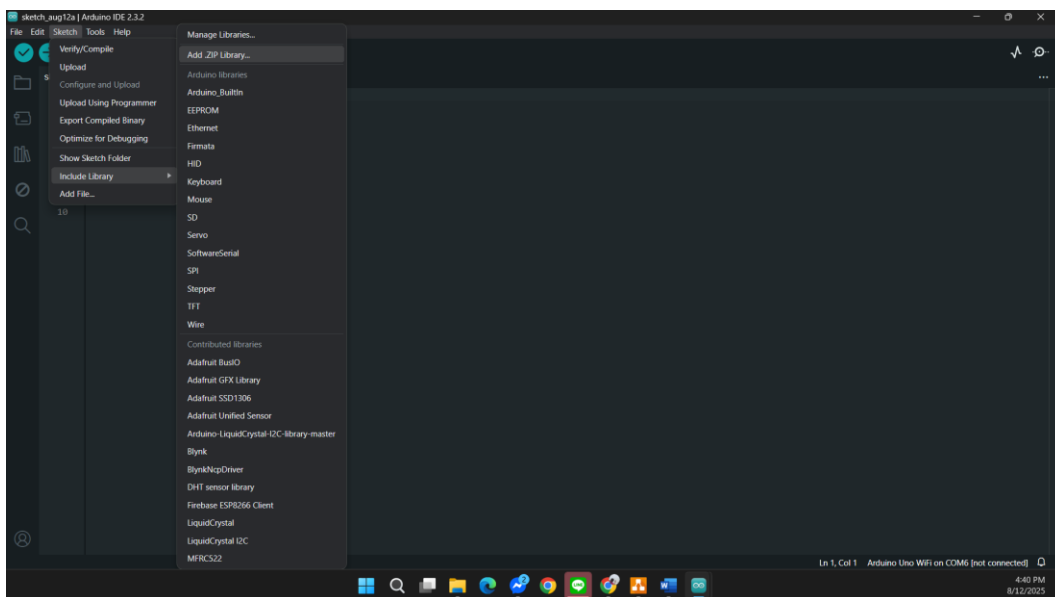
Lab 7.2: การแสดงผลบนจอ LCD

อุปกรณ์ที่ต้องใช้

- บอร์ด NodeMCU 1 ตัว
- โมดูลจอ 1603 Character LCD 1 ตัว
- สาย Jumper
- Breadboard 1 อัน

ขั้นตอนในการทำ

- ใช้วงจรเดิมจากแล็บที่ 7.1
- ติดตั้ง Library LiquidCrystal_I2C โดยดาวน์โหลดไฟล์ Library ที่แนบมาใน Google Classroom จากนั้นทำการติดตั้งโดยไปที่เมนู Sketch -> Include Library -> Add .Zip Library -> เลือกไฟล์ที่ดาวน์โหลดมาเพื่อทำการติดตั้ง



รูปที่ 7.3 การติดตั้ง Library LiquidCrystal I2C

- พิมพ์โค้ดต่อไปนี้แล้วทำการ Verify โค้ด แล้วอัปโหลดโค้ดลงบอร์ด (ตรวจสอบ)

```

#include <LiquidCrystal_I2C.h>
#include <Wire.h>

LiquidCrystal_I2C lcd(putYopurAddress, 16, 2);
byte heart[8] = {0x00, 0x0A, 0x1F, 0x1F, 0x0E, 0x04,
0x00, 0x00};

void setup() {
  lcd.init();
}

void loop() {
  lcd.backlight();
  lcd.display();
  lcd.home();

  delay(1000);
  lcd.print("We love");
  delay(2000);

  lcd.setCursor(0,1);
  lcd.print("IoT");
  delay(2000);

  lcd.createChar(0,heart);
  lcd.setCursor(4, 1);
  lcd.write(0);
  delay(2000);

  for (int i = 0; i <= 15; i++) {
    lcd.scrollDisplayLeft();
    delay(200);
  }
  for (int i = 0; i <=32; i++) {
    lcd.scrollDisplayRight();
    delay(200);
  }
  for (int i = 0; i <= 16; i++) {
    lcd.scrollDisplayLeft();
    delay(200);
  }

  delay(2000);
  lcd.noDisplay();
  delay(500);
  lcd.display();
  delay(500);
  lcd.noDisplay();
  delay(500);
  lcd.display();
  delay(500);
  lcd.noDisplay();
  delay(500);
  lcd.display();
  delay(1000);

  lcd.clear();
  delay(1000);
  lcd.noDisplay();
  delay(1000);
  lcd.noBacklight();
  delay(2000);
}

```

ผลการทำงานหลังจากอัปโหลดโค้ดลงบอร์ด

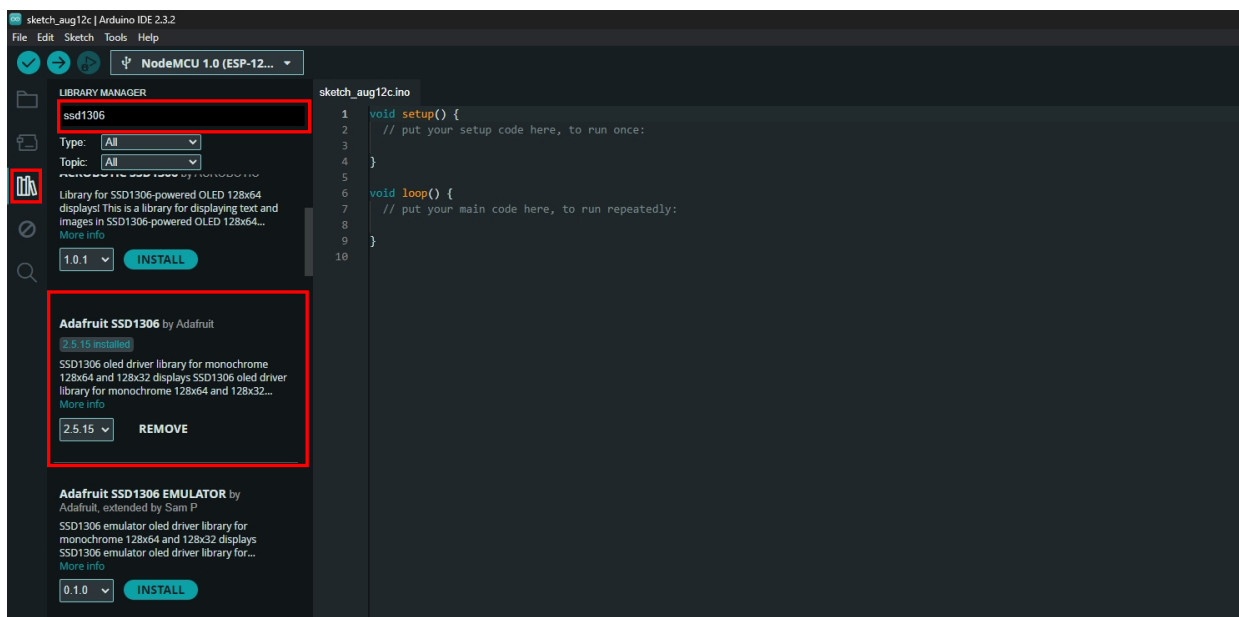
Lab 7.3: การแสดงผลโมดูลจอ OLED

อุปกรณ์ที่ต้องใช้

- บอร์ด NodeMCU 1 ตัว
- โมดูลจอ OLED 1 ตัว
- สาย Jumper
- Breadboard 1 อัน

ขั้นตอนในการทำ

- ติดตั้ง Library Adafruit SSD1306 โดยเข้าไปใน Arduino IDE -> Library Manager -> Search “ssd1306” -> เลือกติดตั้ง Library ที่พัฒนาโดย Adafruit



รูปที่ 7.4 การติดตั้ง Library Adafruit SSD1306

- ไปที่ตำแหน่งเก็บไฟล์ไลบรารี ไปที่โฟลเดอร์ไลบรารี Adafruit_SSD1306 เปิดไฟล์ Adafruit_SSD1306.h ด้วย Notepad
- แก้ไขไฟล์ดังนี้แล้วทำการเซฟ
 - `//#define SSD1306_128_64 -> #define SSD1306_128_64`
 - `#define SSD1306_128_32 -> //#define SSD1306_128_32`

```
File Edit View
* This is part of for Adafruit's SSD1306 library for monochrome
* OLED displays: http://www.adafruit.com/category/63_98
*
* These displays use I2C or SPI to communicate. I2C requires 2 pins
* (SCL+SDA) and optionally a RESET pin. SPI requires 4 pins (MOSI, SCK,
* select, data/command) and optionally a reset pin. Hardware SPI or
* 'bitbang' software SPI are both supported.
*
* Adafruit invests time and resources providing this open source code,
* please support Adafruit and open-source hardware by purchasing
* products from Adafruit!
*
* Written by Limor Fried/Ladyada for Adafruit Industries, with
* contributions from the open source community.
*
* BSD license, all text above, and the splash screen header file,
* must be included in any redistribution.
*
*/

#ifndef _Adafruit_SSD1306_H_
#define _Adafruit_SSD1306_H_

// ONE of the following three lines must be #defined:
// #define SSD1306_128_64 ///< DEPRECATED: old way to specify 128x64 screen
// #define SSD1306_128_32 ///< DEPRECATED: old way to specify 128x32 screen
// #define SSD1306_96_16  ///< DEPRECATED: old way to specify 96x16 screen
// This establishes the screen dimensions in old Adafruit SSD1306 sketches
// (NEW CODE SHOULD IGNORE THIS, USE THE CONSTRUCTORS THAT ACCEPT WIDTH
// AND HEIGHT PARAMETERS)

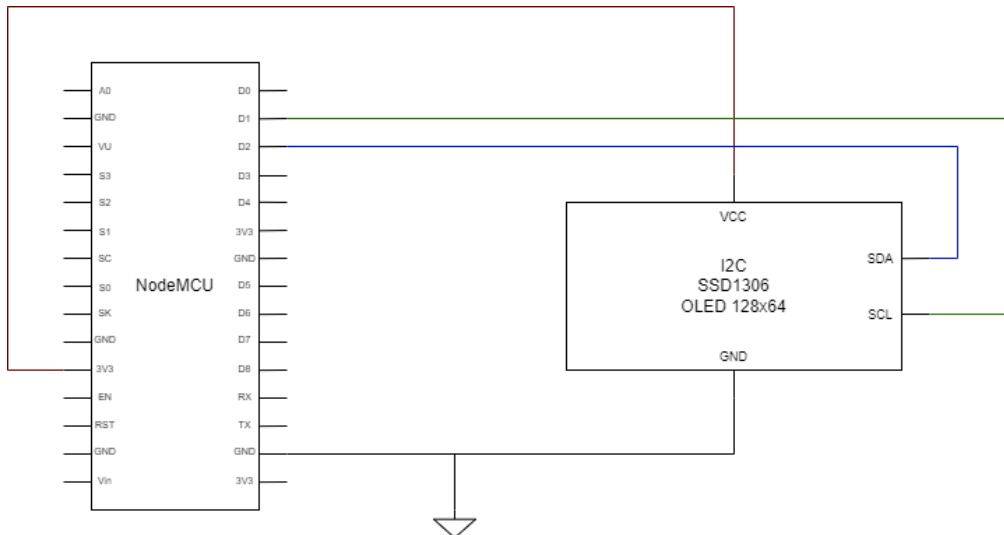
// Uncomment to disable Adafruit splash logo
// #define SSD1306_NO_SPLASH

#ifdef ARDUINO_STM32_FEATHER
typedef class HardwareSPI SPIClass;
#endif

#include <Adafruit_GFX.h>
#include <SPI.h>
#include <Wire.h>
```

รูปที่ 7.5 แก้ไข Header file ของ Adafruit SSD1306 เพื่อให้เข้ากับโมดูล OLED

- ต่อดังที่แสดงในรูป



รูปที่ 7.6 วงจรสำหรับแลบที่ 7.3

- ทำการตรวจสอบ Address ของ โมดูลจอ OLED จาก Lab 7.1

- พิมพ์โค้ดต่อไปนี้พร้อมเติมค่า Address ที่ได้ทำการหาจากขั้นตอนก่อนหน้านี้ แล้วทำการ Verify โค้ด แล้วอัปโหลดโค้ดลงบอร์ด

```
#include <Adafruit_SSD1306.h>
#include <Adafruit_GFX.h>
#include <SPI.h>
#include <Wire.h>

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET -1
#define SCREEN_ADDRESS 0x3C // Address ตรงนี้
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT,
&Wire, OLED_RESET);

void setup() {
  display.begin(SSD1306_SWITCHCAPVCC, SCREEN_ADDRESS);
  display.clearDisplay();

  display.setTextSize(2);
  display.setTextColor(WHITE);
  display.setCursor(0, 20);
  display.println("Hello");

  display.display();
}

void loop() {
}
```

ผลการทำงานหลังจากอัปโหลดโค้ดลงบอร์ด

- ทำการแก้ไขโค้ดในบรรทัดที่ 17 โดยเปลี่ยนจาก WHITE เป็น BLACK, WHITE แล้วทำการ Verify โค้ด แล้วอัปโหลดโค้ดลงบอร์ด

ผลการทำงานหลังจากอัปโหลดโค้ดลงบอร์ด

- จากโค้ดด้านบน แก้ไขโค้ดในบรรทัดที่ 17 กลับมาเป็น WHITE เพิ่มเติมโค้ดต่อไปนี้โดยเริ่มใส่ที่บรรทัดที่ 22 ซึ่งจะต้องอยู่ในฟังก์ชัน void setup() จากนั้นทำการ Verify โค้ด แล้วอัปโหลดโค้ดลงบอร์ด

```
delay(2000);  
display.clearDisplay();  
  
display.setTextSize(1);  
display.setCursor(0, 28);  
display.print("0x");  
display.print(0xE6, HEX);  
display.print("(HEX) = ");  
display.print(0xE6, DEC);  
display.println("(DEC)");  
display.display();
```

ผลการทำงานหลังจากอัปโหลดโค้ดลงบอร์ด

- จากโค้ดด้านบน เพิ่มเติมโค้ดต่อไปนี้โดยเริ่มใส่ที่บรรทัดที่ 33 ซึ่งจะต้องอยู่ในฟังก์ชัน void setup() จากนั้นทำการ Verify โค้ด แล้วอัปโหลดโค้ดลงบอร์ด

```
delay(2000);
display.clearDisplay();

display.setCursor(0, 0);
display.println("Show");
display.println("Display");
display.println("Scrolling!");
display.display();
display.startscrollright(0x00, 0x07);
delay(4500);
display.stopscroll();
delay(1000);
display.startscrollleft(0x00, 0x07);
delay(4500);
display.stopscroll();
delay(1000);
display.startscrollright(0x00, 0x00);
delay(6000);
```

ผลการทำงานหลังจากอัปโหลดโค้ดลงบอร์ด

- จากโค้ดด้านบน เพิ่มเติมโค้ดต่อไปนี้โดยเริ่มใส่ที่บรรทัดที่ 51 ซึ่งจะต้องอยู่ในฟังก์ชัน void setup() จากนั้นทำการ Verify โค้ด แล้วอัปโหลดโค้ดลงบอร์ด

```
display.clearDisplay();

display.setTextColor(WHITE);
display.setCursor(0, 0);
display.println("Filled Circle");
display.fillCircle(64, 35, 20, WHITE);

display.display();
delay(2000);
display.clearDisplay();
```

ผลการทำงานหลังจากอัปโหลดโค้ดลงบอร์ด

Lab 7.4: การแสดงภาพกราฟิกบนโมดูลจอ OLED

ขั้นตอนในการทำ

- ใช้วงจรเดิมจากแลบที่ 7.3
- ไปที่เว็บไซต์ <https://javal.github.io/image2cpp/>
- เปิดรูปภาพที่ต้องการจะแสดงบนโมดูลจอ OLED แล้วตั้งค่าดังที่แสดงในรูป

image2cpp

image2cpp is a simple tool to change images into byte arrays (or arrays back into an images) for use with (monochrome) displays such as OLEDs on your Arduino or Raspberry Pi. It was originally made to work with the Adafruit OLED library (for which you can find an example sketch for Arduino [here](#)) but has been expanded by the community to be useful in all kind of (embedded) projects.

More info (and credits) can be found in the [Github repository](#). This is also where you can report any [issues](#) you might come across.

Did you find this tool useful? Feel free to support my open source software on Github

Sponsor

1. Select image

All processing is done locally in your browser; your images are not uploaded or stored anywhere online.

aesthetic-c-...av5u06.jpg
aesthetic-computer-4k-megrsbupwyav5u06.jpg

1. Paste byte array

128 x 64 px

Read images appear at step 3 below

2. Image Settings

Canvas size(s): 128 x 64 glyph

Background color: ☒ White ☐ Black ☐ Transparent

Invert image colors ☐

Dithering:

Brightness / alpha threshold:
0 - 255; if the brightness of a pixel is above the given level the pixel becomes white, otherwise they become black. When using alpha, opaque and transparent are used instead.

Scaling:

Center image: ☐ horizontally ☐ vertically
Centering the image only works when using a canvas larger than the original image.

Rotate image: degrees

Flip image: ☐ horizontally ☐ vertically

3. Preview

4. Output

Code output format:

Adds some extra Arduino code around the output for easy copy-paste into [this example](#). If multiple images are loaded, generates a byte array for each and appends a counter to the identifier.

Identifier/Prefix:

Draw mode:
If your image looks all messed up on your display, like the image below, try using a different mode.

TEST

Swap bits in byte: ☐ swap
Useful when working with the u8g2 library.

รูปที่ 7.7 การตั้งค่ารูปเพื่อแปลงเป็นโค้ดภาษาซี

- กดปุ่ม Generate Code แล้วกดปุ่ม Copy Output เพื่อคัดลอกโค้ดที่ทำการแปลงแล้ว

Generate code
Copy Output
Download as binary file (.bin)

```

0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x01, 0x07, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x03, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xf8,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0xff, 0xff, 0xff, 0xff, 0x1f, 0xff, 0xe1, 0xc0,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x1f, 0xff, 0xff, 0xf0, 0x0f, 0xff, 0x81, 0x80,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x07, 0xff, 0xff, 0xe0, 0x0f, 0x1f, 0x81, 0x80
);

// Array of all bitmaps for convenience. (Total bytes used to store images in PROGMEM = 1040)
const int testallArray_LEN = 1;
const unsigned char* testallArray[1] = {
    testaesthetic_computer_4k_megrsbupwyav5u06
};

```

รูปที่ 7.8 ตัวอย่างรูปที่ถูกแปลงเป็นโค้ดแล้ว

- พิมพ์โค้ดต่อไปนั้แล้วนำโค้ดรูปภาพที่ได้จากเว็บไซต์มาใส่ในบรรทัดที่ 13 และแก้ (identifier) ให้เป็นชื่อตามที่โค้ดกำหนด จากนั้นทำการ Verify โค้ด แล้วอัปโหลดโค้ดลงบอร์ด

```

#include <SPI.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET -1
#define SCREEN_ADDRESS ใส่ค่า Address ของโมดูลจอ OLED
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);

// นำโค้ดรูปภาพที่แปลงมาใส่ตั้งแต่บรรทัดที่ 13

void setup() {
    display.begin(SSD1306_SWITCHCAPVCC, SCREEN_ADDRESS);
    display.clearDisplay();

    //identifier ได้จากการแปลงรูปภาพแล้วจะอยู่ในบรรทัดต่อจาก const unsigned char* testallArray[1]
    display.drawBitmap(0, 0, (identifier), 128, 64, WHITE);

    display.display();
}

void loop() {
    // put your main code here, to run repeatedly:
}

```

ผลการทำงานหลังจากอัปโหลดโค้ดลงบอร์ด

Lab 8

การทำงานของเซ็นเซอร์ประเภทต่างๆ (1)

เซ็นเซอร์เป็นอุปกรณ์ที่ใช้ในการตรวจจับสภาพแวดล้อมต่าง ๆ ที่อยู่รอบตัว เปรียบเทียบได้กับประสาทในการรับรู้ของมนุษย์ ซึ่งโดยทั่วไปจะเป็นข้อมูลแบบแอนาล็อก ทำให้ต้องมีการแปลงสัญญาณจากแอนาล็อกเป็นดิจิทัลจึงจะสามารถนำมาประมวลผลในระบบ IoT ได้ ซึ่งในการทดลองนี้จะนำเอาเซ็นเซอร์พื้นฐานมาใช้ในการเรียนรู้

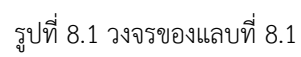
Lab 8.1: การประยุกต์ใช้งาน RFID Module

อุปกรณ์ที่ต้องใช้

- บอร์ด NodeMCU 1 ตัว
- โมดูล RFID-RC522 1 ตัว
- Active Buzzer 1 ตัว
- ตัวต้านทานขนาด 220Ω 2 ตัว
- LED สีแดง 1 ตัว
- LED สีเขียว 1 ตัว
- สาย Jumper
- Breadboard 1 อัน

ขั้นตอนในการทำ

- ที่ Arduino IDE คลิกเมนู Tools -> Manage Library...
- ค้นหา mfrc522 แล้วทำการติดตั้งไลบรารี MFRC522 by GitHub Community
- ต่อดวงจรมุ่งที่แสดงในรูปแบบ



รูปที่ 8.1 วงจรของแลบที่ 8.1

- พิมพ์โค้ดต่อไปนี้แล้วทำการ Verify โค้ด แล้วอัปโหลดโค้ดลงบอร์ด

```
#include <SPI.h>
#include <MFRC522.h>

#define RST_PIN D1
#define SS_PIN D2

MFRC522 rfid(SS_PIN, RST_PIN);

void setup() {
  Serial.begin(115200);
  SPI.begin();
  rfid.PCD_Init();
  Serial.println("Scan your RFID card to see UID...");
}

void loop() {
  if (!rfid.PICC_IsNewCardPresent() || !rfid.PICC_ReadCardSerial()) {
    return;
  }

  Serial.print("UID tag : ");
  for (byte i = 0; i < rfid.uid.size; i++) {
    Serial.print("0x");
    if (rfid.uid.uidByte[i] < 0x10) Serial.print("0");
    Serial.print(rfid.uid.uidByte[i], HEX);
    if (i != rfid.uid.size - 1) Serial.print(", ");
  }
  Serial.println();

  delay(1000);
  rfid.PICC_HaltA();
  rfid.PCD_StopCrypto1();
}
```

- เปิด Serial Monitor
- นำบัตรสีขาวแตะที่โมดูล RFID แล้วดูผลลัพธ์ที่เกิดขึ้น

ผลการทำงานหลังจากอัปโหลดโค้ดลงบอร์ด

- นำผลลัพธ์ที่ได้จากโค้ดก่อนหน้ามาใส่ในโค้ดต่อไปนี้แล้วทำการ Verify โค้ด แล้วอัปโหลดโค้ดลงบอร์ด

```

#include <SPI.h>
#include <MFRC522.h>

#define RST_PIN    D1
#define SS_PIN     D2

#define GREEN_LED  D3
#define RED_LED    D4
#define BUZZER_PIN D0

MFRC522 rfid(SS_PIN, RST_PIN);

byte authorizedUID[] = { Put your UID };

void setup() {
  Serial.begin(115200);
  SPI.begin();
  rfid.PCD_Init();
  Serial.println("RFID Access Control Ready");

  pinMode(GREEN_LED, OUTPUT);
  pinMode(RED_LED, OUTPUT);
  pinMode(BUZZER_PIN, OUTPUT);

  digitalWrite(GREEN_LED, LOW);
  digitalWrite(RED_LED, LOW);
  digitalWrite(BUZZER_PIN, LOW);
}

void loop() {
  if (!rfid.PICC_IsNewCardPresent() ||
  !rfid.PICC_ReadCardSerial()) {
    return;
  }

  Serial.print("UID tag : ");
  for (byte i = 0; i < rfid.uid.size; i++) {
    Serial.print(rfid.uid.uidByte[i] < 0x10 ? " 0" : " ");
    Serial.print(rfid.uid.uidByte[i], HEX);
  }
  Serial.println();

  if (isAuthorized(rfid.uid.uidByte)) {
    Serial.println("Access Granted");
    digitalWrite(GREEN_LED, HIGH);
    digitalWrite(RED_LED, LOW);
    digitalWrite(BUZZER_PIN, LOW);
  } else {
    Serial.println("Access Denied");
    digitalWrite(GREEN_LED, LOW);
    digitalWrite(RED_LED, HIGH);

    // บuzzer 2 ครั้ง
    for (int i = 0; i < 2; i++) {
      digitalWrite(BUZZER_PIN, HIGH);
      delay(150);
      digitalWrite(BUZZER_PIN, LOW);
      delay(150);
    }

    delay(2000);
    digitalWrite(GREEN_LED, LOW);
    digitalWrite(RED_LED, LOW);
    digitalWrite(BUZZER_PIN, LOW);

    rfid.PICC_HaltA();
    rfid.PCD_StopCrypto1();
  }

  bool isAuthorized(byte *uid) {
    for (byte i = 0; i < 4; i++) {
      if (uid[i] != authorizedUID[i]) return false;
    }
    return true;
  }
}

```

- เปิด Serial Monitor
- นำบัตรสีขาวแตะที่โมดูล RFID แล้วดูผลลัพธ์ที่เกิดขึ้น
- นำคีย์การ์ดสีน้ำเงินแตะที่โมดูล RFID แล้วดูผลลัพธ์ที่เกิดขึ้น

ผลการทำงานหลังจากอัปโหลดโค้ดลงบอร์ด

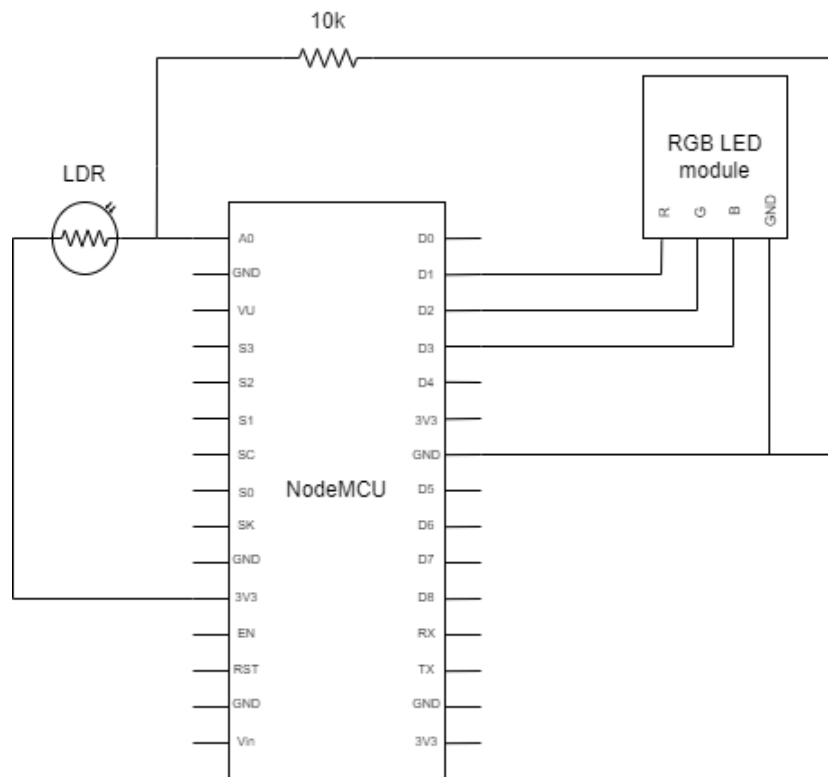
Lab 8.2: การตอบสนองต่อแสงด้วยเซ็นเซอร์ LDR

อุปกรณ์ที่ต้องใช้

- บอร์ด NodeMCU 1 ตัว
- เซ็นเซอร์ LDR 1 ตัว
- ตัวต้านทานขนาด $10k\Omega$ 1 ตัว
- โมดูล LED RGB 1 ตัว
- สาย Jumper
- Breadboard 1 อัน

ขั้นตอนในการทำ

- ต่อยังวงจรดังที่แสดงในรูป



รูปที่ 8.2 วงจรของแลบที่ 8.2

- พิมพ์โค้ดต่อไปนี้แล้วทำการ Verify โค้ด แล้วอัปโหลดโค้ดลงบอร์ด

```
#define LDR_PIN A0
#define R_PIN D1
#define G_PIN D2
#define B_PIN D3

void setup() {
  Serial.begin(115200);
  pinMode(R_PIN, OUTPUT);
  pinMode(G_PIN, OUTPUT);
  pinMode(B_PIN, OUTPUT);
}

void loop() {
  int lightLevel = analogRead(LDR_PIN);
  Serial.print("Light Level: ");
  Serial.println(lightLevel);

  digitalWrite(R_PIN, LOW);
  digitalWrite(G_PIN, LOW);
  digitalWrite(B_PIN, LOW);

  if (lightLevel > 800) {
    digitalWrite(R_PIN, HIGH);
  } else if (lightLevel > 500) {
    digitalWrite(G_PIN, HIGH);
  } else if (lightLevel > 200) {
    digitalWrite(B_PIN, HIGH);
  } else {
    digitalWrite(R_PIN, HIGH);
    digitalWrite(G_PIN, HIGH);
    digitalWrite(B_PIN, HIGH);
  }

  delay(1000);
}
```

- เปิด Serial Monitor
- ทดลองเพิ่ม-ลดแสงที่ตกกระทบเซ็นเซอร์ LDR แล้วดูผลลัพธ์ที่เกิดขึ้น

ผลการทำงานหลังจากอัปโหลดโค้ดลงบอร์ด

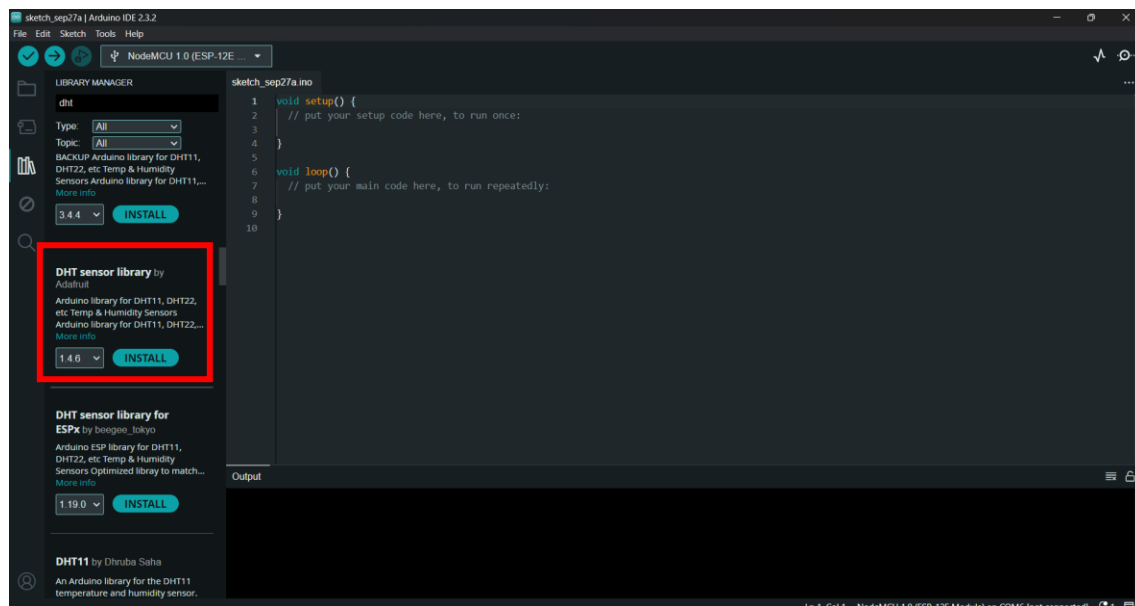
Lab 8.3: การวัดอุณหภูมิและความชื้นในอากาศด้วยโมดูล DHT11

อุปกรณ์ที่ต้องใช้

- บอร์ด NodeMCU 1 ตัว
- โมดูลจอ 1603 Character LCD 1 ตัว
- โมดูล DHT 1 ตัว
- สาย Jumper
- Breadboard 1 อัน

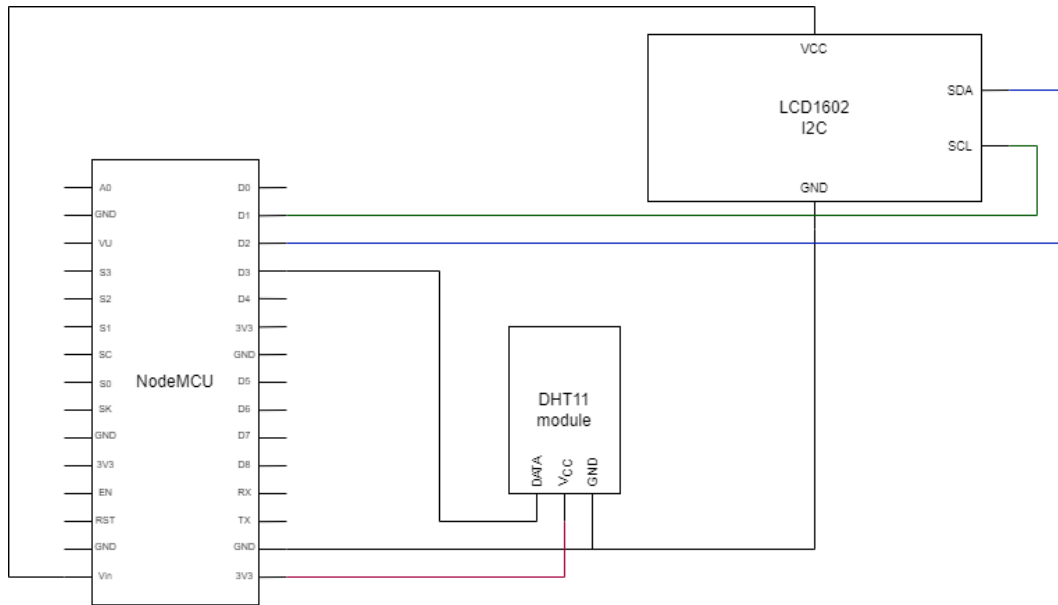
ขั้นตอนในการทำ

- ไปที่ Tools -> Manage Libraries... ค้นหาคำว่า dht แล้วทำการติดตั้งไลบรารี dht ของ Adafruit ที่ชื่อว่า DHT sensor library by Adafruit



รูปที่ 8.3 การติดตั้ง DHT sensor library

- ต่อวงจรดังที่แสดงในรูป



รูปที่ 8.4 วงจรสำหรับแลบที่ 8.3

- ทำการตรวจสอบ Address ของ โมดูลจอ LCD จาก Lab 7.1
- พิมพ์โค้ดต่อไปนี้พร้อมเติมค่า Address ที่ได้ทำการหาจากขั้นตอนก่อนหน้านี้ แล้วทำการ Verify โค้ด แล้วอัปโหลดโค้ดลงบอร์ด

```

#include <Arduino.h>
#include <DHT.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

#define DHTPIN    D3
#define DHTTYPE   DHT11

const float TEMP_THRESHOLD = 30.0;

DHT dht(DHTPIN, DHTTYPE);
LiquidCrystal_I2C lcd(putyouraddress, 16, 2);

void setup() {
  Serial.begin(115200);
  dht.begin();

  lcd.init();
  lcd.backlight();
  lcd.setCursor(0, 0);
  lcd.print("System Ready");
  delay(2000);
  lcd.clear();
}

void loop() {
  float t = dht.readTemperature();
  float h = dht.readHumidity();

  if (isnan(t) || isnan(h)) {
    Serial.println(F("DHT read fail"));
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Sensor Error");
    delay(2000);
    return;
  }

  Serial.print("Temp: "); Serial.print(t);
  Serial.print(" °C, Humidity: "); Serial.println(h);

  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("T:"); lcd.print(t, 1); lcd.print((char)223); lcd.print("C ");
  lcd.print("H:"); lcd.print(h, 0); lcd.print("%");

  lcd.setCursor(0, 1);
  if (t > TEMP_THRESHOLD) {
    lcd.print("Status: HOT ");
  } else {
    lcd.print("Status: COOL");
  }

  delay(2000);
}

```

- ตรวจสอบผลจากหน้าจอ และทดลองเป่าใส่เซ็นเซอร์เพื่อดูการเปลี่ยนแปลง

ผลการทำงานหลังจากอัปโหลดโค้ดลงบอร์ด

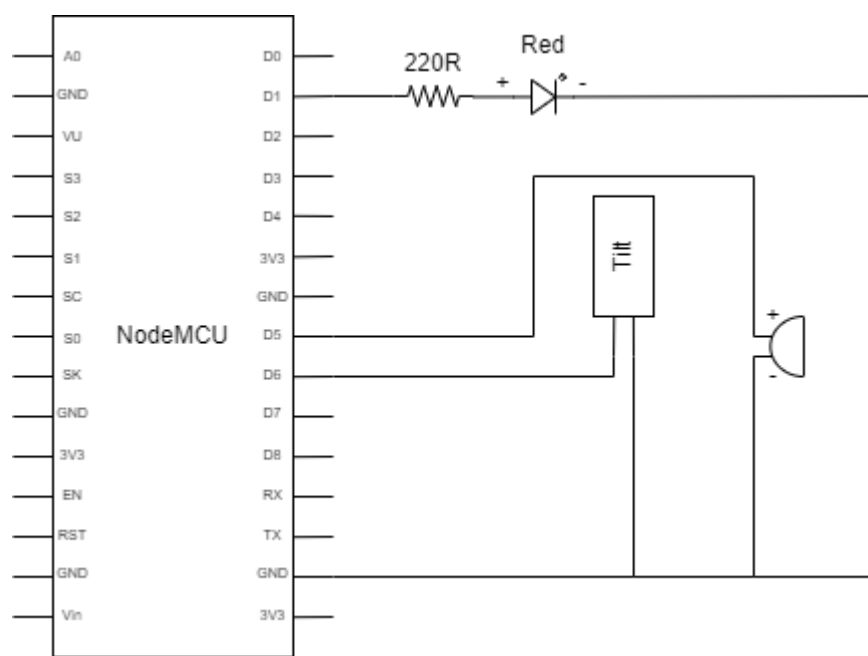
Lab 8.4: จำลองการตรวจจับแรงสั่นสะเทือนด้วยเซ็นเซอร์ Tilt/Vibration

อุปกรณ์ที่ต้องใช้

- บอร์ด NodeMCU 1 ตัว
- Tilt/Vibration sensor 1 ตัว
- Active Buzzer 1 ตัว
- ตัวต้านทานขนาด 220Ω 1 ตัว
- LED สีแดง 1 ตัว
- สาย Jumper
- Breadboard 1 อัน

ขั้นตอนในการทำ

- ต่อดังที่แสดงในรูป



รูปที่ 8.5 วงจรสำหรับแลบที่ 8.4

- พิมพ์โค้ดต่อไปนี้ แล้วทำการ Verify โค้ด แล้วอัปโหลดโค้ดลงบอร์ด

```

#define TILT_PIN    D6
#define LED_PIN     D1
#define BUZZER_PIN  D5

const uint16_t VIBE_THRESHOLD = 20;

const uint16_t BUZZ_ON_MS  = 200;
const uint16_t BUZZ_OFF_MS = 800;

volatile unsigned long pulseCount = 0;

void IRAM_ATTR onPulse() {
    static unsigned long lastUs = 0;
    unsigned long nowUs = micros();
    if (nowUs - lastUs > 3000) {
        pulseCount++;
        lastUs = nowUs;
    }
}

void setup() {
    Serial.begin(115200);

    pinMode(LED_PIN, OUTPUT);
    digitalWrite(LED_PIN, LOW);

    pinMode(BUZZER_PIN, OUTPUT);
    digitalWrite(BUZZER_PIN, LOW);

    pinMode(TILT_PIN, INPUT_PULLUP);
    attachInterrupt(digitalPinToInterrupt(TILT_PIN), onPulse, FALLING);

    Serial.println(F("SW-520D Interrupt + Buzzer ready"));
}

void loop() {
    static unsigned long lastWindowMs = 0;
    static unsigned long lastBuzzToggle = 0;
    static bool buzzing = false;
    static bool overThreshold = false;

    unsigned long now = millis();

    if (now - lastWindowMs >= 1000) {
        lastWindowMs = now;

        noInterrupts();
        unsigned long c = pulseCount;
        pulseCount = 0;
        interrupts();

        overThreshold = (c >= VIBE_THRESHOLD);
        Serial.print("Vibration count/s: "); Serial.print(c);
        Serial.print(" => "); Serial.println(overThreshold ? "STRONG" :
"normal");

        digitalWrite(LED_PIN, overThreshold ? HIGH : LOW);

        buzzing = false;
        digitalWrite(BUZZER_PIN, LOW);
        lastBuzzToggle = now;
    }

    if (overThreshold) {
        if (!buzzing) {
            digitalWrite(BUZZER_PIN, HIGH);
            buzzing = true;
            lastBuzzToggle = now;
        } else {
            if (digitalRead(BUZZER_PIN) == HIGH && now - lastBuzzToggle >=
BUZZ_ON_MS) {
                digitalWrite(BUZZER_PIN, LOW);
                lastBuzzToggle = now;
            }
        }
    } else {
        digitalWrite(BUZZER_PIN, LOW);
    }
}

```

- เปิด Serial Monitor
- ทดลองเขย่าเซ็นเซอร์ เบา/แรง แล้วดูผลลัพธ์ที่เกิดขึ้น

ผลการทำงานหลังจากอัปโหลดโค้ดลงบอร์ด

- พิมพ์โค้ดต่อไปนี้แล้วทำการ Verify โค้ด แล้วอัปโหลดโค้ดลงบอร์ด

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

LiquidCrystal_I2C lcd(LCD_addreess, 16, 2);

#define TRIG D7
#define ECHO D6
#define RED_PIN D3
#define GREEN_PIN D4

long duration, distance;

void setup() {
  pinMode(TRIG, OUTPUT);
  pinMode(ECHO, INPUT);
  pinMode(RED_PIN, OUTPUT);
  pinMode(GREEN_PIN, OUTPUT);

  lcd.init();
  lcd.backlight();
  lcd.home();

  lcd.print("Check Distance");
  lcd.setCursor(0, 1);
  lcd.print("Ready");
  lcd.clear();
  delay(2000);
}

void loop() {
  digitalWrite(TRIG, LOW);
  delayMicroseconds(5);

  digitalWrite(TRIG, HIGH);
  delayMicroseconds(10);

  digitalWrite(TRIG, LOW);

  duration = pulseIn(ECHO, HIGH);
  distance = duration * 0.034 / 2;

  lcd.setCursor(0, 0);
  lcd.print("Distance: ");
  lcd.print(distance);
  lcd.print("cm. ");

  if (distance >= 15) {
    digitalWrite(GREEN_PIN, HIGH);
    digitalWrite(RED_PIN, LOW);
    lcd.setCursor(0, 1);
    lcd.print("Safe");
  } else {
    digitalWrite(GREEN_PIN, LOW);
    digitalWrite(RED_PIN, HIGH);
    lcd.setCursor(0, 1);
    lcd.print("Stop");
  }

  delay(500);
}
```

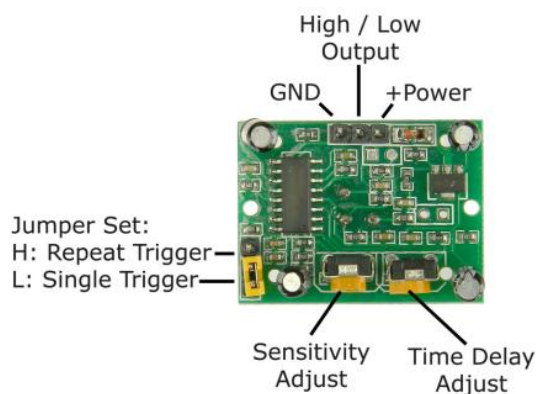
- ทดลองเอาวัตถุชิ้นบริเวณเซ็นเซอร์ในแนวตั้งฉากกับเซ็นเซอร์ แล้วขยับเข้าออก

ผลการทำงานหลังจากอัปโหลดโค้ดลงบอร์ด

Lab 9.2: ตรวจจับการเคลื่อนไหวด้วยโมดูล PIR Motion Sensor

อุปกรณ์ที่ต้องใช้

- บอร์ด NodeMCU 1 ตัว
- โมดูล PIR HC-SR501 1 ตัว
- Passive Buzzer 1 ตัว
- หลอด LED 1 หลอด
- ตัวต้านทานขนาด 220 Ω 1 อัน
- สาย Jumper
- Breadboard 1 อัน



รูปที่ 9.2 องค์ประกอบของโมดูล PIR HC-SR501

สำหรับ Sensitive Adjust กับ Time Delay Adjust เป็นตัวต้านทานที่สามารถหมุนเพื่อปรับค่าได้ โดยมีความหมายดังนี้

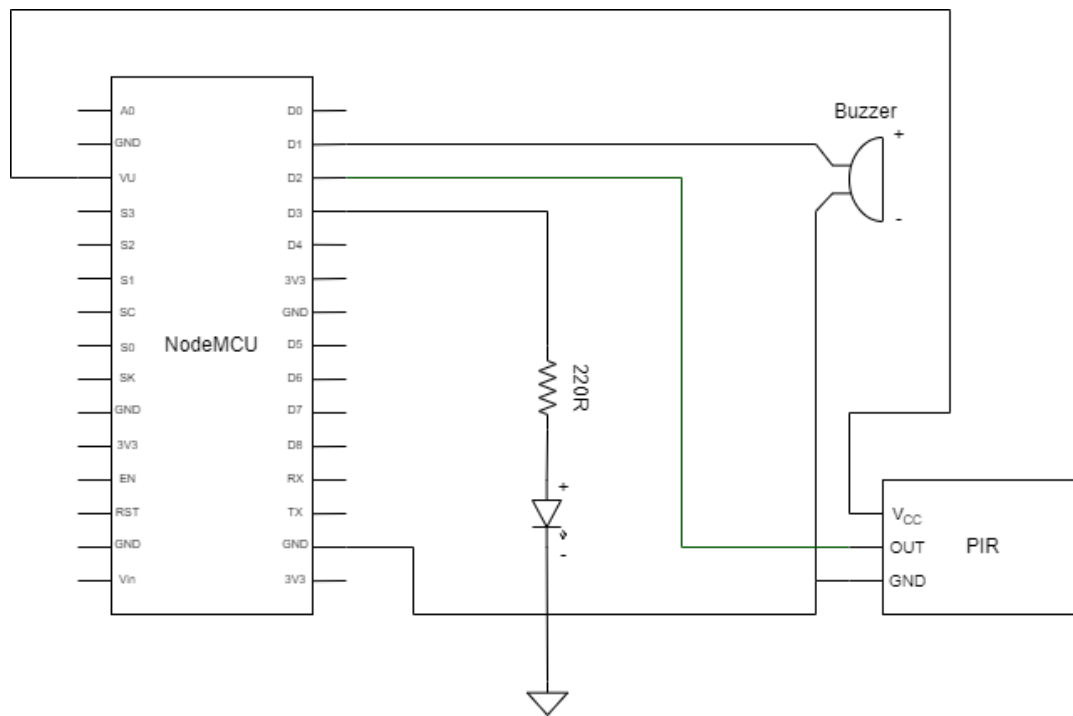
ตารางที่ 9.1 การปรับค่าความไวและการหน่วงเวลา

ตัวต้านทาน	หมุนตามเข็มนาฬิกา (Clockwise)	หมุนทวนเข็มนาฬิกา (Anti Clockwise)
Sensitive Adjust	เพิ่มค่าความไว ตรวจจับในรัศมีระยะ ประมาณ 7 เมตร	ลดค่าความไว ตรวจจับในรัศมีระยะ ประมาณ 3 เมตร
Time Delay Adjust	เพิ่มระยะเวลาคงสถานะของขา Digital OUT สูงสุดประมาณ 300 วินาที	ลดระยะเวลาคงสถานะของขา Digital OUT ต่ำสุดประมาณ 3 วินาที

Jumper Set ใช้ในการกำหนดโหมดในการทำงานของเซ็นเซอร์ ถ้ากำหนดเป็น H (Repeat Trigger) เมื่อเซ็นเซอร์ตรวจจับการเคลื่อนไหวอย่างต่อเนื่องจะคงสถานะ HIGH จนกว่าจะหยุดเคลื่อนไหวหรือพัลส์มีการตรวจจับ ส่วนถ้ากำหนดเป็น L (Single Trigger) เมื่อเซ็นเซอร์ตรวจจับการเคลื่อนไหว จะให้สถานะเอาต์พุต HIGH เพียงครั้งเดียว แล้วมีการหน่วงเวลาตามค่า Time Delay แล้วจึงคืนสถานะกลับมาเป็น LOW

ขั้นตอนในการทำ

- ตัวอย่างจริงดังที่แสดงในรูป



รูปที่ 9.3 วงจรของแลบที่ 9.2

- พิมพ์โค้ดต่อไปนี้นำทำการ Verify โค้ด แล้วอัปโหลดโค้ดลงบอร์ด

```

#define buzzerPin D1
#define pirPin D2
#define ledPin D3

int val = 0;

bool motionState = false;

void setup() {
  pinMode(buzzerPin, OUTPUT);
  pinMode(pirPin, INPUT);
  pinMode(ledPin, OUTPUT);
  Serial.begin(115200);
}

void loop() {
  val = digitalRead(pirPin);

  if (val == HIGH) {
    digitalWrite(ledPin, HIGH);
    alarm(500, 2000);

    delay(150);

    if (motionState == false) {
      Serial.println("Motion Detected");
      motionState = true;
    }
  } else {
    digitalWrite(ledPin, LOW);
    noTone(buzzerPin);

    delay(150);

    if (motionState == true) {
      Serial.println("No Motion");
      motionState = false;
    }
  }
}

void alarm (long duration, int freq) {
  tone(buzzerPin, freq);
  delay(duration);
  noTone(buzzerPin);
}

```

- ทดลองขยับร่างกายหรือโบกมือผ่านเซ็นเซอร์ สามารถปรับตัวด้านทานปรับค่าได้เพื่อปรับความไวของเซ็นเซอร์ได้

ผลการทำงานหลังจากอัปโหลดโค้ดลงบอร์ด

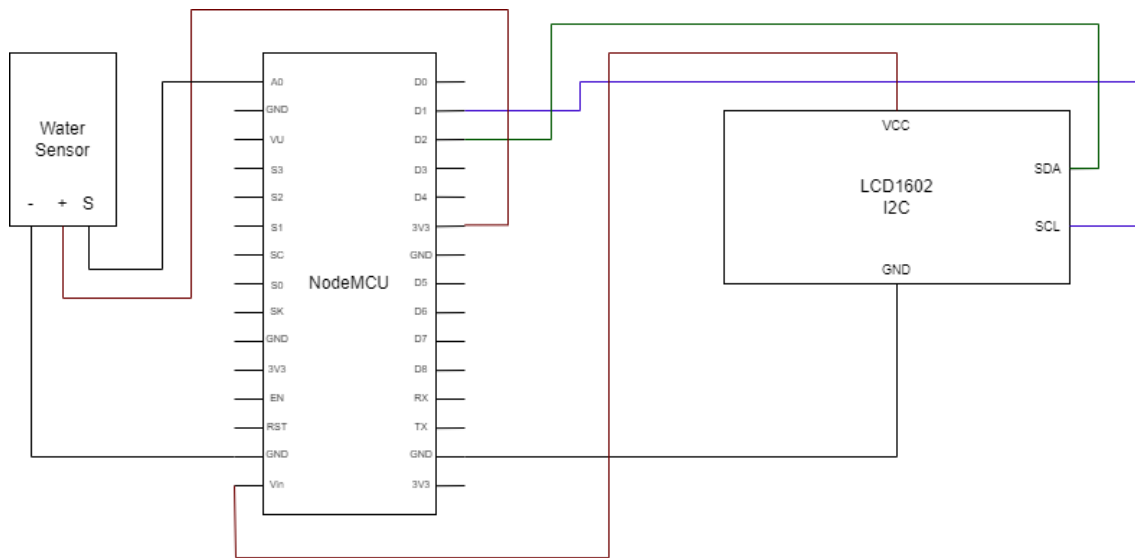
Lab 9.3: วัดระดับน้ำด้วย Water Sensor

อุปกรณ์ที่ต้องใช้

- บอร์ด NodeMCU 1 ตัว
- Water Sensor 1 ตัว
- โมดูลจอ 1603 Character LCD 1 ตัว
- สาย Jumper
- Breadboard 1 อัน

ขั้นตอนในการทำ

- ต่อดังที่แสดงในรูป



รูปที่ 9.4 วงจรของแลบที่ 9.3

- พิมพ์โค้ดต่อไปนี้แล้วทำการ Verify โค้ด แล้วอัปโหลดโค้ดลงบอร์ด

```

#include <Wire.h>
#include <LiquidCrystal_I2C.h>

LiquidCrystal_I2C lcd(LCD_address, 16, 2);

#define WATER_SENSOR_PIN A0

void setup() {
  Serial.begin(115200);

  Wire.begin(D2, D1);
  lcd.init();
  lcd.backlight();

  lcd.setCursor(0, 0);
  lcd.print("Water Level:");
}

void loop() {
  int sensorValue = analogRead(WATER_SENSOR_PIN);
  String level = "";

  if (sensorValue > 700) {
    level = "High";
  } else if (sensorValue > 400) {
    level = "Medium";
  } else {
    level = "Low";
  }

  Serial.print("Sensor: ");
  Serial.print(sensorValue);
  Serial.print(" | Level: ");
  Serial.println(level);

  lcd.setCursor(0, 1);
  lcd.print("                ");
  lcd.setCursor(0, 1);
  lcd.print("Level: " + level);

  delay(1000);
}

```

- เปิด serial monitor
- ทดลองนำเซนเซอร์จุ่มน้ำเพื่อสังเกตการเปลี่ยนแปลง

ผลการทำงานหลังจากอัปโหลดโค้ดลงบอร์ด

Lab 10

การพัฒนาแอปพลิเคชันเพื่อเชื่อมต่อและควบคุมอุปกรณ์ผ่านอินเทอร์เน็ต

ในการพัฒนาระบบ IoT การสื่อสารและควบคุมอุปกรณ์ในระยะไกลนับเป็นองค์ประกอบที่สำคัญที่ต้องพัฒนา ซึ่งจะทำให้การควบคุมผ่านการสื่อสารรูปแบบต่าง ๆ โดยใช้เครือข่ายอินเทอร์เน็ตที่ปัจจุบันใช้ในการควบคุมอุปกรณ์อย่างแพร่หลาย โดยอาศัย Cloud Server เป็นตัวกลางผ่านการพัฒนา Mobile Application บนสมาร์ตโฟน

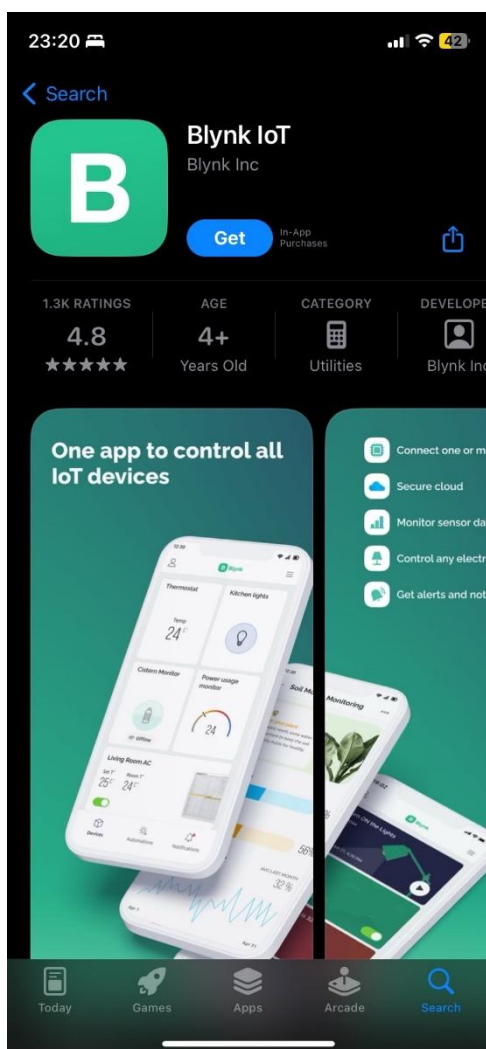
Blynk เป็นแพลตฟอร์มที่สามารถนำมาใช้ในการพัฒนาแอปพลิเคชันสำหรับการเชื่อมต่อและควบคุมอุปกรณ์ IoT ซึ่งแพลตฟอร์ม Blynk ประกอบไปด้วย 3 ส่วน คือ

1. Blynk Application: Mobile Application ที่รองรับทั้งระบบ Android และ iOS ใช้สำหรับเป็นแผงควบคุมอุปกรณ์ IoT ผ่านสมาร์ตโฟน
2. Blynk Server: Cloud Service ซึ่งเป็นตัวกลางในการสื่อสารระหว่างอุปกรณ์ IoT และ Mobile Application
3. Blynk Library: ไลบรารีสำหรับนำไปใช้ในการพัฒนาโค้ดเพื่อให้สามารถเชื่อมต่อระหว่างอุปกรณ์ IoT และ Mobile Application ได้

การเริ่มต้นการใช้งาน Blynk

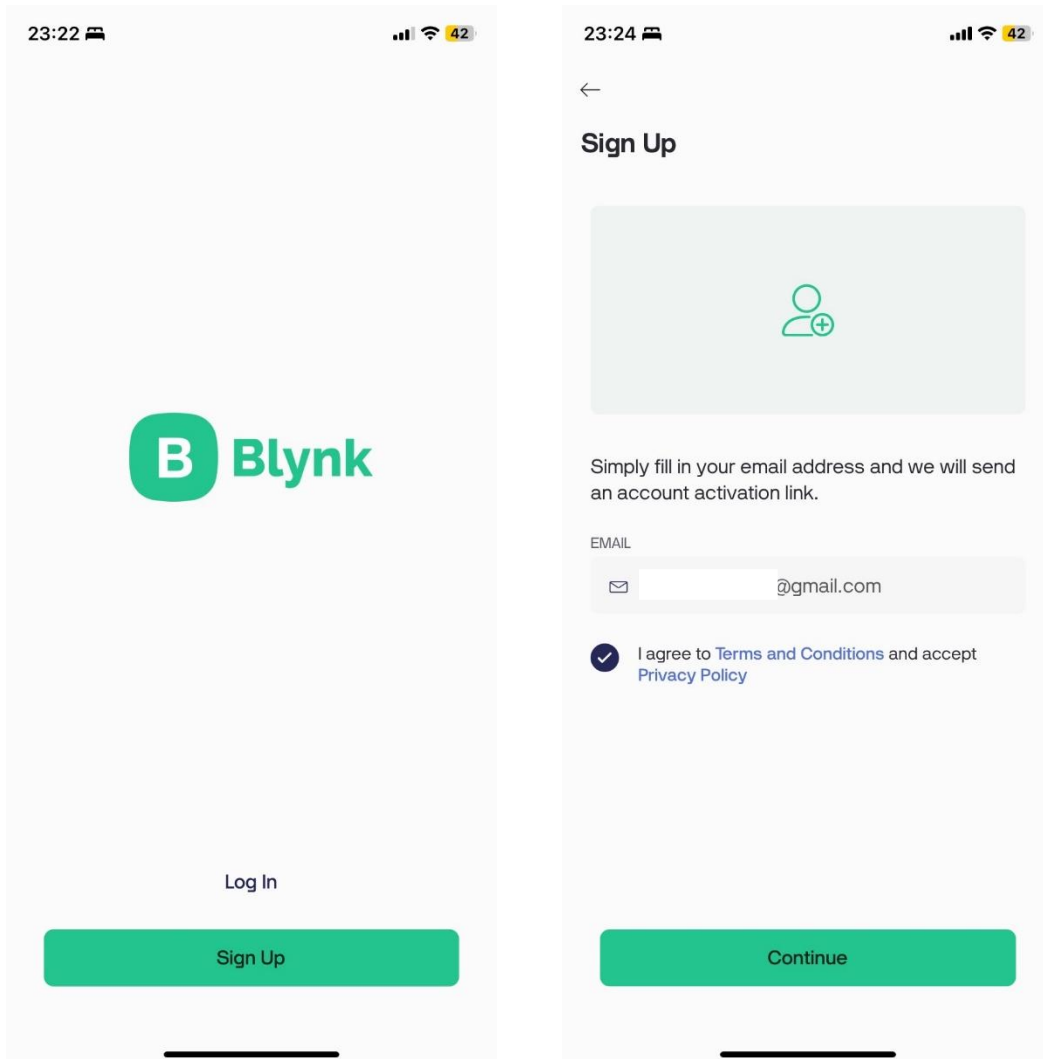
ในการเริ่มต้นการใช้งาน Blynk จะต้องดำเนินการตามขั้นตอนดังนี้

- ดาวน์โหลดและติดตั้ง Blynk App บนสมาร์ทโฟน โดยดาวน์โหลดผ่าน App Store (สำหรับ iOS) หรือ Play Store (สำหรับ Android)



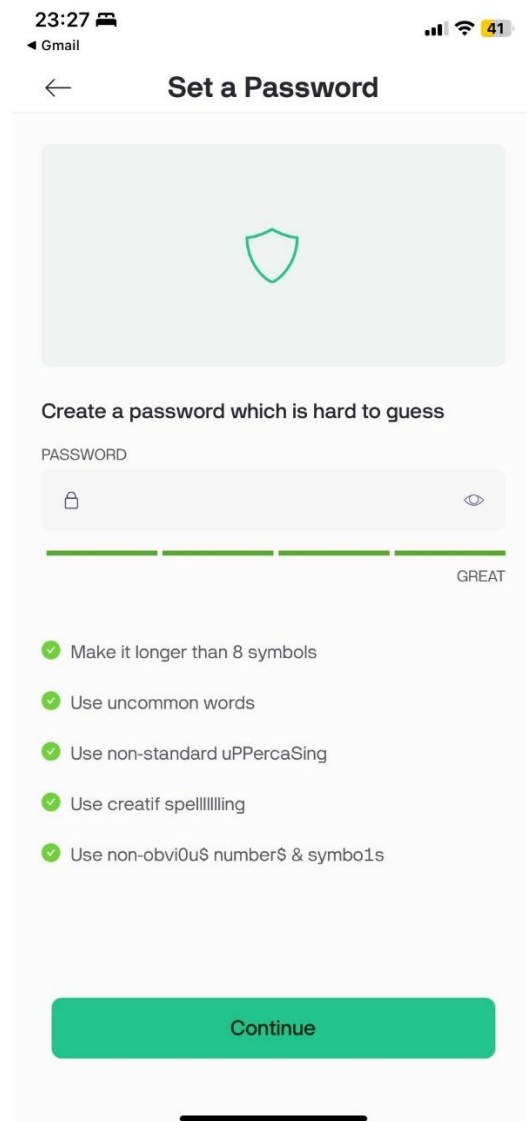
รูปที่ 10.1 แอปพลิเคชัน Blynk บน App Store

- ลงทะเบียน Account ผ่านอีเมล โดยกดที่ Sign Up กรอกอีเมล กดยอมรับนโยบายการทำงาน แล้วกด Continue



รูปที่ 10.2 ขั้นตอนการลงทะเบียนบนแอปพลิเคชัน Blynk

- เข้าไปยังอีเมลที่ใช้ในการลงทะเบียน จะได้รับอีเมลจาก Blynk ให้กด Create Password เพื่อสร้างรหัสผ่าน แล้วทำการตั้งรหัสผ่านให้ตรงกับเงื่อนไขที่แอปพลิเคชันกำหนด แล้วกด Continue



23:27 Gmail

Set a Password

Create a password which is hard to guess

PASSWORD

LOCK EYE

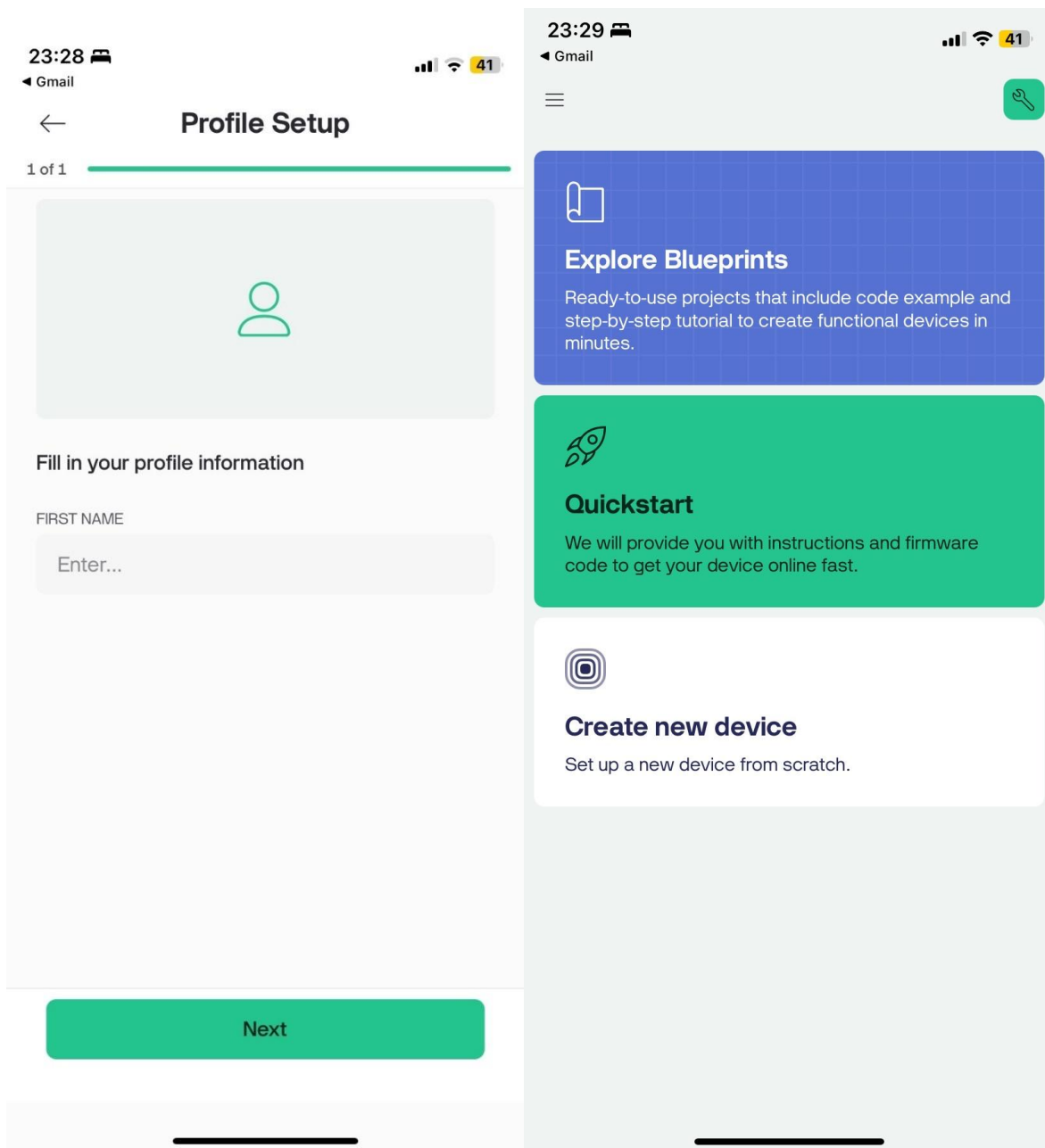
GREAT

- ✓ Make it longer than 8 symbols
- ✓ Use uncommon words
- ✓ Use non-standard uPPercaSing
- ✓ Use creatif spelllllllling
- ✓ Use non-obvi0u\$ number\$ & symbo1s

Continue

รูปที่ 10.3 กำหนดรหัสผ่านบนแอปพลิเคชัน Blynk

- ตั้งค่าชื่อ Profile ตามที่แอปพลิเคชันกำหนด แล้วกด Next เป็นการเสร็จการติดตั้งในสมาร์ทโฟน



รูปที่ 10.4 ตั้งชื่อ Profile เพื่อเข้าสู่แอปพลิเคชัน Blynk

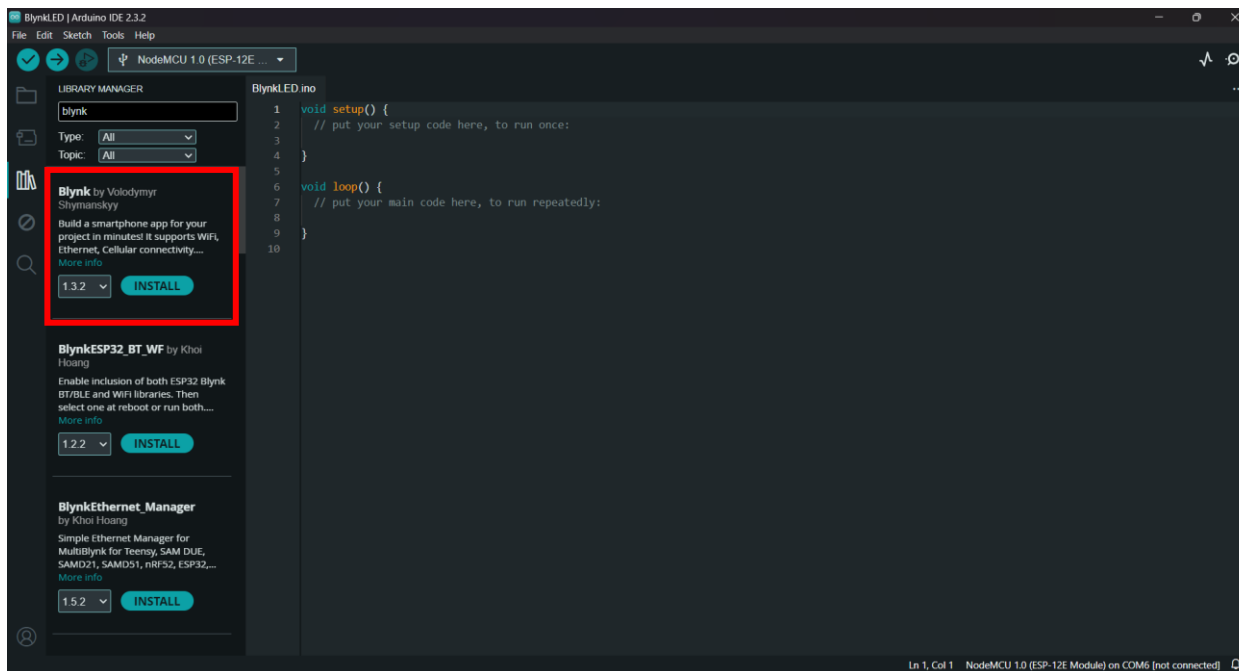
Lab 10.1: ระบบเปิด/ปิดหลอดไฟผ่าน Blynk

อุปกรณ์ที่ต้องใช้

- บอร์ด NodeMCU 1 ตัว
- หลอด LED 1 อัน
- ตัวต้านทานขนาด 220 Ω 1 อัน
- สาย Jumper
- Breadboard 1 อัน
- แอปพลิเคชัน Blynk

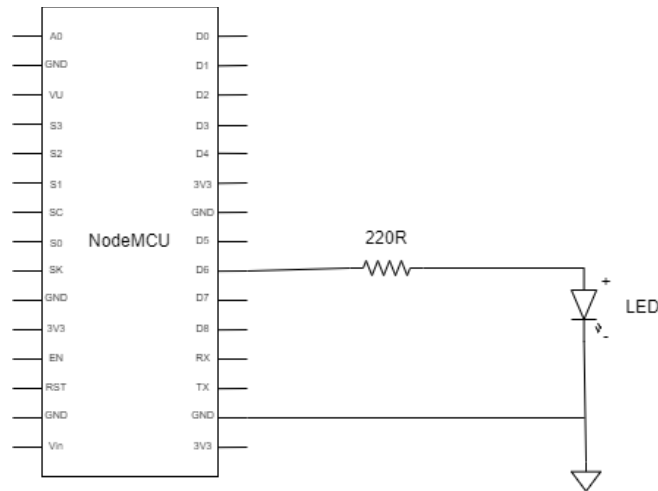
ขั้นตอนในการทำ

- ที่ Arduino IDE คลิกเมนู Tools -> Manage Library...
- ค้นหา blynk แล้วทำการติดตั้งไลบรารี blynk by Volodymyr Shymanskyy



รูปที่ 10.5 ไลบรารีสำหรับ blynk

- ตัวอย่างวงจรที่แสดงในรูป



รูปที่ 10.6 วงจรสำหรับแลบที่ 10.1

- พิมพ์โค้ดต่อไปนี้

```
#define BLYNK_TEMPLATE_ID          "YourTemplateID"
#define BLYNK_TEMPLATE_NAME        "Quickstart Device"
#define BLYNK_AUTH_TOKEN           "YourAuthToken"

#define BLYNK_PRINT Serial

#include <ESP8266WiFi.h>
#include <BlynkSimpleEsp8266.h>

#define ledPin D6

char ssid[] = "YourNetworkName";
char pass[] = "YourPassword";

BlynkTimer timer;

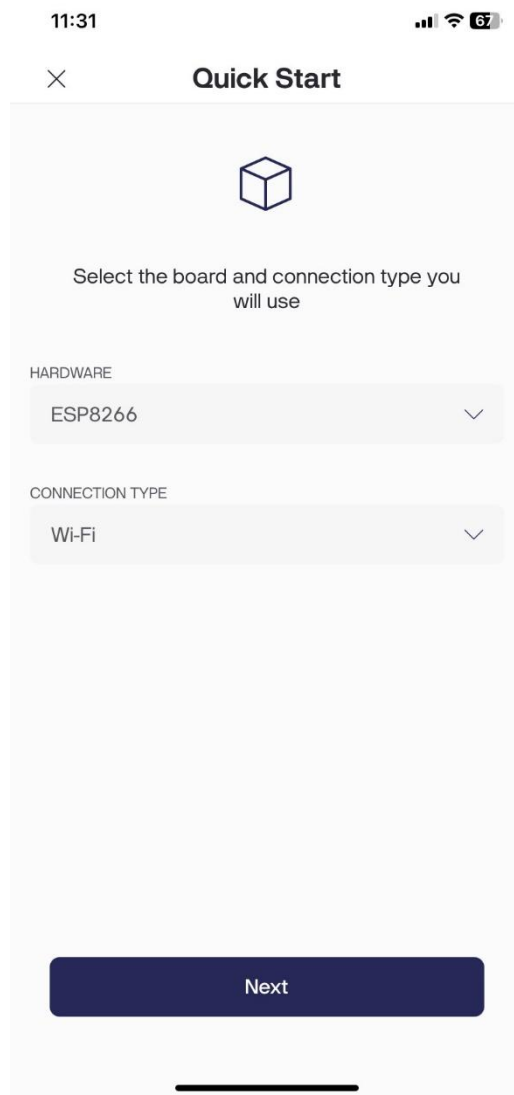
BLYNK_WRITE(V0)
{
  int pinValue = param.asInt();

  if (pinValue == 1) {
    digitalWrite(ledPin, HIGH);
    Serial.println("LED On");
  } else {
    digitalWrite(ledPin, LOW);
    Serial.println("LED Off");
  }
}

void setup()
{
  Serial.begin(115200);
  Blynk.begin(BLYNK_AUTH_TOKEN, ssid, pass);
  pinMode(ledPin, OUTPUT);
  digitalWrite(ledPin, LOW);
}

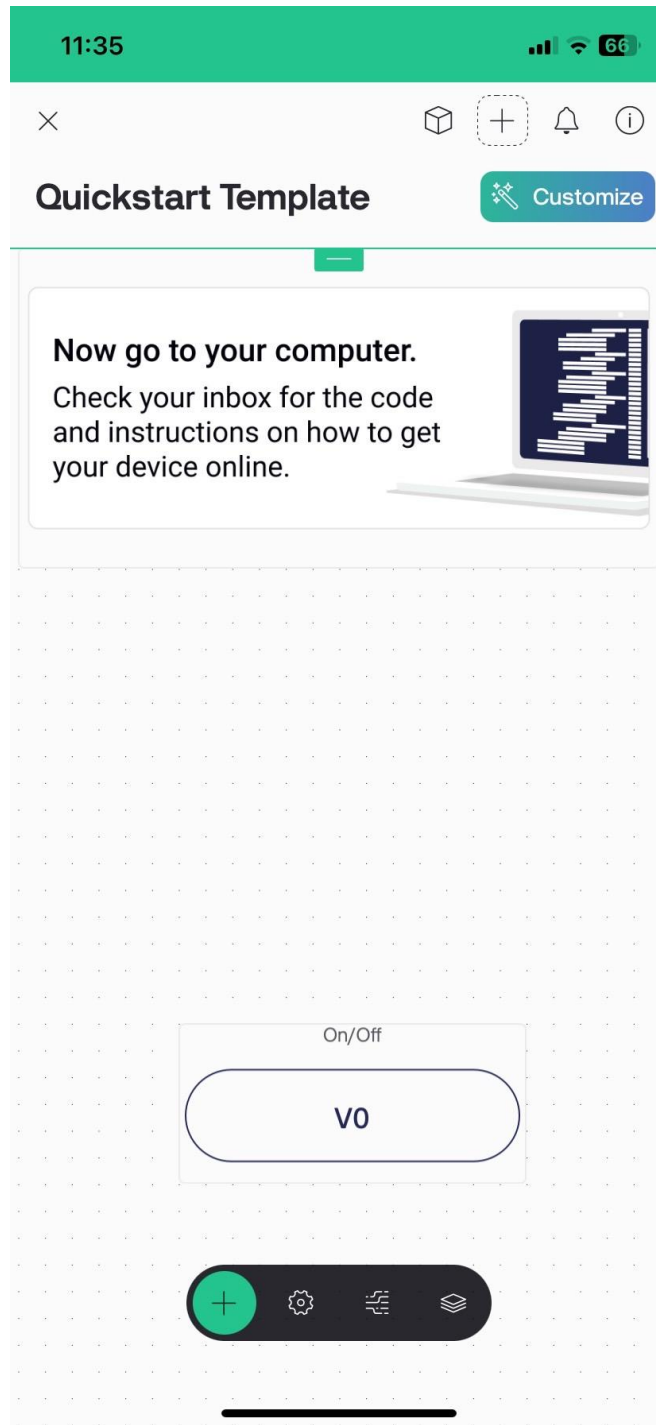
void loop()
{
  Blynk.run();
  timer.run();
}
```

- YourNetworkName ให้กรอกชื่อเครือข่ายอินเทอร์เน็ตที่จะทำการเชื่อมต่อ แนะนำให้ใช้ Hotspot ของโทรศัพท์มือถือ
- YourPassword ให้กรอกรหัสผ่านของเครือข่ายอินเทอร์เน็ตที่จะทำการเชื่อมต่อ
- YourAuthToken ให้นำ Token จากขั้นตอนถัดไปมากรอก
- YourTemplateID ให้นำ Template ID จากขั้นตอนถัดไปมากรอก
- ไปที่แอปพลิเคชัน Blynk และที่ Quick Start จะปรากฏหน้าต่างค่า ให้เลือก Hardware เป็น ESP8266 และ Connection Type เป็น Wi-Fi จากนั้นกด Next



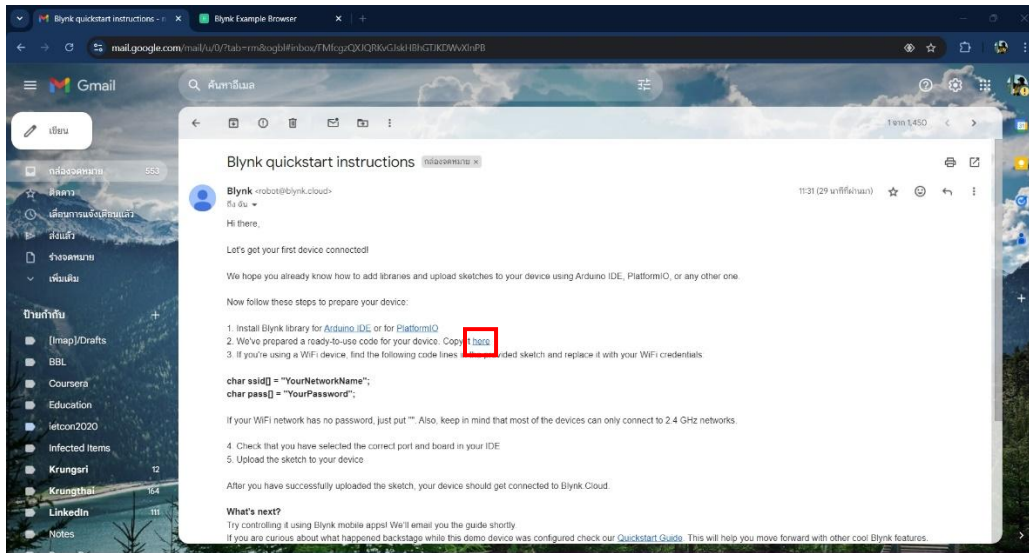
รูปที่ 10.7 การตั้งค่า Hardware บนแอปพลิเคชัน Blynk

- แอปพลิเคชันจะแสดงหน้าต่างบนมือถือ ซึ่งจะเป็นส่วนที่ใช้ในการควบคุมฮาร์ดแวร์ ซึ่งสามารถตั้งค่าและปรับแต่งหน้า UI ได้ โดยใน Quick Start จะมีสวิตช์กดติดกดดับ (V0) กำหนดมาให้ 1 อัน



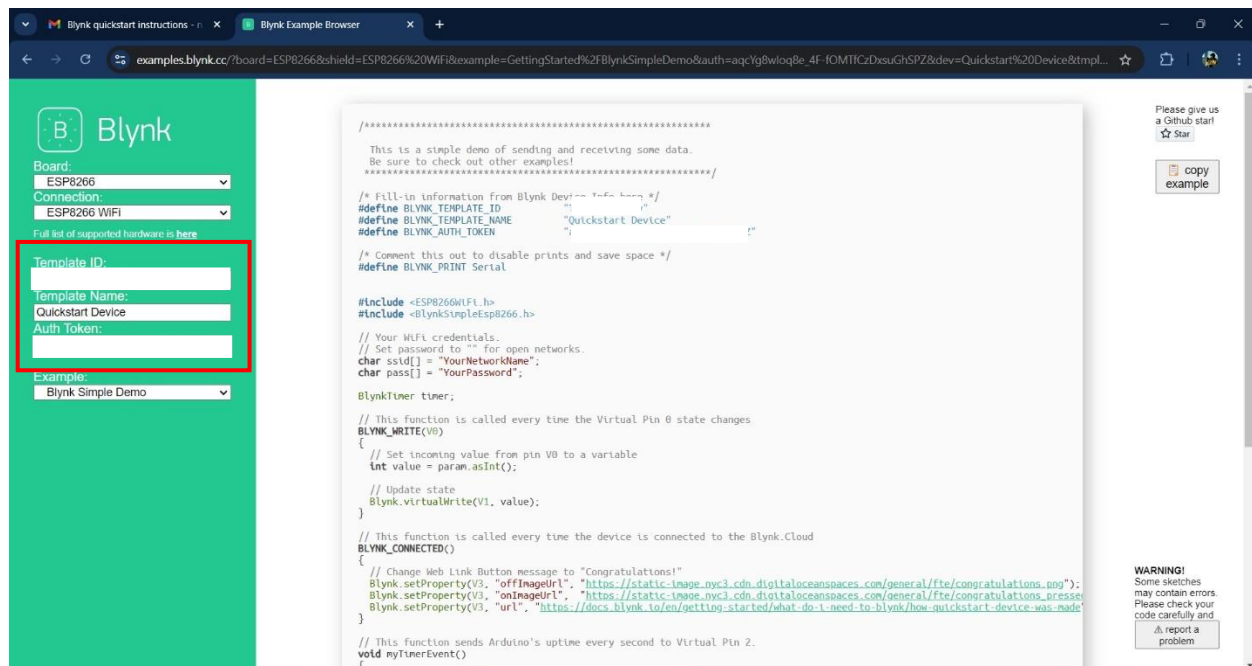
รูปที่ 10.8 หน้า UI บนแอปพลิเคชัน Blynk

- เมื่อสร้างแอปพลิเคชันจาก Quick Start เสร็จ จะมีอีเมลส่งไปยังอีเมลที่ใช้ในการสมัครแอปพลิเคชัน ให้ไปที่อีเมล แล้วคลิกลิงก์ที่คำว่า [here](#)



รูปที่ 10.9 อีเมลเพื่อขอ Auth Token และ Template ID

- ลิงก์จะพาไปสู่หน้า Template ของ Quick Start ให้คัดลอก Auth Token และ Template ID ไปใส่ในโค้ดที่อยู่ใน Arduino IDE



รูปที่ 10.10 หน้าตัวอย่างโค้ดสำหรับโปรแกรมลงบนบอร์ด ESP8266

- ทำการ Verify โค้ด แล้วอัปโหลดโค้ดลงบอร์ด
- เปิดหน้า Serial Monitor

- ทดลองกดเปิดปิดสวิตช์บนโทรศัพท์แล้วสังเกตผลลัพธ์ที่เกิดขึ้น

ผลการทำงานหลังจากอัปโหลดโค้ดลงบอร์ด

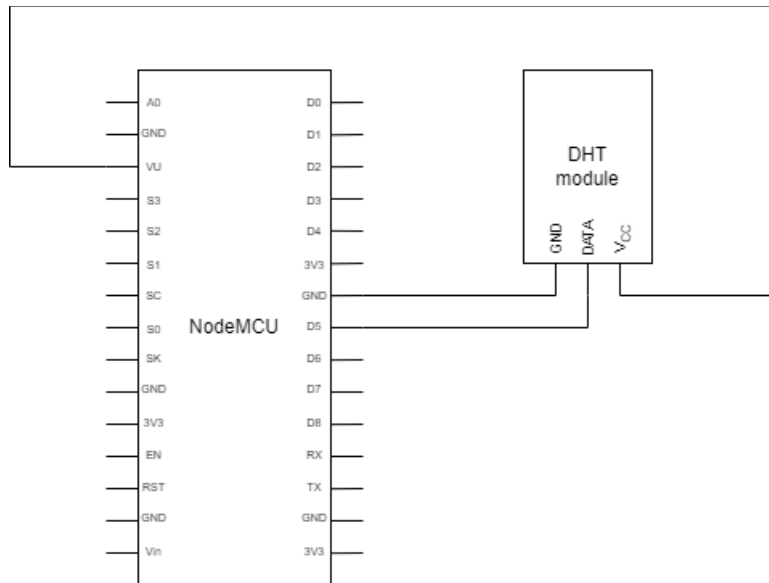
Lab 10.2: การสร้าง Dashboard ในการรายงานอุณหภูมิและความชื้นบนแอปพลิเคชัน Blynk

อุปกรณ์ที่ต้องใช้

- บอร์ด NodeMCU 1 ตัว
- โมดูล DHT 1 ตัว
- สาย Jumper
- Breadboard 1 อัน
- แอปพลิเคชัน Blynk

ขั้นตอนในการทำ

- ต่อดังต่อไปนี้แสดงในรูป



รูปที่ 10.11 วงจรสำหรับแลบที่ 10.2

- พิมพ์โค้ดต่อไปนี้

```
#define BLYNK_TEMPLATE_ID          "YourTemplateID"
#define BLYNK_TEMPLATE_NAME       "Quickstart Device"
#define BLYNK_AUTH_TOKEN          "YourAuthToken"

#define BLYNK_PRINT Serial

#include <ESP8266WiFi.h>
#include <BlynkSimpleEsp8266.h>
#include <DHT.h>

char ssid[] = "YourNetworkName";
char pass[] = "YourPassword";

#define DHTPIN D5
#define DHTTYPE DHT11

DHT dht(DHTPIN, DHTTYPE);

void setup() {
  Serial.begin(115200);
  Blynk.begin(BLYNK_AUTH_TOKEN, ssid, pass);
  dht.begin();
}

void loop() {
  Blynk.run();
  float t = dht.readTemperature();
  float h = dht.readHumidity();

  if (isnan(t) || isnan(h)) {
    Serial.println("DHT sensor failure.");
    return;
  }

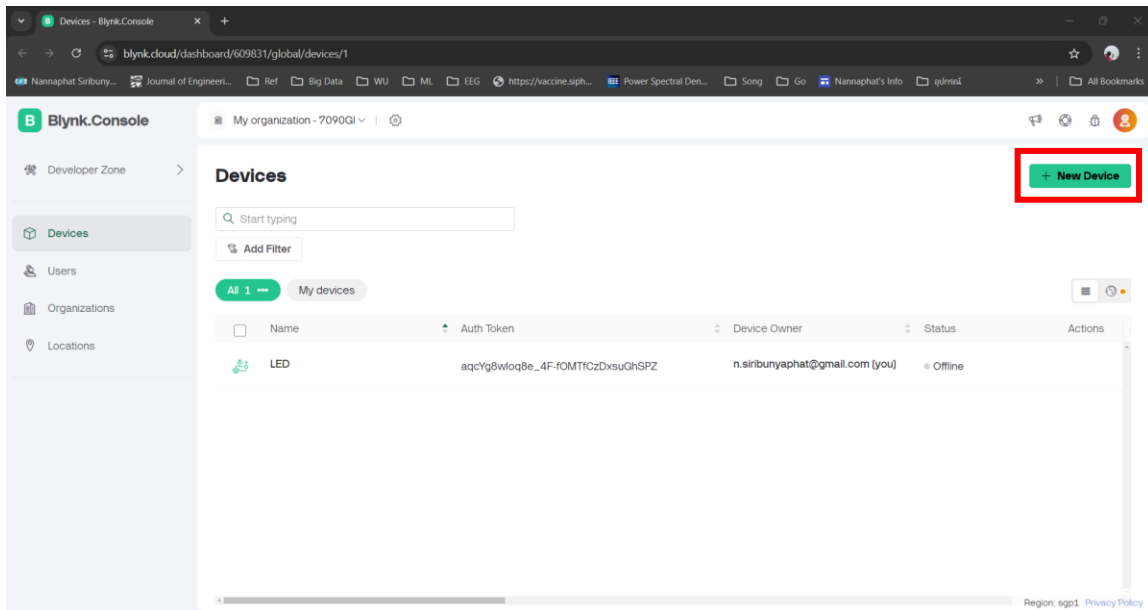
  Blynk.virtualWrite(V0, t);
  Blynk.virtualWrite(V1, h);

  Serial.print("Temperature: ");
  Serial.print(t);
  Serial.print("°C | ");
  Serial.print("Humidity: ");
  Serial.print(h);
  Serial.println("%");

  delay(2000);
}
```

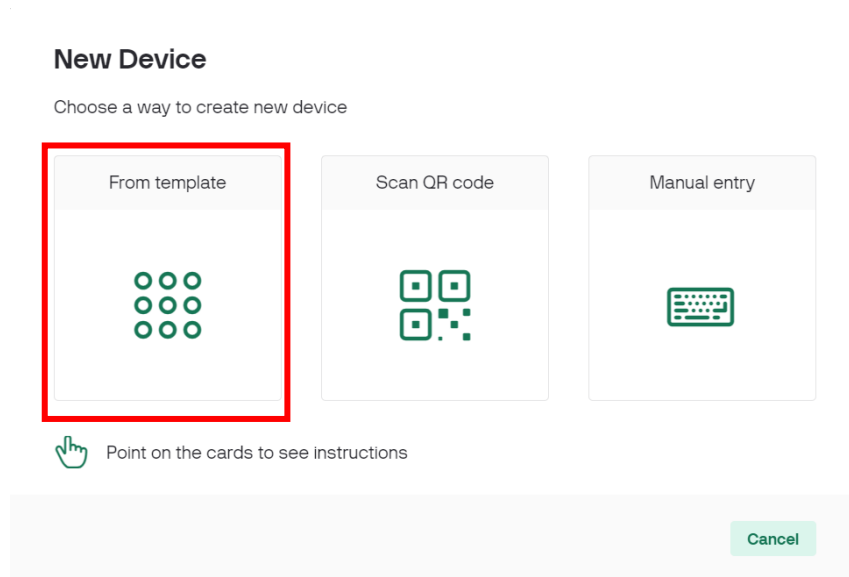
- YourNetworkName ให้กรอกชื่อเครือข่ายอินเทอร์เน็ตที่จะทำการเชื่อมต่อ แนะนำให้ใช้ Hotspot ของโทรศัพท์มือถือ
- YourPassword ให้กรอกรหัสผ่านของเครือข่ายอินเทอร์เน็ตที่จะทำการเชื่อมต่อ
- YourAuthToken ให้นำ Token จากขั้นตอนถัดไปมากรอก
- YourTemplateID ให้นำ Template ID จากขั้นตอนถัดไปมากรอก

- เข้าไปที่เว็บไซต์ <https://blynk.io/> แล้วทำการล็อกอินด้วยอีเมลที่สมัครเอาไว้ในตอนแรก
- เมื่อล็อกอินแล้วจะเข้าสู่หน้า Blynk.Console ที่เห็น Devices ให้กดปุ่ม New Device เพื่อสร้างอุปกรณ์ใหม่



รูปที่ 10.11 หน้า Website Blynk.Console

- เมื่อหน้า New Device ปรากฏขึ้น ให้เลือก From Template



รูปที่ 10.12 เลือก Template สำหรับสร้างอุปกรณ์ใหม่

- เลือก Template เป็น Quickstart Template แล้วตั้งชื่อ Device Name เป็น DHT จากนั้นกดปุ่ม Create

New Device

Create new device by filling in the form below

TEMPLATE

Quickstart Template

DEVICE NAME

DHT

3 / 50

Cancel

Create

รูปที่ 10.13 กรอกค่าสำหรับอุปกรณ์ใหม่

- เมื่อสร้างอุปกรณ์ใหม่สำเร็จ มุมขวาวจะมี Template ID และ Auth Token ปรากฏขึ้นมา ให้คัดลอกแล้วนำไปใส่ในโค้ดในขั้นตอนก่อนหน้านี้

New Device Created!

×

```
#define BLYNK_TEMPLATE_ID " "
#define BLYNK_TEMPLATE_NAME "Quickstart Template"
#define BLYNK_AUTH_TOKEN
" "
```

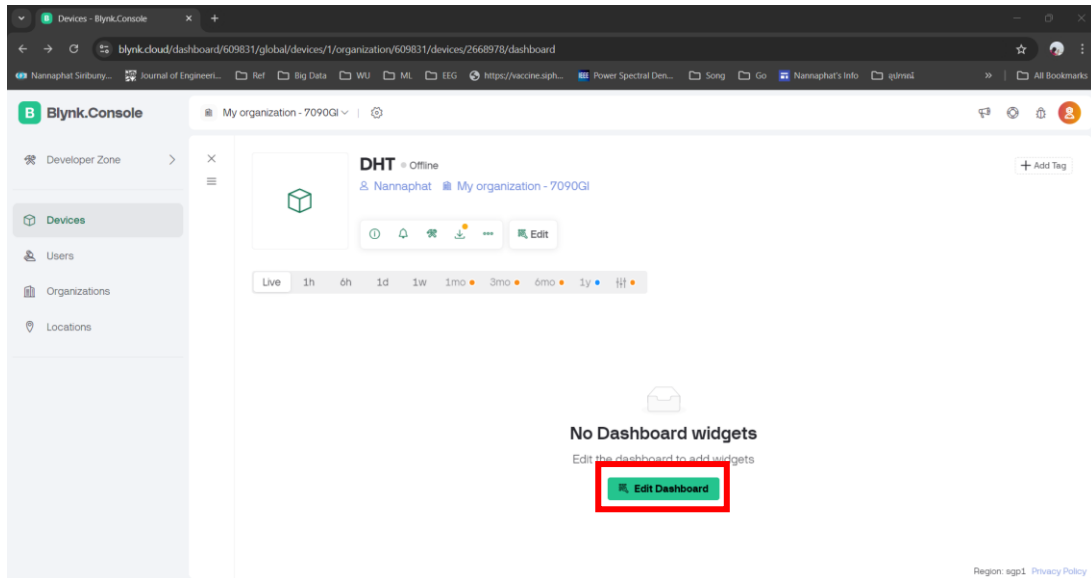
Template ID, Template Name, and AuthToken should be declared at the very top of the firmware code.

[Documentation](#)

[Copy to clipboard](#)

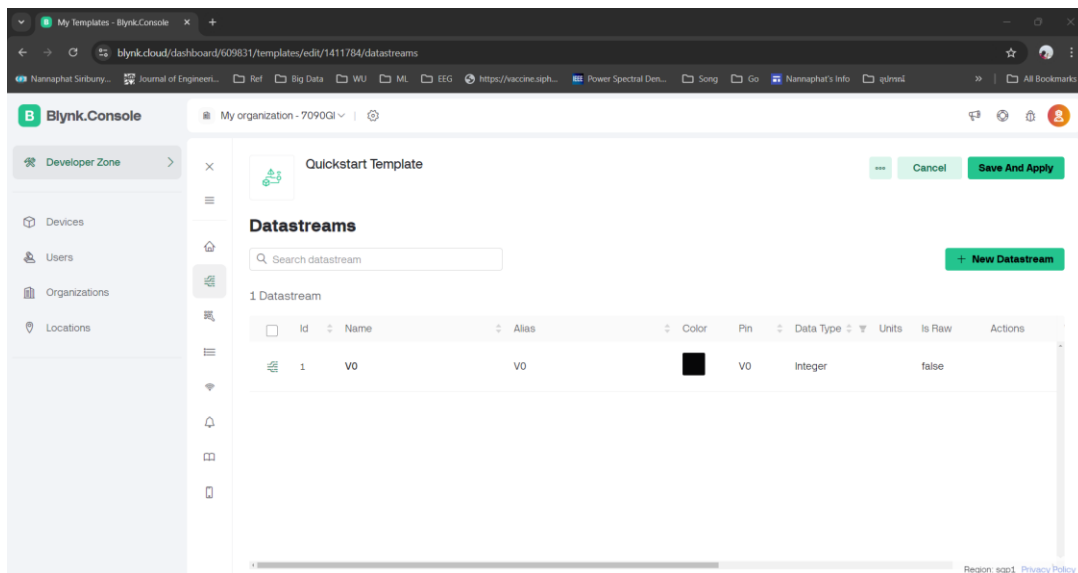
รูปที่ 10.14 Template ID และ Auth Token สำหรับอุปกรณ์ใหม่

- กดปุ่ม Edit Dashboard เพื่อทำการแก้ไข



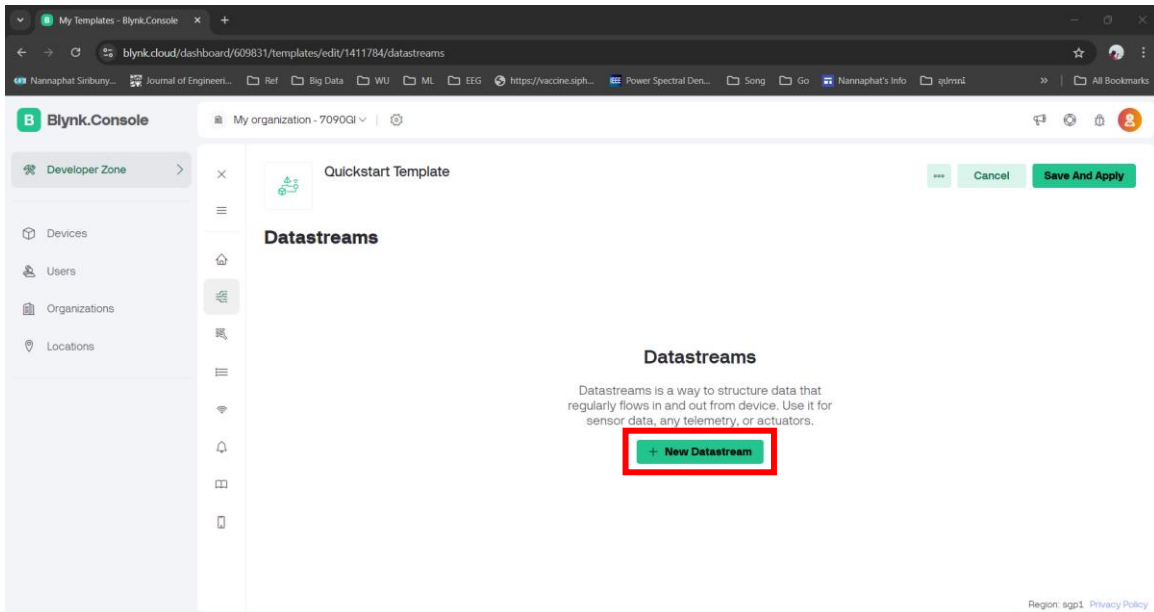
รูปที่ 10.15 หน้า Dashboard ของอุปกรณ์ใหม่

- ที่หน้า Web Dashboard ให้คลิกที่แท็บ Datastreams



รูปที่ 10.16 หน้า Datastreams

- เนื่องจาก Blynk ในเวอร์ชันฟรีจะสามารถสร้าง Datastream ได้สูงสุด 5 Datastream เท่านั้น ให้เลื่อนเมาส์ไปที่ Action แล้วทำการลบ Datastream ที่มีออก จากนั้นกดปุ่ม New Datastreams แล้วเลือก Virtual Pin



รูปที่ 10.17 หน้า Datastreams ที่ลบ Datastream ออกแล้ว

- ทำการตั้งค่า Datastream ดังรูปที่ 10.18 จากนั้นกด Create

Virtual Pin Datastream

General Expose to Automations

NAME		ALIAS	
Temperature		Temperature	
PIN		DATA TYPE	
V0		Double	
UNITS			
Celsius, °C			
MIN	MAX	DECIMALS	DEFAULT VALUE
0	100	#.00	0
☐ Enable history data			
		Cancel	Create

รูปที่ 10.18 การตั้งค่า Datastream สำหรับค่าอุณหภูมิ

- คลิกที่ New Datastream อีกครั้ง แล้วเลือก Virtual Pin จากนั้นทำการตั้งค่า Datastream ดังรูปที่ 10.19 จากนั้นกด Create

Virtual Pin Datastream

General Expose to Automations

NAME: Humidity ALIAS: Humidity

PIN: V1 DATA TYPE: Double

UNITS: Percentage, %

MIN: 0 MAX: 100 DECIMALS: #.00 DEFAULT VALUE: 0

Cancel Create

รูปที่ 10.19 การตั้งค่า Datastream สำหรับค่าความชื้น

- จะได้ Datastream ใหม่ 2 Datastreams

Blynk.Console My organization - 7090GI

Quickstart Template

Datastreams

Search datastream

+ New Datastream

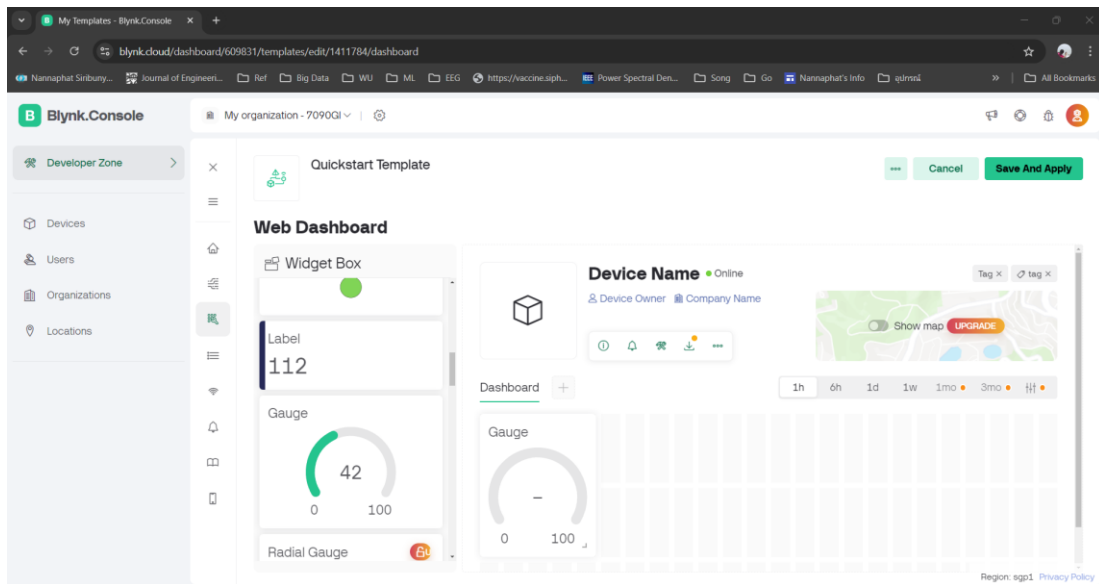
2 Datastreams

	Id	Name	Alias	Color	Pin	Data Type	Units	Is Raw	Actions
1		Temperature	Temperature		V0	Double	°C	false	
2		Humidity	Humidity		V1	Double	%	false	

Region: sg01 Privacy Policy

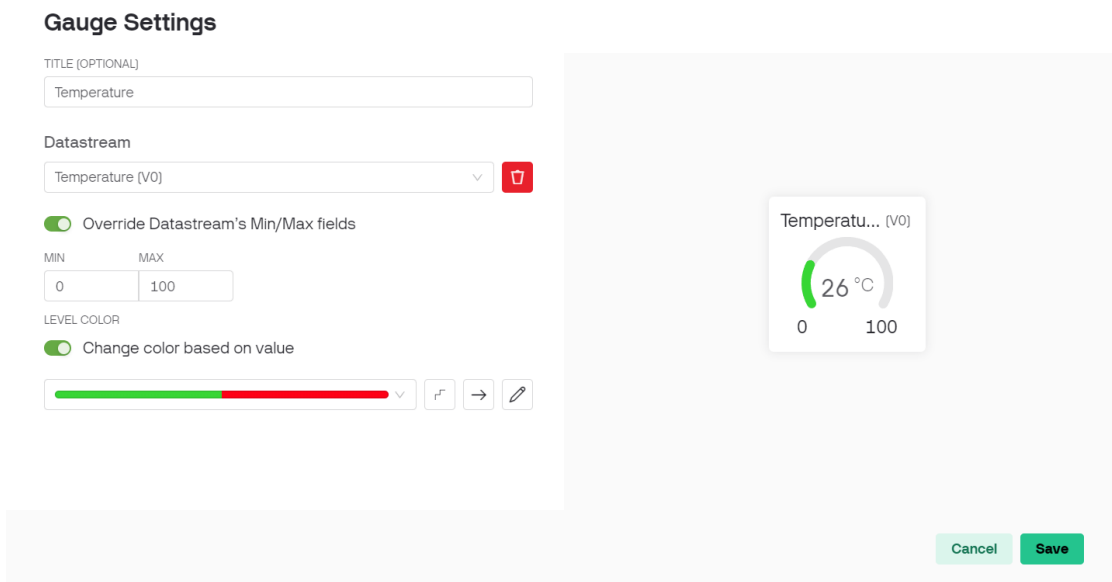
รูปที่ 10.20 Datastreams สำหรับแลบที่ 9.2

- กลับมาที่หน้า Web Dashboard ที่ Widget Box ลาก Gauge มาใส่ที่หน้า Dashboard



รูปที่ 10.21 สร้าง Dashboard โดยใช้ Gauge

- วางเคอร์เซอร์บน Widget Gauge แล้วคลิกที่ปุ่มฟันเฟืองเพื่อแก้ไข Widget จะปรากฏหน้าต่าง Gauge Settings ตั้งค่าดังที่แสดงในรูปที่ 9.20 แล้วกด Save



รูปที่ 10.22 การตั้งค่า Gauge สำหรับค่าอุณหภูมิ

- ทำซ้ำโดยการลาก Gauge มาที่ Dashboard แล้วทำการตั้งค่าสำหรับค่าความชื้น แล้วกด Save

Gauge Settings

TITLE (OPTIONAL)

Datastream
 

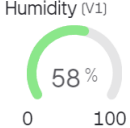
☒ Override Datastream's Min/Max fields

MIN	MAX
0	100

LEVEL COLOR
☒ Change color based on value

Humidity (V1)

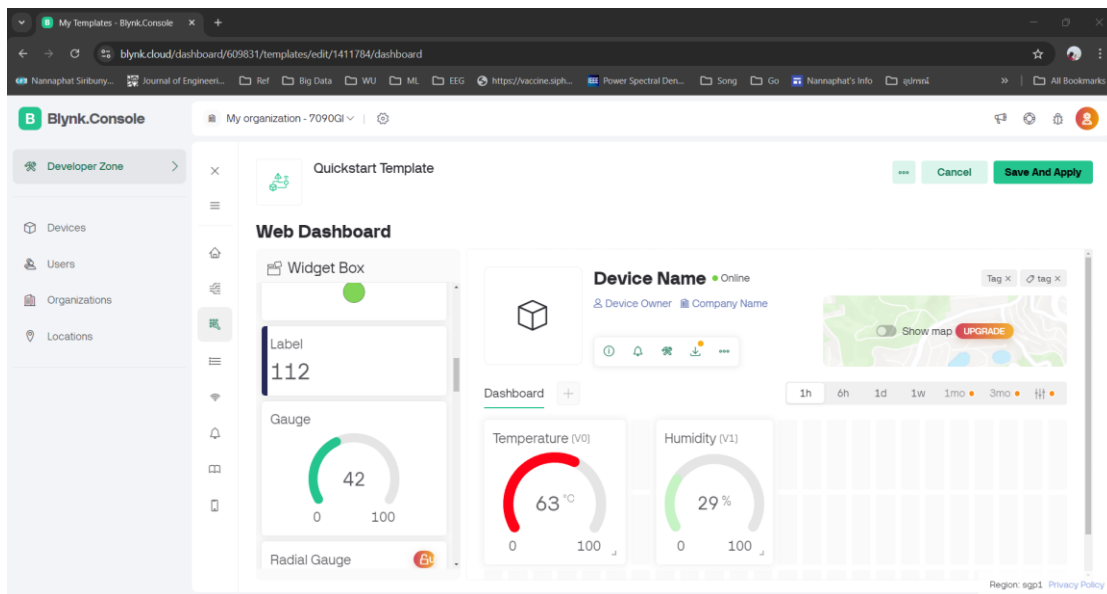


0 100

Cancel Save

รูปที่ 10.23 การตั้งค่า Gauge สำหรับค่าความชื้น

- คลิก Save And Apply เพื่อบันทึกแอปพลิเคชัน



รูปที่ 10.24 Dashboard สำหรับแอปพลิเคชันตรวจสอบอุณหภูมิและความชื้น

- ทำการ Verify โค้ด แล้วอัปโหลดโค้ดลงบอร์ด
- เปิดหน้า Serial Monitor

- ตรวจสอบผลลัพธ์บนหน้าแอปพลิเคชัน Blynk ในโทรศัพท์และ Serial Monitor

ผลการทำงานหลังจากอัปโหลดโค้ดลงบอร์ด

Firebase เป็นหนึ่งในบริการของ Google ที่ให้บริการฐานข้อมูลออนไลน์ในรูปแบบของ Real Time Database สำหรับ Application และ Web Application ซึ่งระบบ IoT สามารถนำ Firebase มาเป็นตัวกลางในการเชื่อมต่ออุปกรณ์เข้าด้วยกัน ซึ่ง Firebase จะเป็นฐานข้อมูลแบบ NoSQL โดยมีการเก็บข้อมูลแบบ JSON ทำให้มีความยืดหยุ่นในการใช้งาน

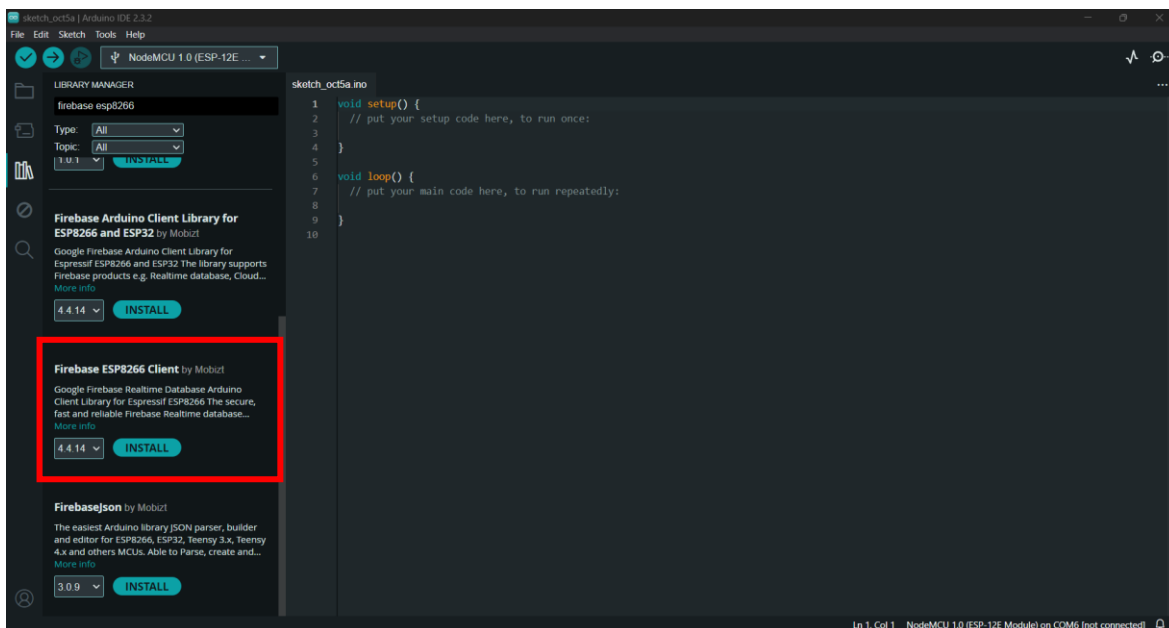
Lab 10.3: การเชื่อมต่อ NodeMCU กับ Firebase

อุปกรณ์ที่ต้องใช้

- บอร์ด NodeMCU 1 ตัว
- Firebase

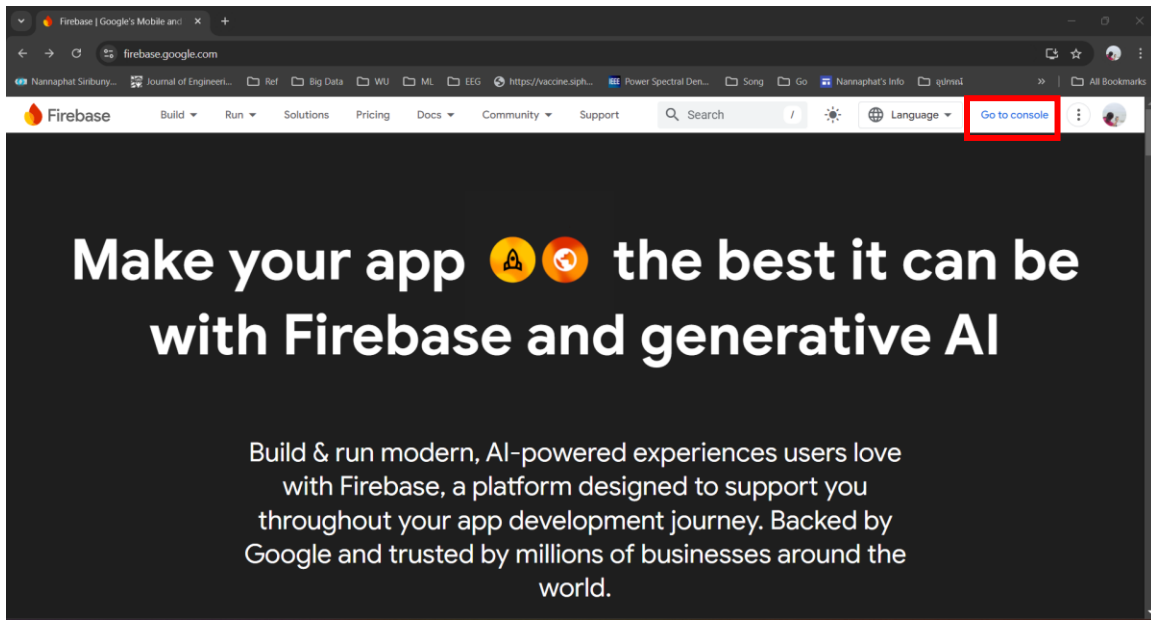
ขั้นตอนในการทำ

- ที่ Arduino IDE คลิกเมนู Tools -> Manage Library...
- ค้นหา firebase esp8266 แล้วทำการติดตั้งไลบรารี Firebase ESP8266 Client



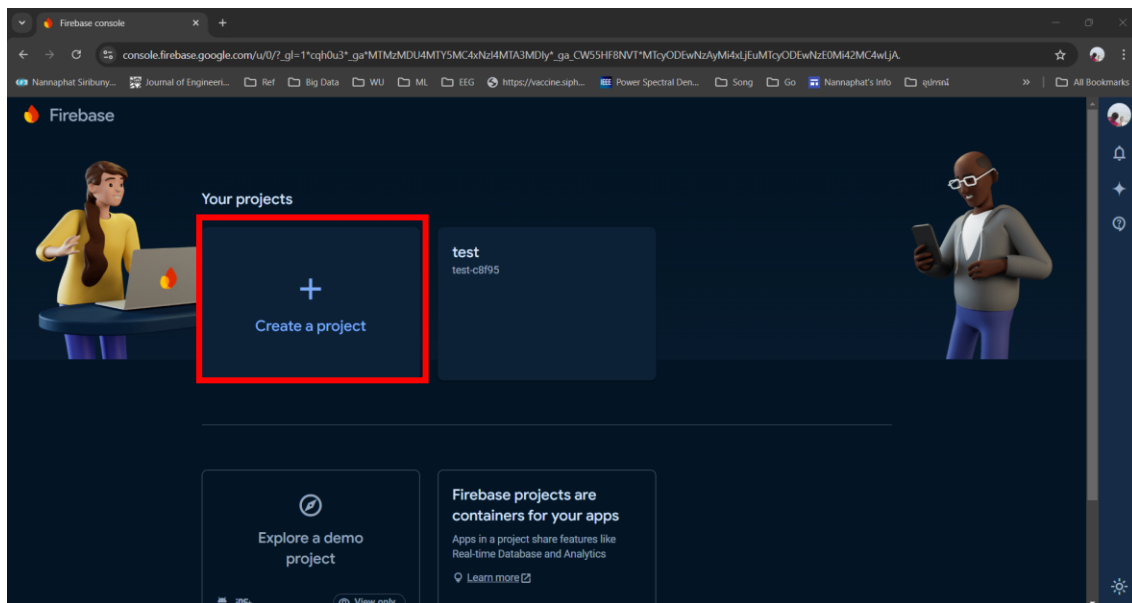
รูปที่ 10.25 ไลบรารี Firebase ESP8266 Client

- ไปที่ <https://firebase.google.com/> ทำการ sign in ด้วยแอคเคานท์ Google แล้วคลิกที่ Go to console



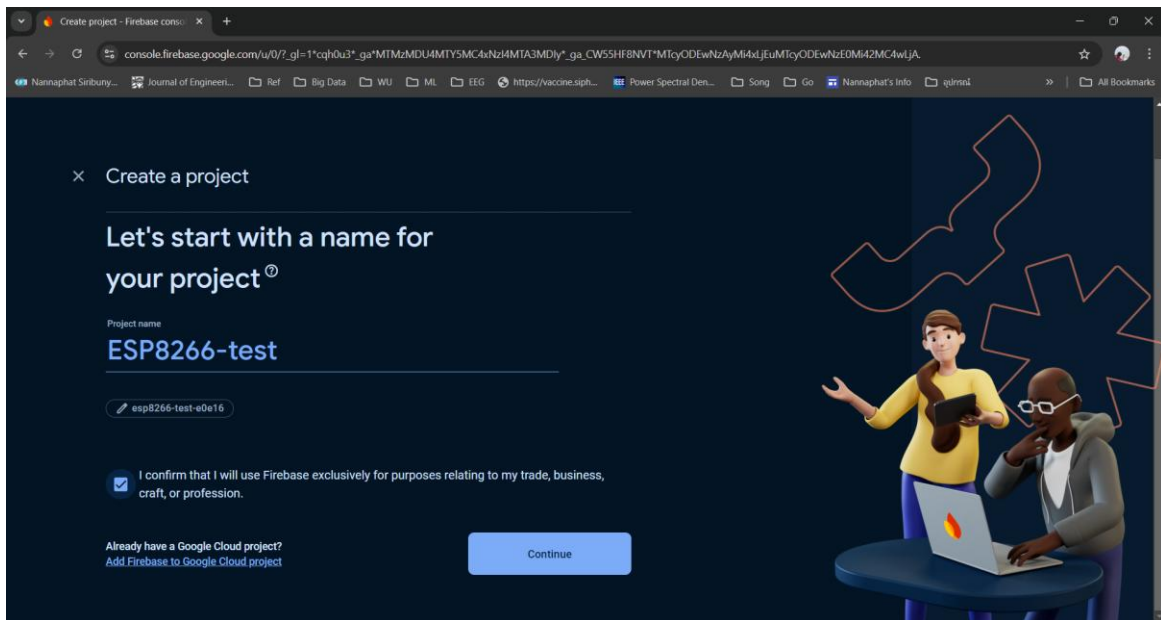
รูปที่ 10.26 หน้าเว็บ Firebase

- Click ที่ Create a project เพื่อสร้างโปรเจกต์ใหม่



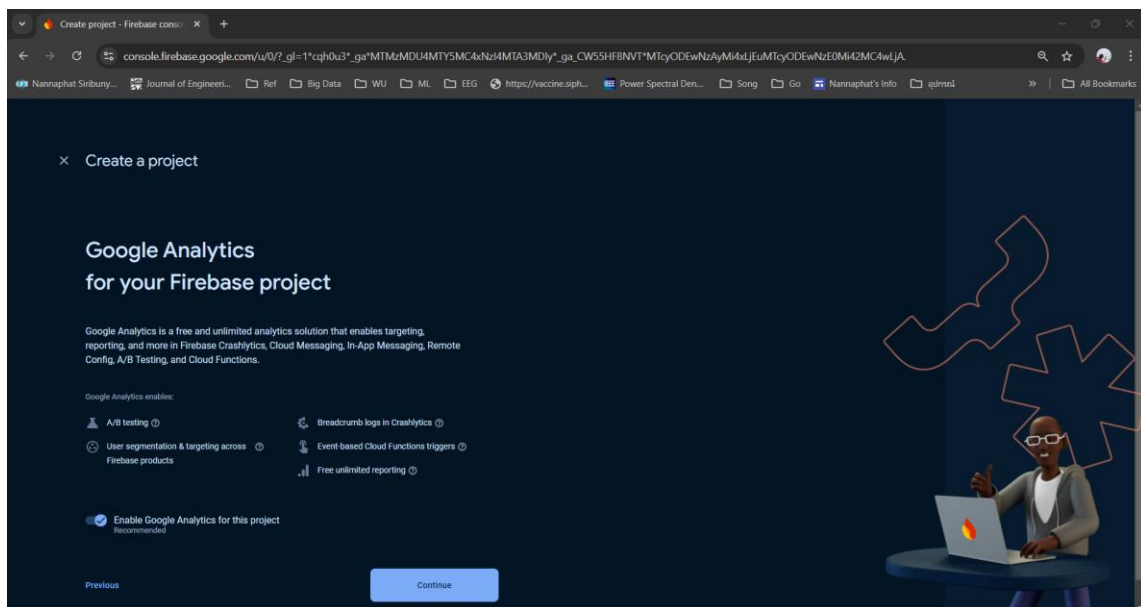
รูปที่ 10.27 หน้า console ของ Firebase

- ตั้งชื่อโปรเจกต์ เลือกยอมรับการใช้งานของ Firebase แล้วกด Continue



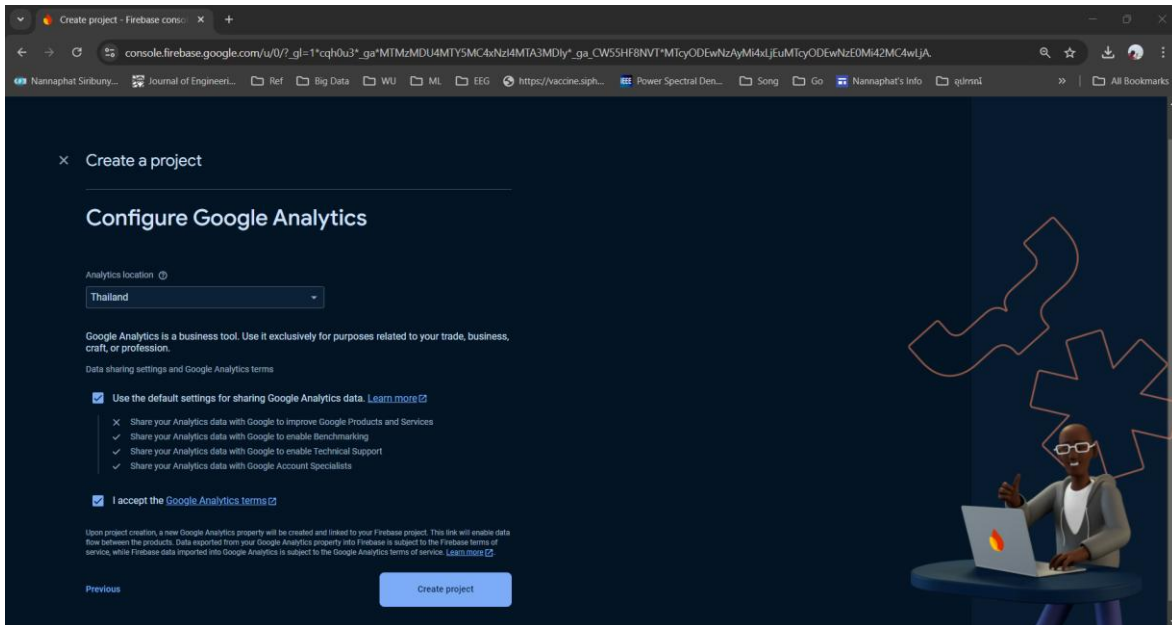
รูปที่ 10.28 การสร้างโปรเจกต์ใน Firebase

- ในหน้า Google Analytics ให้ Enable Google Analytics for this project แล้วกด Continue



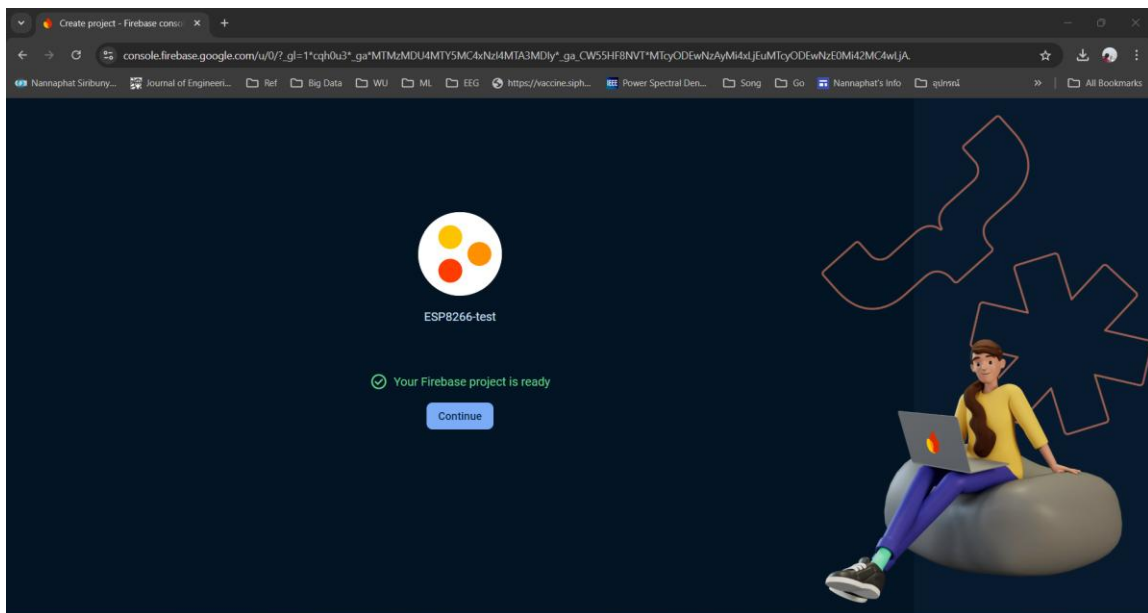
รูปที่ 10.29 ตั้งค่า Google Analytics ใน Firebase

- ตั้งค่า Analytics location เป็น Thailand เลือกยอมรับเงื่อนไขของ Google Analytics แล้วกด Create project



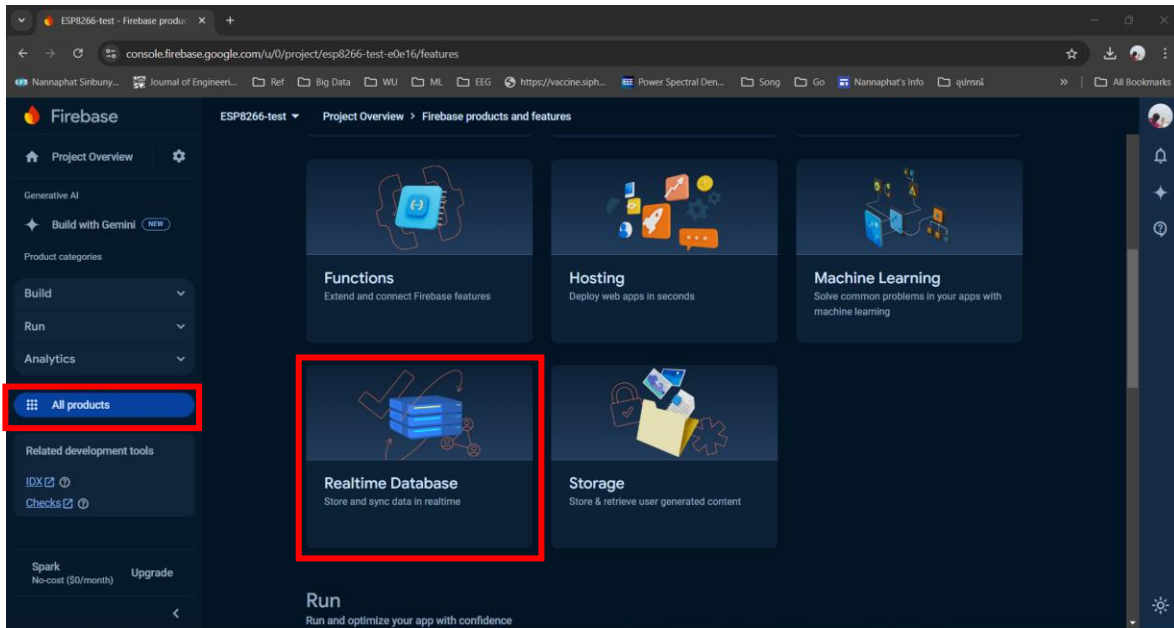
รูปที่ 10.30 ตั้งค่า Google Analytics Location ใน Firebase

- เมื่อระบบสร้างโปรเจกต์เสร็จให้กด Continue



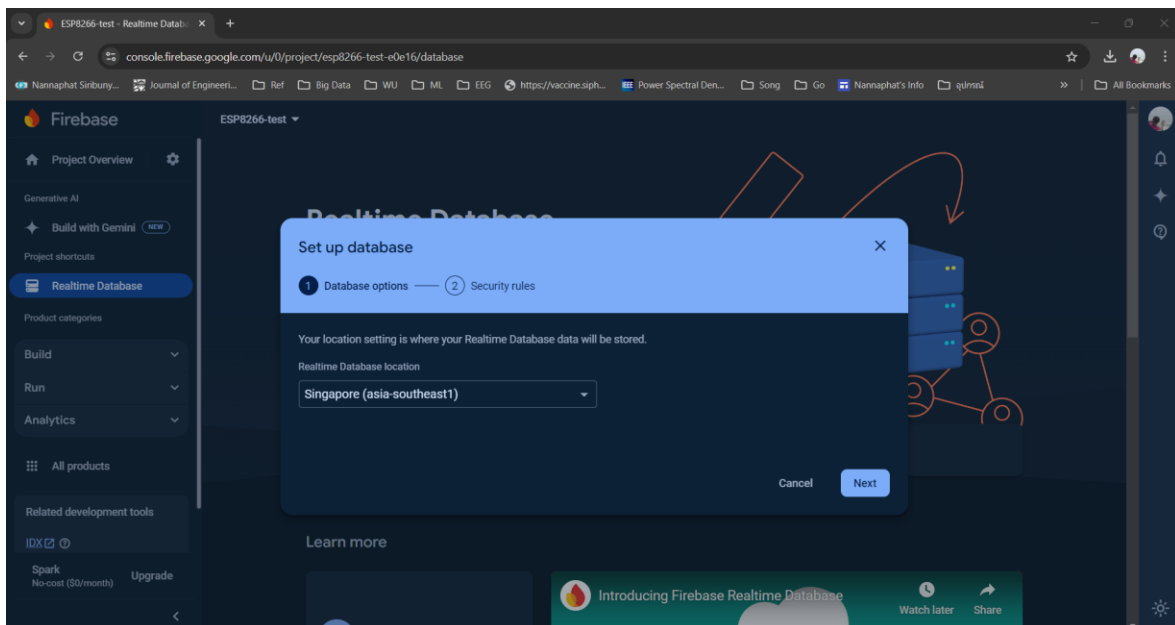
รูปที่ 10.31 ระบบสร้างโปรเจกต์เสร็จสิ้น

- ในหน้าโปรเจกต์ เลือกแท็บ All products ที่แถบด้านซ้ายมือ แล้วเลือก Realtime Database แล้วกด Create Database



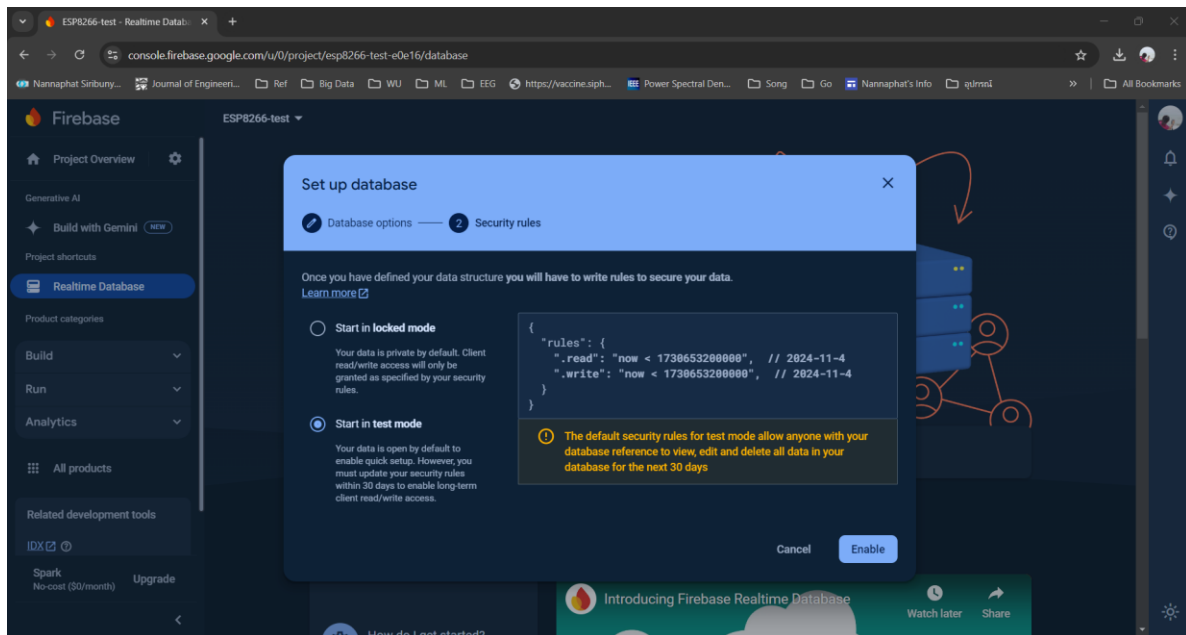
รูปที่ 10.32 สร้าง Realtime Database

- ตั้งค่า Location เป็น Singapore (asia-southeast1) แล้วกด next



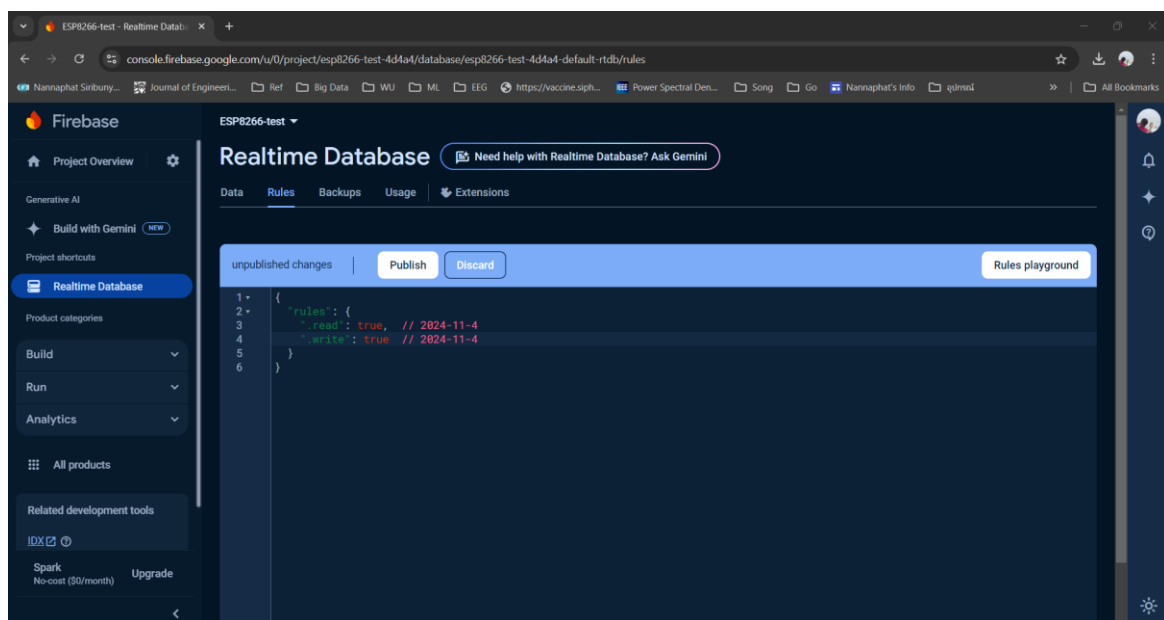
รูปที่ 10.33 เลือกภูมิภาคของ Database

- เลือก Start in test mode แล้วกด Enable



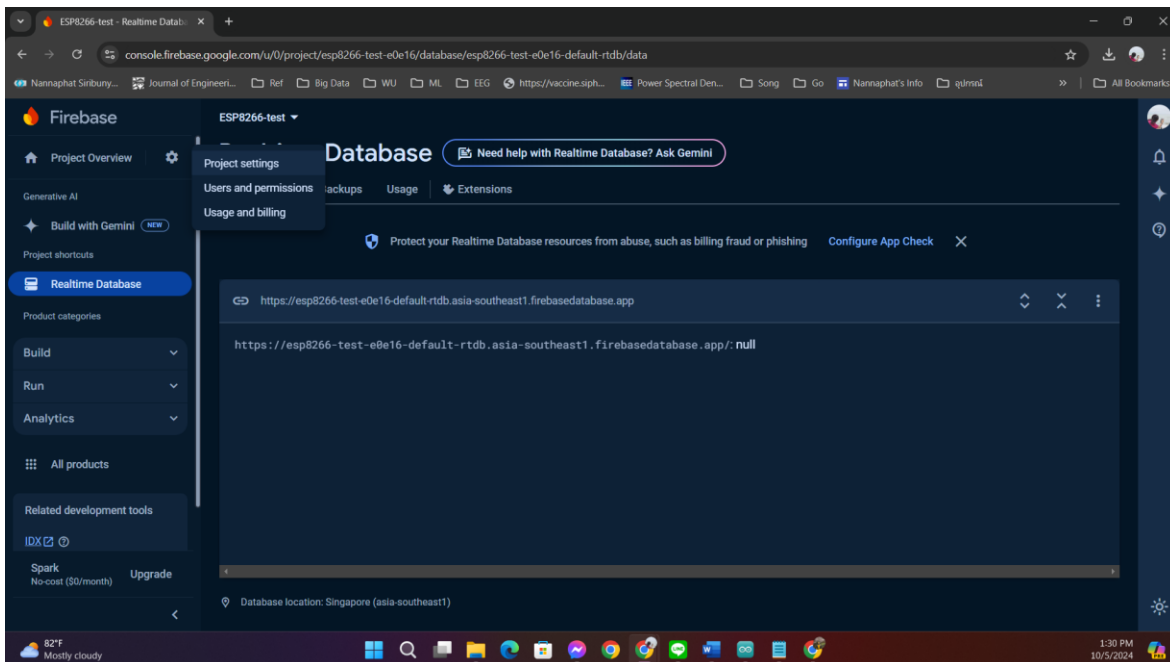
รูปที่ 10.34 ตั้งค่า Security rules ของ Database

- เมื่อสร้าง Database เสร็จ คลิกที่แท็บ Rules แล้วให้เปลี่ยน rules เป็น True ทั้ง read และ write ดังรูปที่ 10.35 แล้วกด Publish



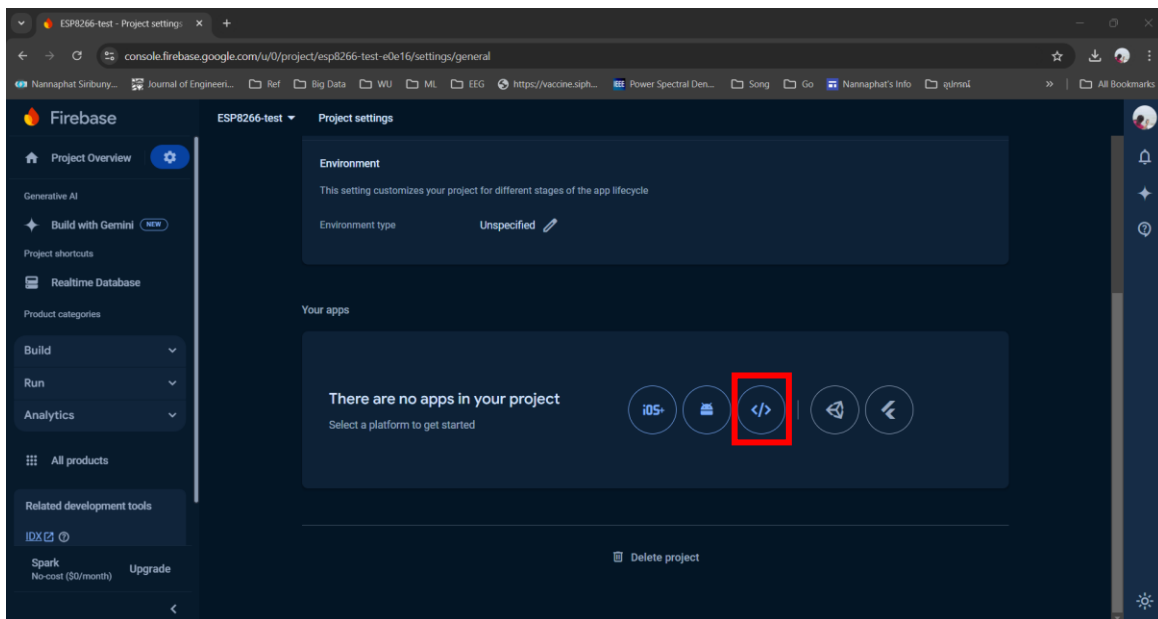
รูปที่ 10.35 ตั้งค่า rules สำหรับ Test mode

- คลิกที่รูปฟันเฟืองข้าง Project Overview เลือก Project settings



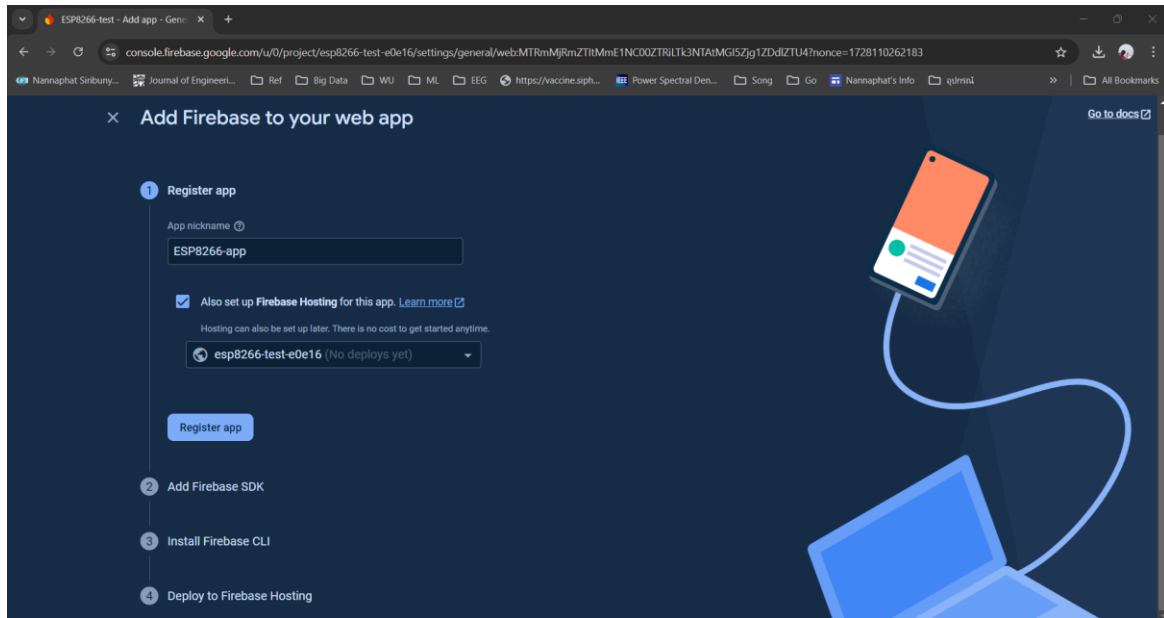
รูปที่ 10.36 Project Settings

- ในแท็บ General เลื่อนลงมาที่หัวข้อ Your apps แล้วเลือก Web API Key



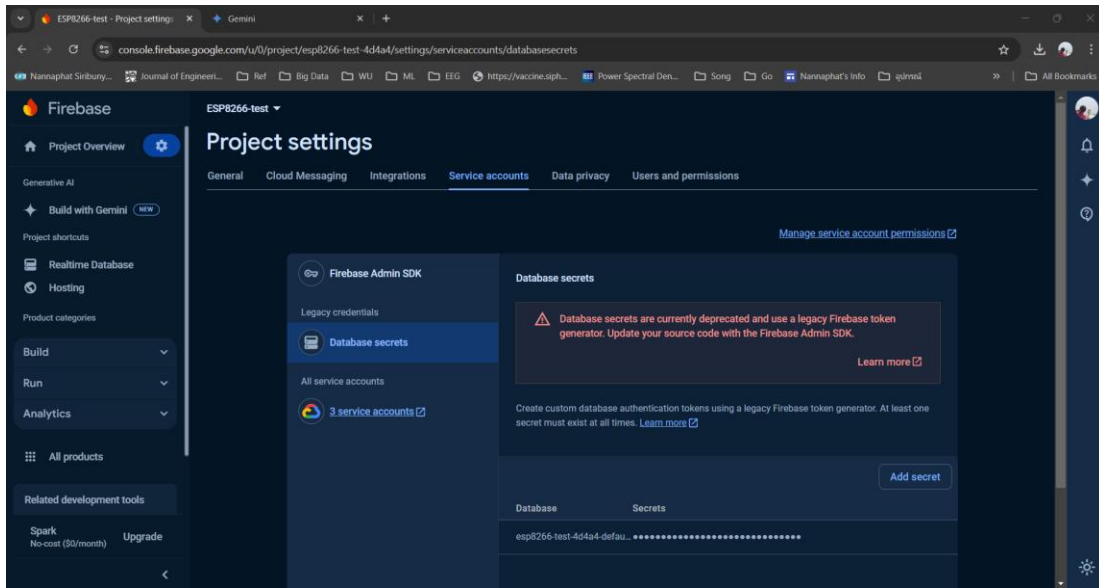
รูปที่ 10.37 Web API Key

- ตั้งชื่อ app เลือก Also set up Firebase Hosting for this app แล้วกด Register app



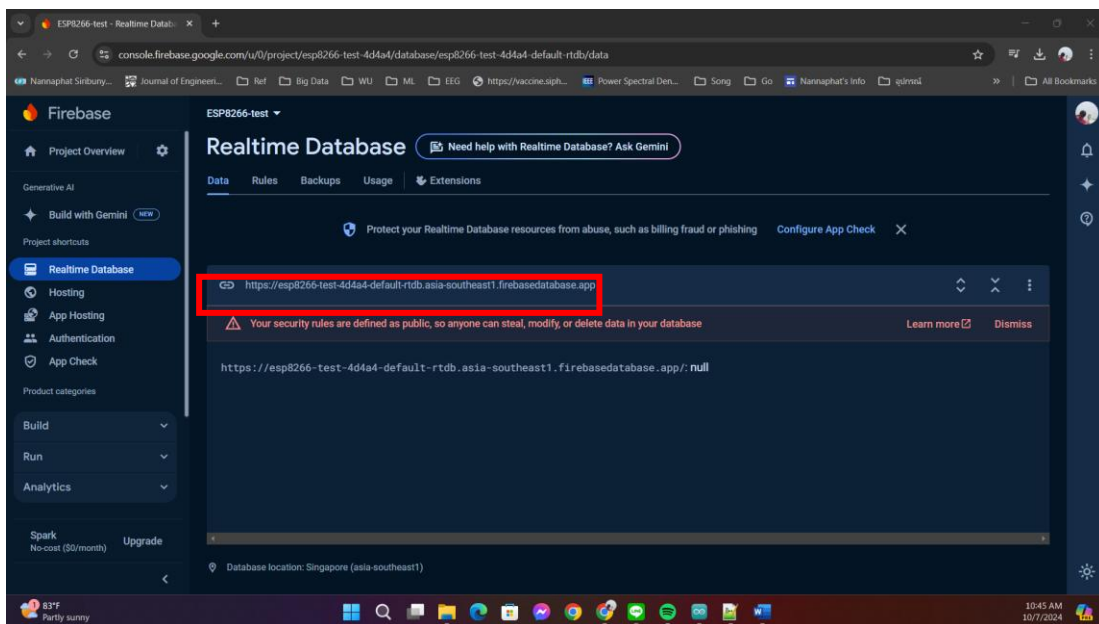
รูปที่ 10.38 Register App

- ที่หัวข้อ Add Firebase SDK ให้กด Next
- ที่หัวข้อ Install Firebase CLI ให้กด Next
- ที่ Deploy to Firebase Hosting ให้กด Continue to console
- ที่หน้า Project Settings เลือกแท็บ Service accounts เลือก Database secrets จะเห็น Secrets Key ให้กด show แล้วคัดลอกไว้เพื่อนำไปใส่ในโค้ดของ ESP8266



รูปที่ 10.39 Secret Key ของ Firebase

- กลับมาที่หน้า Realtime Database ที่แท็บ Data จะมี Database URL ให้คัดลอกไว้เพื่อนำไปใส่ในโค้ดของ ESP8266



รูปที่ 10.40 Database URL ของ Firebase

- พิมพ์โค้ดต่อไป่นี้แล้วทำการ Verify โค้ด แล้วอัปโหลดโค้ดลงบอร์ด

```
#include <Arduino.h>
#include <ESP8266WiFi.h>
#include <FirebaseESP8266.h>

#include <addons/RTDBHelper.h>

#define WIFI_SSID "YourNetworkName"
#define WIFI_PASSWORD "YourPassword"

#define DATABASE_URL "DatabaseURL"
#define DATABASE_SECRET "DatabaseSecret"

FirebaseData fbdo;
FirebaseAuth auth;
FirebaseConfig config;

unsigned long dataMillis = 0;
int count = 0;

void setup()
{
    Serial.begin(115200);

    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

    Serial.print("Connecting to Wi-Fi");
    unsigned long ms = millis();
    while (WiFi.status() != WL_CONNECTED)
    {
        Serial.print(".");
        delay(300);
    }
    Serial.println();
    Serial.print("Connected with IP: ");
    Serial.println(WiFi.localIP());
    Serial.println();

    Serial.printf("Firebase Client v%s\n\n", FIREBASE_CLIENT_VERSION);

    config.database_url = DATABASE_URL;
    config.signer.tokens.legacy_token = DATABASE_SECRET;

    Firebase.reconnectNetwork(true);

    fbdo.setBSSLBufferSize(4096 /* Rx buffer size in bytes from 512 - 16384 */, 1024 /* Tx buffer size in bytes from 512 - 16384 */);

    Firebase.begin(&config, &auth);
}

void loop()
{
    if (millis() - dataMillis > 5000)
    {
        dataMillis = millis();
        Serial.printf("Set int... %s\n", Firebase.setInt(fbdo, "/test/int", count++) ? "ok" : fbdo.errorReason().c_str());
    }
}
```

- YourNetworkName ให้กรอกชื่อเครือข่ายอินเทอร์เน็ตที่จะทำการเชื่อมต่อ แนะนำให้ใช้ Hotspot ของโทรศัพท์มือถือ
- YourPassword ให้กรอกรหัสผ่านของเครือข่ายอินเทอร์เน็ตที่จะทำการเชื่อมต่อ
- DatabaseURL ให้นำ Database URL ที่คัดลอกไว้มาใส่ในรูปแบบ
<databaseName>.<region>.firebase.database.app
- DatabaseSecret ให้นำ Secret Key ของ Firebase ที่คัดลอกไว้มาใส่

- เปิดหน้า Serial Monitor
- เปิดหน้า Realtime Database ของ Firebase

ผลการทำงานหลังจากอัปโหลดโค้ดลงบอร์ด

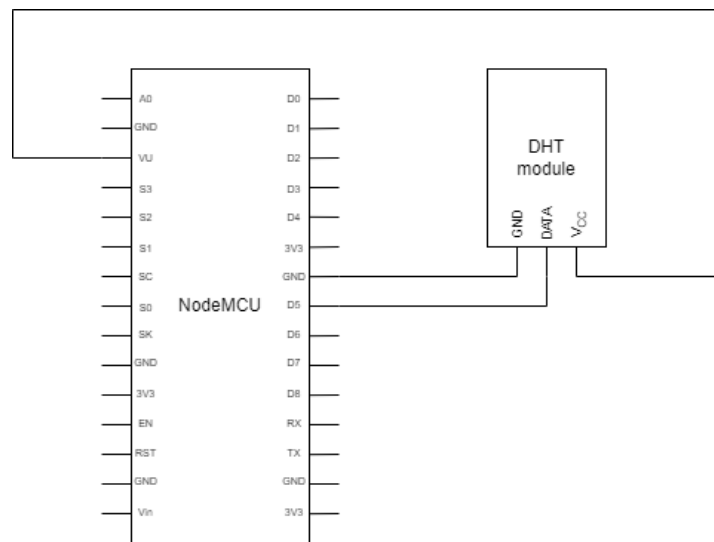
Lab 10.4: การบันทึกค่าอุณหภูมิและความชื้นลงใน Realtime Database ของ Firebase

อุปกรณ์ที่ต้องใช้

- บอร์ด NodeMCU 1 ตัว
- โมดูล DHT 1 ตัว
- สาย Jumper
- Breadboard 1 อัน
- Firebase

ขั้นตอนในการทำ

- ต่อดังต่อไปนี้แสดงในรูป



รูปที่ 10.41 วงจรสำหรับแลบที่ 10.4

- พิมพ์โค้ดต่อไปนี้แล้วทำการ Verify โค้ด แล้วอัปโหลดโค้ดลงบอร์ด

```
#include <Arduino.h>
#include <ESP8266WiFi.h>
#include <FirebaseESP8266.h>
#include <DHT.h>

#include <addons/RTDBHelper.h>

#define WIFI_SSID "YourNetworkName"
#define WIFI_PASSWORD "YourPassword"

#define DATABASE_URL "DatabaseURL"
#define DATABASE_SECRET "DatabaseSecret"

#define DHTPIN D5
#define DHTTYPE DHT11

FirebaseData fbdo;
FirebaseAuth auth;
FirebaseConfig config;
DHT dht(DHTPIN, DHTTYPE);

unsigned long dataMillis = 0;

void setup()
{
    Serial.begin(115200);

    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

    Serial.print("Connecting to Wi-Fi");
    while (WiFi.status() != WL_CONNECTED)
    {
        Serial.print(".");
        delay(300);
    }
    Serial.println();
    Serial.print("Connected with IP: ");
    Serial.println(WiFi.localIP());
    Serial.println();

    Serial.printf("Firebase Client v%s\n\n", FIREBASE_CLIENT_VERSION);
    config.database_url = DATABASE_URL;
    config.signer.tokens.legacy_token = DATABASE_SECRET;

    Firebase.reconnectNetwork(true);
    fbdo.setBSSLBufferSize(4096, 1024);
    Firebase.begin(&config, &auth);

    dht.begin();
}
```

```

void loop()
{
  if (millis() - dataMillis > 5000)
  {
    dataMillis = millis();

    float humidity = dht.readHumidity();
    float temperature = dht.readTemperature();

    if (isnan(humidity) || isnan(temperature))
    {
      Serial.println("Failed to read from DHT sensor!");
      return;
    }

    Serial.printf("Temperature: %.2f °C\n", temperature);
    Serial.printf("Humidity: %.2f %%\n", humidity);

    String timestamp = String(millis());

    if (Firebase.pushFloat(fbdo, "/sensor/temperature/" + timestamp, temperature))
    {
      Serial.println("Temperature uploaded successfully");
    }
    else
    {
      Serial.println(fbdo.errorReason());
    }

    if (Firebase.pushFloat(fbdo, "/sensor/humidity/" + timestamp, humidity))
    {
      Serial.println("Humidity uploaded successfully");
    }
    else
    {
      Serial.println(fbdo.errorReason());
    }
  }
}

```

- YourNetworkName ให้กรอกชื่อเครือข่ายอินเทอร์เน็ตที่จะทำการเชื่อมต่อ แนะนำให้ใช้ Hotspot ของโทรศัพท์มือถือ
- YourPassword ให้กรอกรหัสผ่านของเครือข่ายอินเทอร์เน็ตที่จะทำการเชื่อมต่อ
- DatabaseURL ให้นำ Database URL ที่คัดลอกไว้มาใส่ในรูปแบบ
<databaseName>.<region>.firebase.database.app
- DatabaseSecret ให้นำ Secret Key ของ Firebase ที่คัดลอกไว้มาใส่
- เปิดหน้า Serial Monitor
- เปิดหน้า Realtime Database ของ Firebase

ผลการทำงานหลังจากอัปโหลดโค้ดลงบอร์ด

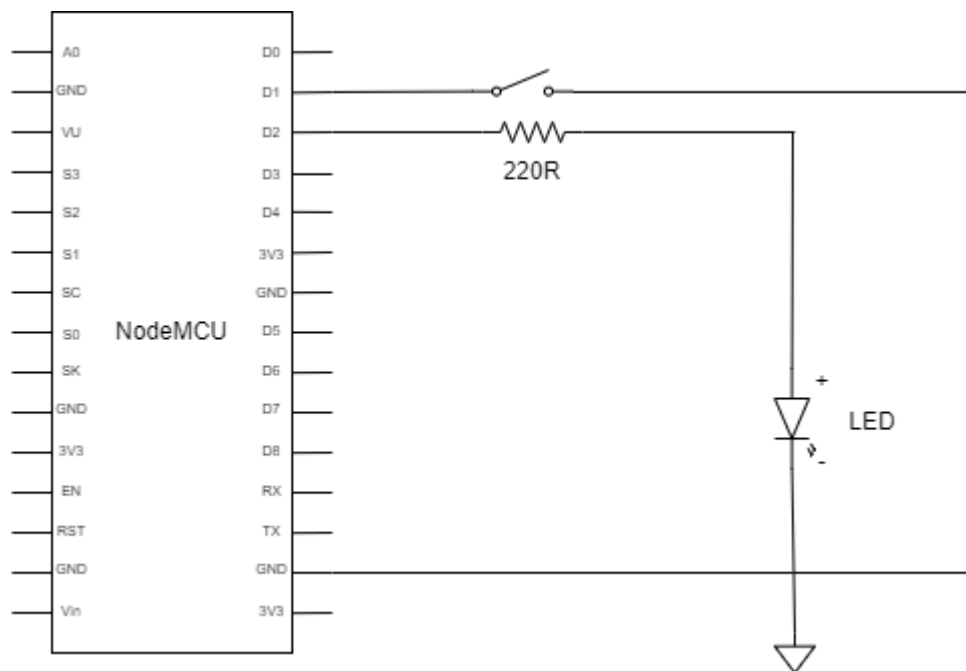
Lab 10.5: การบันทึกสถานะของสวิตช์ลงใน Realtime Database ของ Firebase

อุปกรณ์ที่ต้องใช้

- บอร์ด NodeMCU 1 ตัว
- หลอด LED 1 อัน
- ตัวต้านทานขนาด 220 Ω 1 อัน
- สาย Jumper
- Breadboard 1 อัน
- Tact Switch (สวิตช์กดติดปล่อยดับ) 1 อัน
- Firebase

ขั้นตอนในการทำ

- ต่อดังต่อไปนี้ดังที่แสดงในรูป



รูปที่ 10.42 วงจรสำหรับแลบที่ 10.5

- พิมพ์โค้ดต่อไปนี้นำไปทำการ Verify โค้ด แล้วอัปโหลดโค้ดลงบอร์ด

```
#include <Arduino.h>
#include <ESP8266WiFi.h>
#include <FirebaseESP8266.h>
#include <addons/RTDBHelper.h>

#define WIFI_SSID "YourNetworkName"
#define WIFI_PASSWORD "YourPassword"

#define DATABASE_URL "DatabaseURL"
#define DATABASE_SECRET "DatabaseSecret"

FirebaseData fbdo;
FirebaseAuth auth;
FirebaseConfig config;

unsigned long dataMillis = 0;
int switchState = 0;

#define SWITCH_PIN D1
#define LED_PIN D2

void setup()
{
  Serial.begin(115200);

  pinMode(SWITCH_PIN, INPUT_PULLUP);
  pinMode(LED_PIN, OUTPUT);

  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

  Serial.print("Connecting to Wi-Fi");
  while (WiFi.status() != WL_CONNECTED)
  {
    Serial.print(".");
    delay(300);
  }
  Serial.println();
  Serial.print("Connected with IP: ");
  Serial.println(WiFi.localIP());
  Serial.println();

  Serial.printf("Firebase Client v%s\n\n", FIREBASE_CLIENT_VERSION);

  config.database_url = DATABASE_URL;
  config.signer.tokens.legacy_token = DATABASE_SECRET;

  Firebase.reconnectNetwork(true);
  fbdo.setBSSLBufferSize(4096, 1024);

  Firebase.begin(&config, &auth);
}
```

```
void loop()
{
  int currentSwitchState = digitalRead(SWITCH_PIN);

  if (currentSwitchState != switchState)
  {
    switchState = currentSwitchState;

    String path = "/switchState";
    String stateStr = switchState == HIGH ? "OFF" : "ON";
    Serial.printf("Set switch state to %s... %s\n", stateStr.c_str(), Firebase.setString(fbdo, path, stateStr) ? "ok" : fbdo.errorReason().c_str());

    if (switchState == LOW)
    {
      digitalWrite(LED_PIN, HIGH);
    }
    else
    {
      digitalWrite(LED_PIN, LOW);
    }
  }

  delay(100);
}
```

- YourNetworkName ให้กรอกชื่อเครือข่ายอินเทอร์เน็ตที่จะทำการเชื่อมต่อ แนะนำให้ใช้ Hotspot ของโทรศัพท์มือถือ
- YourPassword ให้กรอกรหัสผ่านของเครือข่ายอินเทอร์เน็ตที่จะทำการเชื่อมต่อ
- DatabaseURL ให้นำ Database URL ที่คัดลอกไว้มาใส่ในรูปแบบ
<databaseName>.<region>.firebase.database.app
- DatabaseSecret ให้นำ Secret Key ของ Firebase ที่คัดลอกไว้มาใส่
- เปิดหน้า Serial Monitor
- เปิดหน้า Realtime Database ของ Firebase

ผลการทำงานหลังจากอัปโหลดโค้ดลงบอร์ด

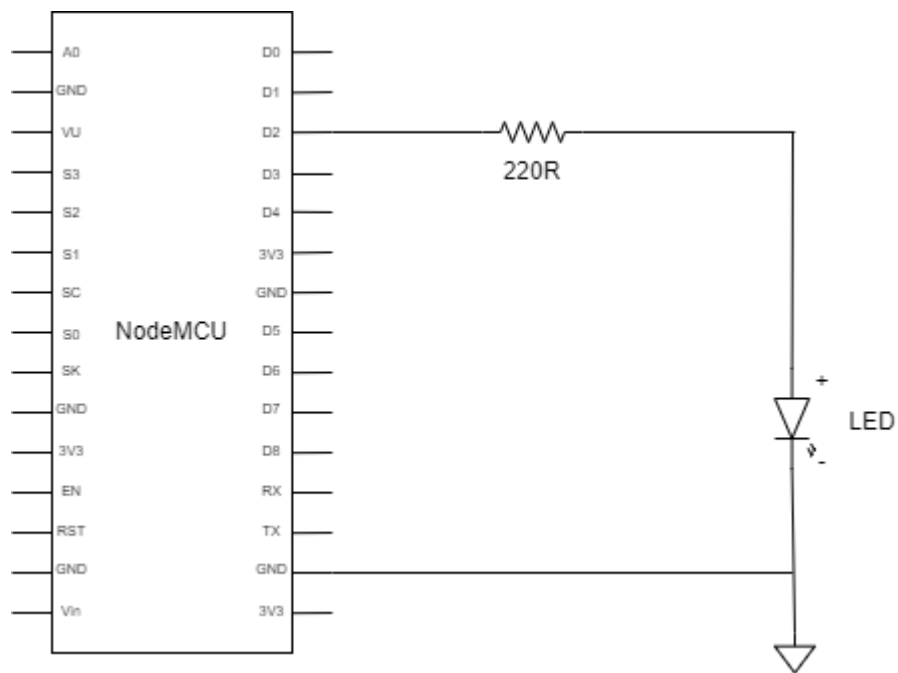
Lab 10.6: การบันทึกสถานะของสวิตช์ลงใน Realtime Database ของ Firebase ร่วมกับการสั่งการบน Blynk

อุปกรณ์ที่ต้องใช้

- บอร์ด NodeMCU 1 ตัว
- หลอด LED 1 อัน
- ตัวต้านทานขนาด 220 Ω 1 อัน
- สาย Jumper
- Breadboard 1 อัน
- Blynk
- Firebase

ขั้นตอนในการทำ

- ต่อดังต่อไปนี้



รูปที่ 10.43 วงจรสำหรับแลบที่ 10.6

- พิมพ์โค้ดต่อไปนี้

```
#define BLYNK_TEMPLATE_ID          "YourTemplateID"
#define BLYNK_TEMPLATE_NAME       "Quickstart Device"
#define BLYNK_AUTH_TOKEN          "YourAuthToken"

#include <Arduino.h>
#include <ESP8266WiFi.h>
#include <BlynkSimpleEsp8266.h>
#include <FirebaseESP8266.h>
#include <addons/RTDBHelper.h>

#define WIFI_SSID "YourNetworkName"
#define WIFI_PASSWORD "YourPassword"

#define DATABASE_URL "DatabaseURL"
#define DATABASE_SECRET "DatabaseSecret"

FirebaseData fbdo;
FirebaseAuth authFirebase;
FirebaseConfig config;

int ledState = LOW;

#define LED_PIN D2

void setup()
{
  Serial.begin(115200);

  pinMode(LED_PIN, OUTPUT);
  digitalWrite(LED_PIN, ledState);

  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
  Serial.print("Connecting to Wi-Fi");
  while (WiFi.status() != WL_CONNECTED)
  {
    Serial.print(".");
    delay(300);
  }
  Serial.println();
  Serial.print("Connected with IP: ");
  Serial.println(WiFi.localIP());

  config.database_url = DATABASE_URL;
  config.signer.tokens.legacy_token = DATABASE_SECRET;
  Firebase.reconnectNetwork(true);
  Firebase.begin(&config, &authFirebase);

  Blynk.begin(BLYNK_AUTH_TOKEN, WIFI_SSID, WIFI_PASSWORD);
}

BLYNK_WRITE(V2)
{
  ledState = param.asInt();

  digitalWrite(LED_PIN, ledState);

  String path = "/ledState";
  String stateStr = ledState == HIGH ? "ON" : "OFF";
  Serial.printf("Set LED state to %s... %s\n", stateStr.c_str(), Firebase.setString(fbdo, path, stateStr) ? "ok" : fbdo.errorReason().c_str());
}

void loop()
{
  Blynk.run();
}
```

- YourNetworkName ให้กรอกชื่อเครือข่ายอินเทอร์เน็ตที่จะทำการเชื่อมต่อ แนะนำให้ใช้ Hotspot ของโทรศัพท์มือถือ
- YourPassword ให้กรอกรหัสผ่านของเครือข่ายอินเทอร์เน็ตที่จะทำการเชื่อมต่อ
- YourAuthToken ให้นำ Token จากการสร้างแอปพลิเคชันใน Blynk มากรอก
- YourTemplateID ให้นำ Template ID จากการสร้างแอปพลิเคชันใน Blynk มากรอก
- DatabaseURL ให้นำ Database URL ที่คัดลอกไว้มาใส่ในรูปแบบ
<databaseName>.<region>.firebase.database.app
- DatabaseSecret ให้นำ Secret Key ของ Firebase ที่คัดลอกไว้มาใส่
- สร้างแอปพลิเคชัน Blynk เช่นเดียวกับเลขที่ 10.2 แล้วนำค่า AuthToken กับ TemplateID ไปใส่ในโค้ด

- ไปที่แท็บ Datastreams ทำการสร้าง Virtual Pin สำหรับ Switch ดังแสดงในรูปที่ 10.44

Virtual Pin Datastream

General
Expose to Automations

NAME
Switch

ALIAS
Switch

PIN
V2

DATA TYPE
Integer

UNITS
None

MIN
0

MAX
1

DEFAULT VALUE
0

Cancel
Save

(ก)

Virtual Pin Datastream

General
Expose to Automations

AUTOMATION TYPE
Switch

Condition

Allow users to use this datastream as an Automation condition

Action

Allow users to change this datastream's state using Automations

Cancel
Save

(ข)

รูปที่ 10.44 การกำหนด Virtual Pin (ก) แท็บ General (ข) แท็บ Expose to Automations

- ที่ Web Dashboard ทำการกำหนดหน้า Dashboard เป็น Switch โดยทำการแก้ไขให้เชื่อมกับ Datastream V2 จากนั้นกด Save And Apply

Switch Settings

TITLE (OPTIONAL)

Switch

Datastream

Switch (V2)

ON VALUE

1

OFF VALUE

0

☐ Show on/off labels

☐ Hide widget name

Switch (V2)

☐

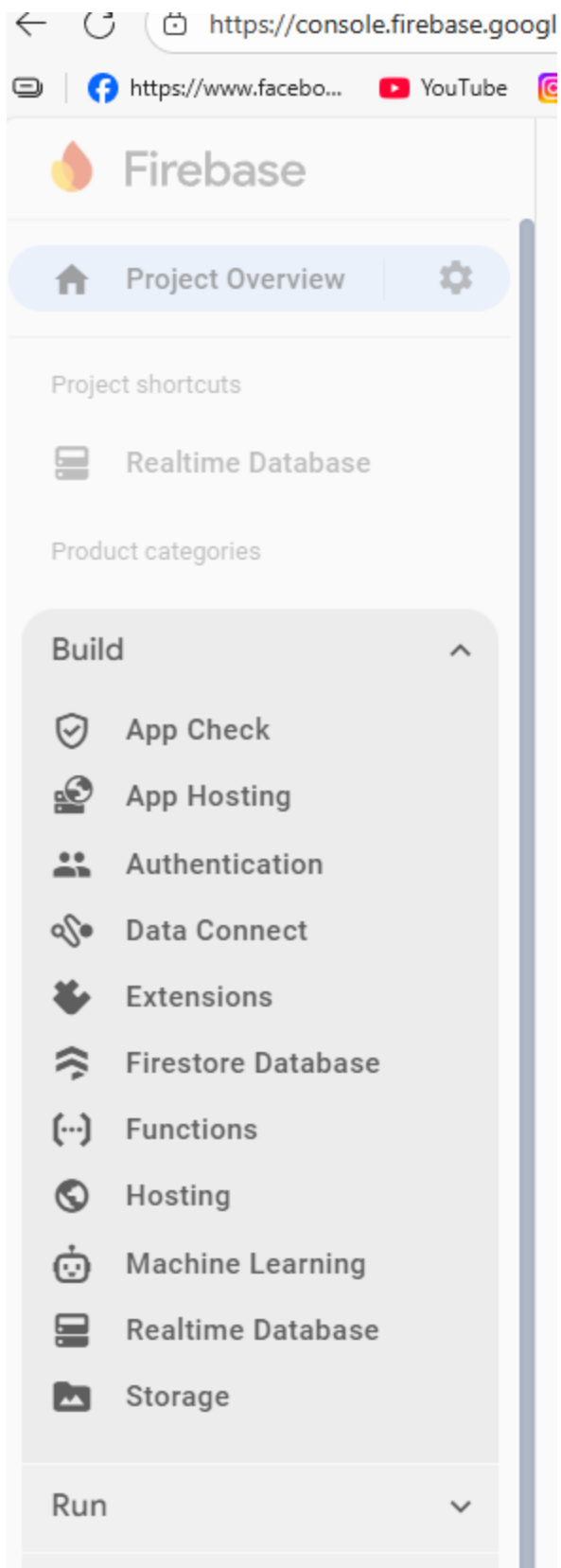
Cancel

Save

รูปที่ 10.45 การตั้งค่าสวิตช์

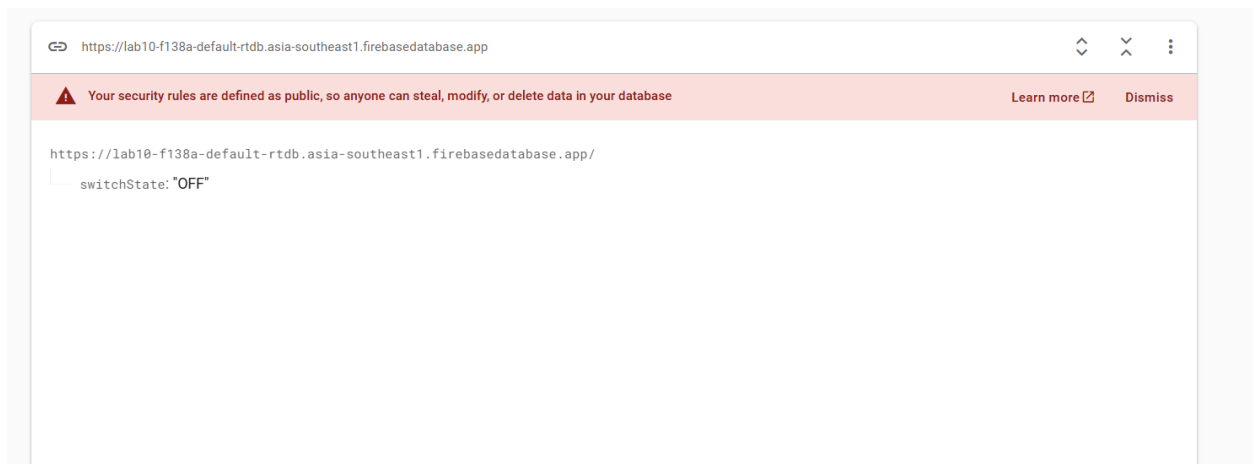
- ทำการ Verify โค้ด แล้วอัปโหลดโค้ดลงบอร์ด
- เปิด Serial Monitor
- เปิด Firebase
- ทดลองเปิดปิดสวิตช์บน Blynk

ผลการทำงานหลังจากอัปโหลดโค้ดลงบอร์ด

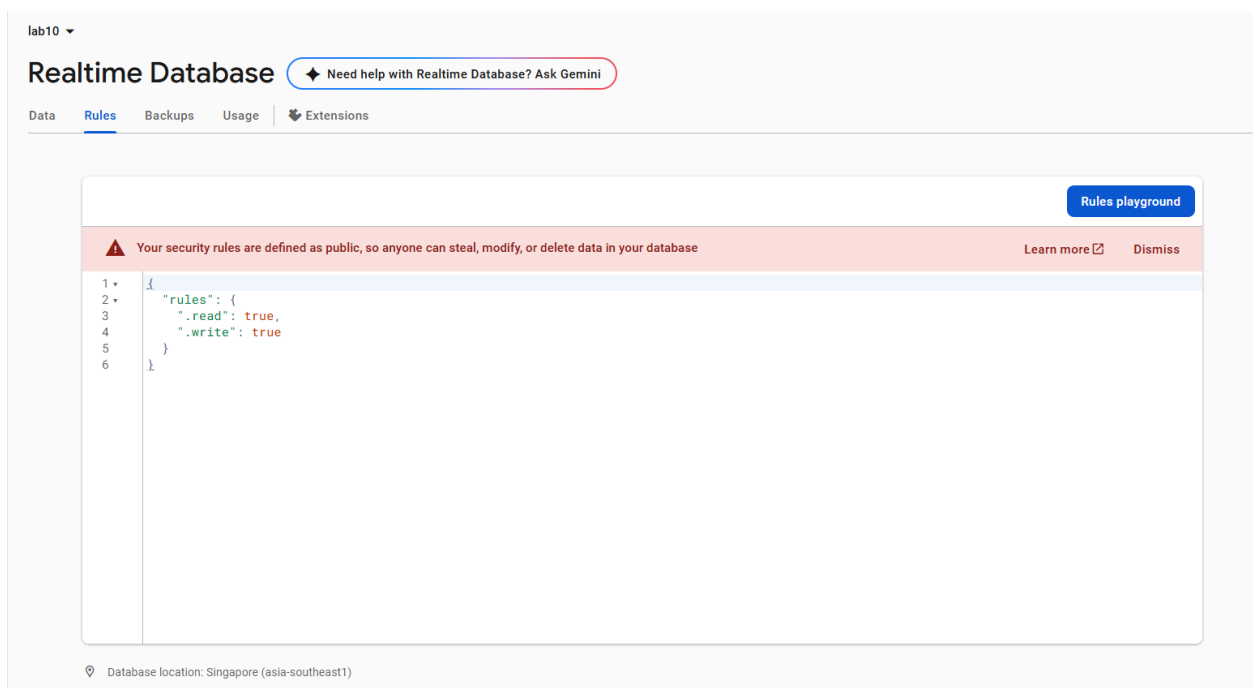


ไปที่ build เลือก Realtime Database

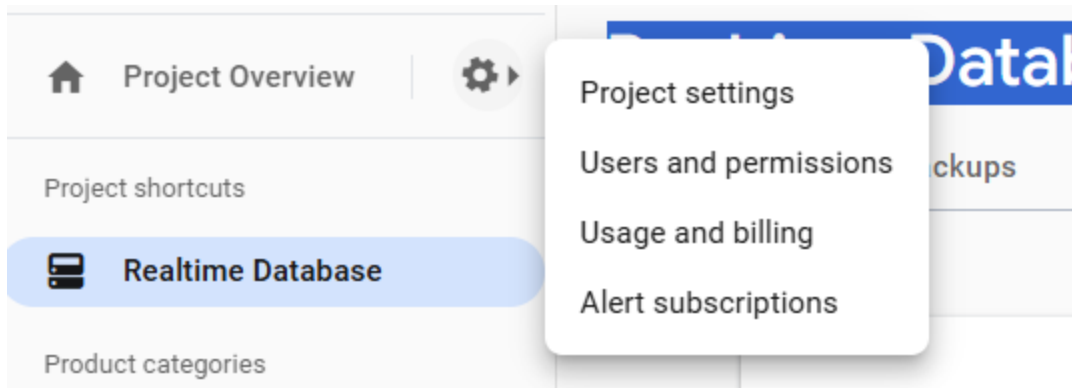
ก็อปลิ้งค์เชื่อมต่อไฟล์เนสไปใส่ ตัวอย่าง <https://lab10-f138a-default-rtdb.asia-southeast1.firebaseio.com/>



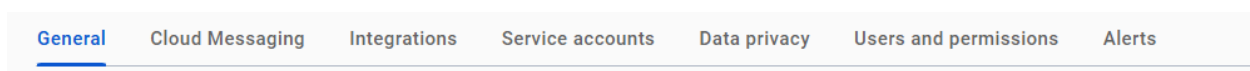
ปรับตามรูป



ไปที่ตั้งค่า



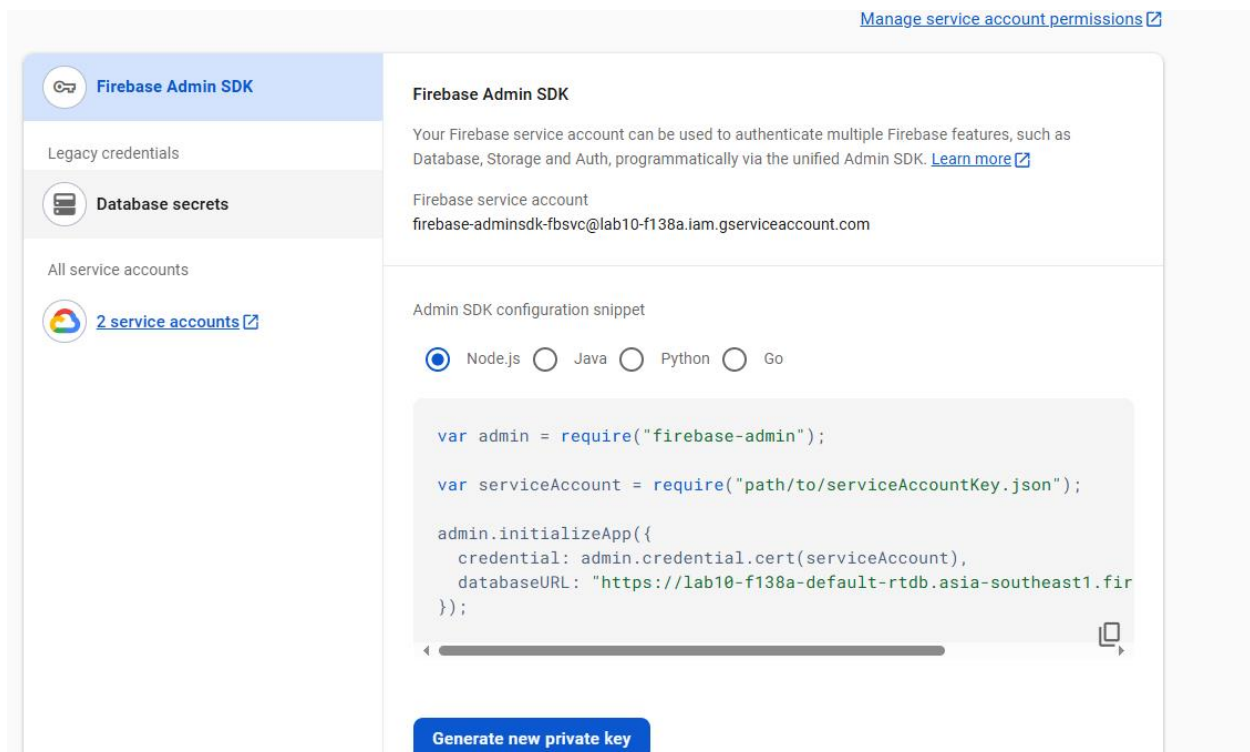
เลือก Project settings



เลือก



ไปที่ Database secrets ถ้ายังไม่มีให้เลือก python และกดสร้าง



ก็อปคีย์ไปใส่

 **Firebase Admin SDK**

Legacy credentials

 **Database secrets**

All service accounts

 [2 service accounts](#) 

Database secrets

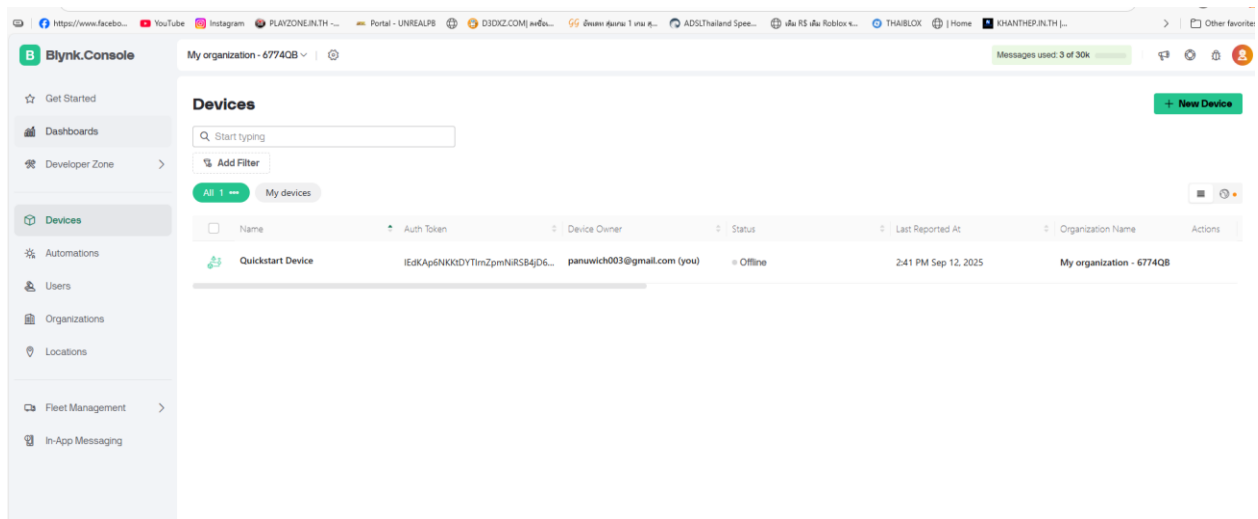
 Database secrets are currently deprecated and use a legacy Firebase token generator. Update your source code with the Firebase Admin SDK.

[Learn more](#) 

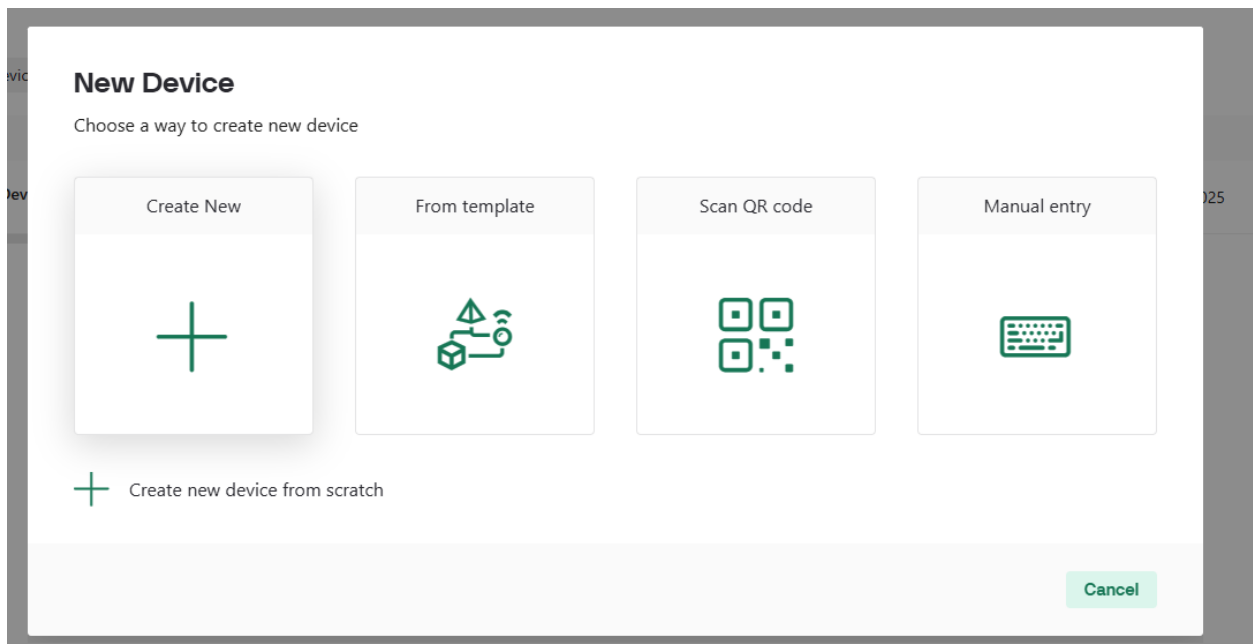
Create custom database authentication tokens using a legacy Firebase token generator. At least one secret must exist at all times. [Learn more](#) 

Add secret

Database	Secrets
lab10-f138a-default-rtdb	0affMvUAuYZKK6X39t2sBk4vuraP9hdg5fwJyA8a



กด **Devices** เลือก **newDevices**



เลือก **Create New Device**

Create New Device

Let's find a best way to connect your device

8266



Arduin



Quickstart

A step-by-step guide to get your first device online in less than 5 minutes.



Arduino MQTT Sample

Sample code to connect Arduino devices to Blynk via MQTT.



Blynk.Edgent Overview

Dynamic authentication and OTA updates with Blynk.Edgent.



Blynk.NCP Overview

Offload connectivity and OTA logic using a Network Co-Processor.



Firmware API (Blynk Library)

Guide to installing Blynk firmware library.

+89

tion

Environments

Quickstart

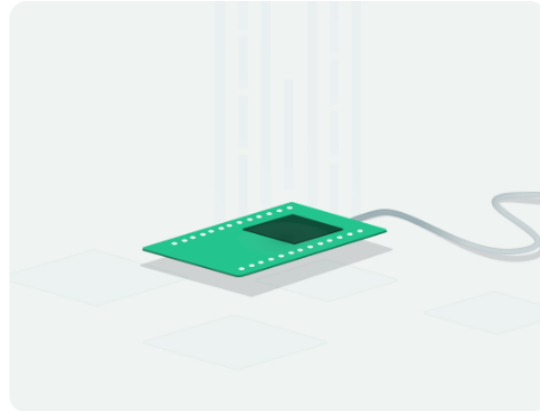
This is a step by step guide to get your first device online and start controlling it from anywhere in the world in **less than 5 minutes**

What you will need:

- Supported hardware. Check the full list of supported hardware [here](#).
- IDE. You can use Arduino IDE or PlatformIO or any other editor.
- Blynk Library
- It will be beneficial if you already know how to upload code to your hardware.

What we will do together:

- Create your first device in Blynk.Cloud
- Prepare firmware code and upload it to your device
- See your device online and control it from Blynk.Console and Blynk.Apps.



Cancel

Let's Go!

กด let's Go

g

Quickstart

1 Hardware — 2 IDE — 3 Blynk Library — 4 Code — 5 Device activation

Which hardware are you using?

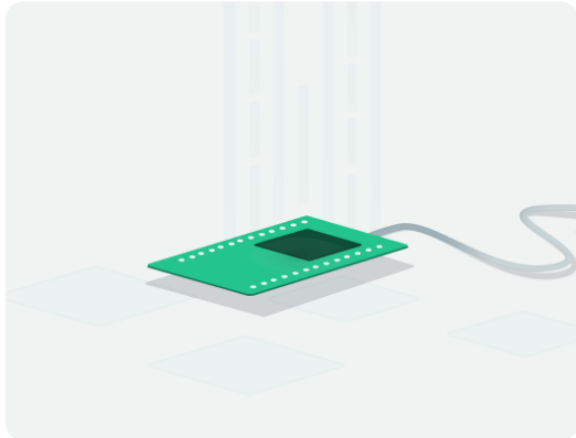
We will help you prepare the code for you board

Choose hardware

ESP32
ESP8266
Arduino
Microduino
Particle
Seeed Wio Terminal
SparkFun Blynk Board
Teensy

Activity type

types (BLE is not supported yet).



Cancel

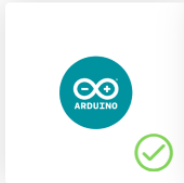
Next →

เลือก 8266 ข้างล่างเลือก WIFI

Quickstart

✓ Hardware — 2 IDE — 3 Blynk Library — 4 Code — 5 Device activation

Which IDE do you use?



Arduino

[Download →](#)



PlatformIO

[Download →](#)



Other



[← Back](#)

[Cancel](#)

[Next →](#)

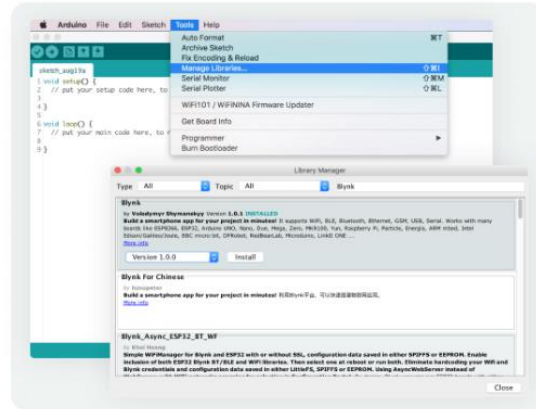
เลือกดูใน

Quickstart

✓ Hardware — ✓ IDE — 3 Blynk Library — 4 Code — 5 Device activation

Install Blynk Library for Arduino

1. Go to Arduino -Tools - Manage Libraries...
2. Search for Blynk there.
3. Choose the latest version and press Install.



← Back

Cancel

Next →

๓๓ next

Quickstart

✓ Hardware — ✓ IDE — ✓ Blynk Library — **4** Code — 5 Device activation

Here is a code for your device

1. Enter your Wi-Fi network SSID (name) and password to connect your device.

* We never store or send this information anywhere. It's only used to generate the firmware code . You can leave these fields empty and manually add WiFi credentials in your sketch.

Wi-Fi Network Name (SSID)

Password

2. Copy code from the right panel (or download it as a file).

3. Create a new sketch in your IDE and paste the code.

4. In IDE check that you are using correct board and port settings

5. Upload sketch to your board.

You can find other examples for your board [here](#).

```

/* *****
  This is a simple demo of sending and receiving some data.
  Be sure to check out other examples!

  *****/

/* Fill-in information from Blynk Device Info here */

#define BLYNK_TEMPLATE_ID           "TMPL6SaTRAjvr"
#define BLYNK_TEMPLATE_NAME         "Quickstart Template"
#define BLYNK_AUTH_TOKEN             "BDRlgjwP286tIeveG66lvkUjTeafFn2L"

/* Comment this out to disable prints and save space */
#define BLYNK_PRINT Serial

#include <ESP8266WiFi.h>
#include <BlynkSimpleEsp8266.h>

// Your WiFi credentials.
// Set password to "" for open networks.
char ssid[] = "";
char pass[] = "";

BlynkTimer timer;

```

[← Back](#)

Cancel

Next →

ใส่wifi และรหัสผ่าน นำโค้ดข้างๆไปแทนที่ในโค้ดเราแล้วอัปและโรมันเชื่อมต่อกัน เสร็จแล้วจะเชื่อมต่อกับบอร์ดและควบคุมได้



Quickstart Device

● Offline

[Panuwich](#) [My organization - 6774QB](#)



 **Edit**

Live

1h

6h

1d

1w

1mo ●

3mo ●

6mo ●

1y ●

⋮ ●

switch

☐

เปิดปิดสวิต จริงๆต้องขึ้น **online** แต่ไม่ได้ทำจ้า